



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

Trabajo Terminal I

“PROTOTIPO DE APLICACIÓN MÓVIL PARA PEDIDOS DE AGUA POTABLE EN GARRAFONES POR REPARTIDORES EN LA ALCALDÍA BENITO JUÁREZ”

No. 2026-B176

Para obtener el título de ingeniero en sistemas
computacionales por opción curricular

Presentan

Márquez Rodríguez Gael Alejandro

Trejo Hernández Brandon

Directores

M. en C. Hurtado Avilés Gabriel

M. en C. Rodríguez Jordán Gabriel de Jesús

D de 2026

INSTITUTO POLITÉCNICO NACIONAL



Contenido

Índice de lustraciones	4
Índice de Tablas.....	4
Introducción	1
Capítulo I. Definición del Proyecto	2
1.1 Planteamiento del problema	2
1.2 Justificación.....	2
1.3 Objetivo General	2
1.4 Objetivos Específicos	2
1.5 Metodología	3
Capitulo II. Marco teórico	5
2.1 Contexto de la distribución de agua potable y logística	5
2.1.1 Problemática en la Alcaldía Benito Juárez.....	5
2.1.2 Distribución de agua potable	5
2.1.3 Logística en repartición	5
2.2 Desarrollo de aplicaciones móviles y geolocalización.....	6
2.2.1 Aplicación Movil.....	6
2.2.2 Kotlin	6
2.2.3 GPS	6
2.2.4 Google Maps	7
2.3 Arquitectura de software y servicios web	7
2.3.1 Java.....	7
2.3.2 Framework	7
2.3.3 Spring	8
2.3.4 Spring Boot	8
2.3.5 API	9
2.3.6 Endpoint.....	9
2.3.7 REST	9
2.4. Gestión y Persistencia de Datos.....	10
2.4.1 Base de datos relacional	10
2.4.2 Normalización de bases de datos	10
2.4.4. Modelo entidad relación	11
2.4.3 MySQL.....	11

2.4.4 MariaDB.....	11
2.5 Algoritmo de búsqueda y heurística	12
2.5.1 Grafos.....	12
2.5.2 Algoritmos de búsqueda	12
2.5.4 Heurística	13
2.5.4 Algoritmo A*.....	13
2.5.5 Heurística Haversine	13
Capítulo III. Estado del arte	13
3.1 Ámbito Tecnológico	13
3.1.1 API Google Maps	13
3.1.2 Open Street Maps	14
3.1.2 Aplicaciones móviles.....	14
3.1.3 Aplicaciones web.....	16
3.1.4 Lenguajes de programación.....	16
3.2 Sistemas existentes.....	17
3.3 Tendencias actuales	18
Capitulo IV. Análisis del Sistema	19
4.1 Encuesta y análisis de datos.....	19
4.2 Requerimientos	21
4.2.1 Requerimientos funcionales.....	22
4.2.2 Requerimientos no funcionales.....	24
4.3 Reglas de negocio.....	25
4.4 Análisis de tecnologías	28
4.4.1 Tecnologías para el Front-End.....	29
4.4.2 Tecnologías para el Back-End	30
4.4.3 Tecnologías para la base de datos	31
4.4.4 Tecnologías para la aplicación móvil.....	32
4.4.5 API de visualización del mapa.....	33
4.5 Análisis de riesgos.....	34
4.5.1 Factibilidad técnica.....	34
4.5.2 Factibilidad económica.....	35
4.5.3 Factibilidad operativa	37
4.5.4 Factibilidad temporal.....	37

4.6 Arquitectura.....	39
4.6.1 Diagrama de contexto.....	39
4.6.2 Diagrama de Contenedores (Nivel 2)	42
4.6.3 Diagrama de Componentes (Nivel 3)	43
4.7 Casos de uso.....	48
4.8 Diagrama de actividad.....	83
4.8.3 Diagrama de actividad para el usuario (cliente).....	83
4.8.4 Diagrama de actividad para el usuario (repartidor).....	84
4.8.5 Diagrama de actividad para el administrador.....	85
Referencias.....	93
Anexos	103

Índice de ilustraciones

<i>Ilustración 1</i> Modelado de prototipos de características selectas tomado de [8].....	3
<i>Ilustración 2</i> Metodología del proyecto. <i>Elaboración propia.</i>	4
<i>Ilustración 3</i> Grafo ponderado tomado de [123].....	12
<i>Ilustración 4</i> Gráfica de la encuesta ENDUTIH realizada por el INEGI [59].....	15
<i>Ilustración 5</i> Crecimiento de las instalaciones en México 2023.....	16
<i>Ilustración 6</i> Arquitectura general del sistema, montada en Microsoft Azure. <i>Elaboración propia.</i>	40
<i>Ilustración 8</i> Diagrama de arquitectura en bloques.....	41
<i>Ilustración 10</i> Diagrama de Contexto (C4 – Nivel 1). <i>Elaboración propia.</i>	42
<i>Ilustración 11</i> Diagrama de Contenedores (C4 – Nivel 2). <i>Elaboración propia.</i>	43
<i>Ilustración 12</i> Diagrama de componentes del Backend Spring Boot (C4 – Nivel 3). <i>Elaboración propia</i>	45
<i>Ilustración 13</i> Diagrama de componentes de la Aplicación Móvil (C4 – Nivel 3). <i>Elaboración propia.</i>	46
<i>Ilustración 14</i> Diagrama de componentes de la Base de Datos (C4 – Nivel 3). <i>Elaboración propia</i>	47
<i>Ilustración 15</i> Diagrama de actividades Cliente. <i>Elaboración propia.</i>	83
<i>Ilustración 16</i> Diagrama de actividad del repartidor. <i>Elaboración propia.</i>	84
<i>Ilustración 17</i> Diagrama de actividad orientado al administrador. <i>Elaboración propia.</i>	85

Índice de Tablas

<i>Tabla 1</i> Comparación de aplicaciones de reparto.....	17
<i>Tabla 2.</i> Requerimientos funcionales. <i>Elaboración propia.</i>	22
<i>Tabla 3.</i> Requerimientos no funcionales. <i>Elaboración propia.</i>	24
<i>Tabla 4.</i> Reglas de negocio. <i>Elaboración propia</i>	25
<i>Tabla 5.</i> Tecnologías para el Front-End	29
<i>Tabla 6.</i> Análisis de factibilidad	35
<i>Tabla 7</i> Factibilidad económica Operativa.....	35
<i>Tabla 8</i> Factibilidad económica Tecnológica HW.....	36
<i>Tabla 9</i> Factibilidad económica Tecnológica SW	36
<i>Tabla 10</i> Factibilidad económica Tecnológica Servicios	36

Tabla 11 Cronograma Márquez Rodríguez Gael Alejandro37
Tabla 12 Cronograma Trejo Hernández Brandon38

Introducción

En la actualidad, el acceso eficiente al agua potable representa uno de los desafíos logísticos más importantes, aún más en zonas urbanas como lo es la Ciudad de México, donde el consumo de agua embotellada en garrafones constituye el 72.10% del gasto anual de los hogares [1]. Ante este escenario, la integración de sistemas informáticos y el auge de las aplicaciones móviles se han convertido en herramientas primordiales. El despliegue de arquitecturas de software orientadas a la movilidad y logística permite que las organizaciones alcancen la agilidad para responder rápidamente a las peticiones de los usuarios, así como ampliar su alcance mediante el uso de sistemas informáticos y procesamiento de datos actualizados [2]. En la industria de la logística y el transporte, las aplicaciones móviles han surgido como herramientas clave para integrar procesos y optimizar la cadena de suministro, acelerando la toma de decisiones y reduciendo costos operativos mediante la automatización de flujos de información [3]. Al aprovechar tecnologías de geolocalización y sincronización en la nube, los sistemas de información no solo automatizan tareas administrativas, como puede ser el cálculo de las entregas en un solo día, sino que proporcionan una infraestructura que maximiza la eficiencia global de los recursos y garantiza la trazabilidad del servicio [4].

Sin embargo, la implementación de las tecnologías por sí solas no garantizan la adopción del usuario. Una de las barreras más importantes a las que se enfrentan las aplicaciones móviles es la confianza de los usuarios, el cual es un factor que provoca que los usuarios abandonen plataformas nuevas que no ofrecen una experiencia protegida o transparente [5]. En el contexto del reparto a domicilio, la incertidumbre sobre la ubicación del repartidor y la falta de validación de los datos generan una experiencia negativa. Por ello, la integración de sistemas de seguridad informática y mecanismos de trazabilidad del pedido en tiempo real no son solo añadidos técnicos, sino requisitos fundamentales para garantizar una experiencia de usuario satisfactoria [6].

En este sentido, el presente Trabajo Terminal propone el desarrollo de HToGo, un prototipo de aplicación móvil diseñado para optimizar la distribución de agua potable en la Alcaldía Benito Juárez. A través del uso de la API de Google Maps y una arquitectura basada en servicios en la nube, el sistema busca centralizar la información logística para ofrecer un canal directo y seguro entre repartidor y consumidor. En las siguientes secciones, se detallará el origen de la problemática, los objetivos técnicos del proyecto, así como una idea general sobre la implementación propuesta.

Capítulo I. Definición del Proyecto

1.1 Planteamiento del problema

Debido a una contaminación en el año 2024 específicamente en el pozo Alfonso XIII, se contaminó el agua de la alcaldía Benito Juárez con hidrocarburos [7]. Este suceso produjo muchos problemas para tener acceso al agua en la alcaldía. Con este suceso, la población de la alcaldía no tenía acceso a agua potable en sus casas, por lo que necesitaban comprar muchos garrafones para evitar el uso del agua contaminada. Esto causó un desabasto de este producto y observamos la deficiencia de logística para entrega de garrafones. Además, los repartidores de garrafones suelen tener a sus clientes ya establecidos, y es complicado encontrar un dato de contacto, haciendo que el acceso al agua sea complicado. Por lo tanto, hemos observado esa necesidad y llegamos a la conclusión de realizar mejor una aplicación que facilite la entrega de agua potable por medio de garrafones. Este proyecto ayudaría a que se tenga un acceso al agua en la alcaldía, así como un mejor control para los repartidores de agua sobre sus entregas, clientes y rutas, facilitando la distribución de esta.

1.2 Justificación

Esta aplicación plantea una solución a la distribución de garrafones para las personas ubicadas en la alcaldía Benito Juárez, basándonos en la experiencia que se tuvo hace un año con el problema del agua contaminada, en la cual se pudo observar la deficiencia en la compra de estos garrafones ya que no se cuenta con información de los repartidores, no se tiene un control sobre los pedidos, causando un desabasto en las tiendas y un descontento de la población que reside en dicha alcaldía, que a día de hoy sigue preocupada por este incidente. Nuestro prototipo de aplicación sería una solución para los vendedores de garrafones en la alcaldía, mejorando sus rutas y, a su vez, ayudando al usuario por medio de notificaciones cuando se acerque a su ubicación, logrando un mejor entendimiento entre repartidor y usuario. Este proyecto estaría mejorando la logística para vendedores individuales o empresas pequeñas de garrafones, apoyando todo este sector, mientras que el usuario final de la aplicación se vería beneficiado por tener un control de pedidos y un contacto con el repartidor de garrafones. Al implementar la API de Google Maps, trabajaremos el manejo de peticiones e implementaremos esta funcionalidad directamente con la aplicación móvil, además teniendo acceso al GPS del usuario y repartidor podremos obtener la ubicación en tiempo real, contando además con notificaciones para la entrega. Esto incrementaría la satisfacción del usuario y la eficiencia del vendedor.

1.3 Objetivo General

Desarrollar un prototipo de aplicación móvil que permita la distribución de agua potable en la alcaldía Benito Juárez, proporcionando la ruta del repartidor, la ubicación y ofreciendo la opción de recibir notificaciones cuando el repartidor esté cerca de su domicilio.

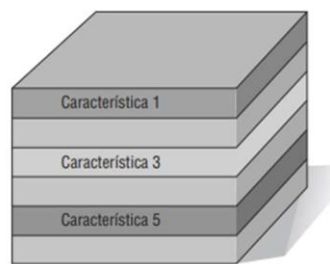
1.4 Objetivos Específicos

- Crear una base de datos para almacenar información del repartidor, estatus del pedido e información del usuario.

- Diseñar una interfaz intuitiva de la aplicación móvil con un enfoque en la asignación de pedidos.
- Proporcionar al repartidor una ruta para realizar la entrega del pedido, obtenida por medio de la conexión a la API de Google Maps.
- Implementar un sistema de notificaciones que permita a los usuarios estar pendientes de su pedido.
- Implementar medidas de seguridad que prevengan pedidos falsos.

1.5 Metodología

Para realizar este prototipo se optó por elegir la metodología “Prototipo de características selectas” [8] ya que tiene flexibilidad y adaptación a los cambios, permitiéndonos desarrollar las funcionalidades como módulos, para que nos permita ver el progreso de la aplicación conforme se vaya realizando, probando unitariamente las funcionalidades y recolectando retroalimentación. Este tipo de prototipos se ve representado por el siguiente diagrama:



Prototipo de características selectas

Ilustración 1 Modelado de prototipos de características selectas tomado de [8]

En el cual se observa las diferentes capas que se tiene al desarrollar por módulos. Nosotros seguiremos la metodología por las siguientes fases:

- 1.- Investigación preliminar: En esta primera sección se llevará a cabo una investigación, la cual tendrá como objetivo un entendimiento profundo del problema, haciendo un estudio del estado del aparte para tener más conocimiento sobre los antecedentes de este mismo. Aquí mismo buscaremos usuarios potenciales del sistema.
- 2.- Análisis de requerimientos: Así como la elaboración de los requerimientos y requisitos del sistema, esto será flexible ya que conforme se vayan desarrollando las características se irá adecuando, dependiendo de la retroalimentación, mejorando la funcionalidad de los requerimientos del sistema para tener un sistema factible.
- 3.- Diseño y construcción de las características selectas: Una vez contemos con nuestros requerimientos del sistema desarrollaremos el diseño de nuestro sistema. Terminado el diseño, empezaremos por las características esenciales para el programa, buscando una mayor organización y poder lograr los objetivos deseados sabiendo asignar los tiempos para cada una de las actividades.

4.- Construcción del prototipo funcional: Cuando contemos con nuestros principales módulos, empezaremos a construir el sistema completo, unificando los módulos y creando el sistema como un solo ecosistema.

5.- Pruebas unitarias y de integración: En esta fase, probaremos el prototipo y módulos que desarrollamos anteriormente, haciendo pruebas unitarias para evaluar cada módulo. Una vez las funcionalidades pasen este filtro, se probará la aplicación en conjunto verificando que cada componente que integra este mismo cumpla con su propósito.

6.- Despliegue del sistema: Una vez que hayamos concluido con las pruebas de nuestro sistema, contaremos con la documentación completa de nuestro sistema en conjunto con un manual de usuario. En esta fase, nuestro sistema estará listo para que aquellos usuarios que buscamos en la investigación preliminar puedan probar en tiempo real el sistema y observemos nuestro prototipo funcionando en un ambiente real.

El siguiente diagrama representa la metodología a seguir:

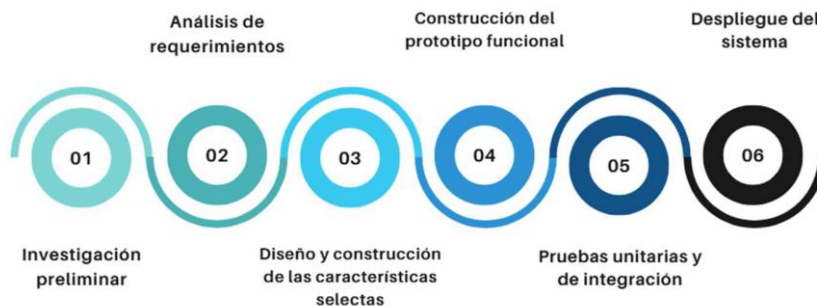


Ilustración 2 Metodología del proyecto. Elaboración propia.

Capítulo II. Marco teórico

2.1 Contexto de la distribución de agua potable y logística

2.1.1 Problemática en la Alcaldía Benito Juárez

Durante abril del 2024 se detectó la presencia de aceites y algunos otros químicos en el suministro de agua que abastece a las colonias Del Valle, Nápoles y San Juan siendo estas dañinas para los colonos e incluso para algunos animales

En 2025 se tuvo otra ruptura en el suministro llevando a varios perros a enfermarse y morir por consumir el líquido que se encontraba en sus tanques de agua, generando inconformidad por los habitantes de la colonia, y llegando a exhibir muestras del agua contaminada a medios de comunicación para que esa situación fuera difundida [9].

Con esto se aumentó la demanda de garrafones, ya que de igual forma medios de comunicación indicaron que algunos repartidores de agua tuvieron un aumento en su repartición de agua superando la capacidad de rutas establecida e incluso algunos lucraron con el problema rellenando los mismos garrafones con agua contaminada [10]

2.1.2 Distribución de agua potable

Históricamente, la Ciudad de México ha dependido de sistemas externos para el suministro de agua. El "Aguador" Colonial: En los siglos XVIII y XIX, debido a la falta de tuberías domiciliarias, surgió el oficio del aguador.

En donde estas personas transportaban agua en cántaros de barro desde las fuentes públicas hasta las casas. Estos envases donde depositaban el agua eran pequeños jarros los cuales solo servían para cosas indispensables en la casa, en algunas casas más grande o con mayores recursos llegaban a usar contenedores más grandes llamados “chochocol” [11]

Durante los años de 1980, debido a que se tuvo un crecimiento urbano y por lo mismo aumento la contaminación, se cambió la manera en la que se repartía el agua, donde se incluyó el modelo de garrafón de vidrio, pero al ser este muy difícil de transportar se optó por usar el plástico ya que en el temblor del 85 dañó la red hidráulica generando desconfianza en el grifo [12].

2.1.3 Logística en repartición

En la logística moderna tenemos muchos algoritmos para reducir los tiempos de entrega, donde las aplicaciones web o móviles implementan rutas dinámicas que permiten a los repartidores darles la ruta óptima en tiempo real para que tengan una mayor organización en sus entregas [13].

Pero no solo se tiene la entrega ya que en la distribución se tiene un proceso donde se almacena el producto, se procesa, posteriormente se prepara, se asigna el medio de transporte para que finalmente en la fase de la entrega este producto o servicio llegue a manos del cliente para que se pueda cumplir con la necesidad [14].

2.2 Desarrollo de aplicaciones móviles y geolocalización

2.2.1 Aplicación Movil

Una aplicación móvil con el paso de los años se ha vuelto algo fundamental en el día a día de las personas ya que al año se descargan más de 200,000 millones de aplicaciones, ya que los móviles o smartphones son algo que usamos en nuestras vidas cotidianas desde elegir donde vamos a comer, obtener indicaciones de cómo llegar a algún lugar o solo para mantenernos comunicados con nuestros familiares o amigos.

Actualmente tenemos muchas herramientas para el desarrollo de aplicaciones móviles, los desarrolladores pueden usar matrices de plantillas, así como herramientas visuales, ya que con eso se facilita mucho la creación de aplicaciones e incluso están nos ayudan a la incorporación de tecnologías más complejas como el uso de inteligencia artificial.

Existen 2 tipos de aplicaciones, de lienzo y controladas por modelos, las de lienzo proporcionan una interfaz intuitiva y fácil de usar que permite a los empleados crear interfaces personalizadas teniendo así un gran control en la parte del diseño, por otro lado, tenemos las de controladas por modelos, estas están dirigidas a trabajar con datos o con procesos más complejos para visualizar datos [15].

2.2.2 Kotlin

Kotlin es un lenguaje de programación diseñado para la plataforma Java, el cual se distingue por ser conciso y seguro. Su objetivo principal es ofrecer una alternativa más productiva y eficiente en todos los contextos donde Java se utiliza actualmente, popularmente utilizado para el desarrollo de aplicaciones para Android hasta sistemas robustos en el lado del servidor, permitiendo trabajar sin problemas con todas las bibliotecas y marcos de trabajo existentes de Java con el mismo nivel de rendimiento. [16]

Kotlin es administrado por la Kotlin Foundation, un grupo co-creado por Google, lo cual convierte a este lenguaje de programación en una opción compatible para el desarrollo oficial de Android, de manera que las herramientas de Android fueron diseñadas para admitir Kotlin de manera nativa. [17]

La ventaja de utilizar Kotlin y Java en un mismo proyecto, se basa en que Kotlin fue diseñado con la funcionalidad para llamar código nativo de Java para su uso, lo cual permite una mejor compatibilidad. [18]

2.2.3 GPS

Es un sistema de radionavegación que ayuda a conocer la ubicación de un objeto, vehículo o persona mediante el uso de satélites, las siglas en español significan “Sistema de Posicionamiento Global” proporcionando los servicios a todos los usuarios, El uso y alcance de esta tecnología dependen de la compañía que nos ofrece este servicio, el principal motivo por el cual se creó GPS fue para la navegación aérea y marítima, pero en la actualidad es más usado en tierra, siendo más específicos en cómo funciona, se usan satélites que transmiten señales a los dispositivos que luego estos interpretan las señales para determinar la ubicación, ya que nos dan lugar con latitud y longitud, fecha del momento en el que se encontró el objeto y la hora [19].

2.2.4 Google Maps

Google Maps fue publicado el 5 de febrero de 2005 bajo el nombre de “Google local”, el cual solo mostraba un mapa de Estados Unidos permitiendo hacer zoom y arrastrar el mapa para mejorar la experiencia del usuario por una más cómoda. Desde su salida ya contaba con funciones como vista satelital y algunos servicios de transporte público.

En 2006 se expandió a otras regiones como Asia y Sudamérica y de la misma forma se incorporó la función de tráfico pero de igual forma primero se probó en Estados Unidos, en 2007 se implementó la parte del Street view que nos ayuda a tener una mejor referencia de los lugares a los que se quería visitar.

En 2009 se actualizó la interfaz para que fuera más intuitiva con funcionalidades de Google Earth, al igual se actualizó su paleta de colores para diferenciar entre bosques, lagos, carreteras, etc

En 2016 google Maps ya es un poco como lo conocemos actualmente donde cuenta con una interfaz muy intuitiva y con varios atajos que nos sirven para buscar direcciones, restaurantes o calles con una mayor fluidez, a su vez las imágenes y render fueron hechos con mejor calidad e incluso más nítidos, agregando funciones que nos dejan apreciar la tierra o luna como globo terráqueo [20].

2.3 Arquitectura de software y servicios web

2.3.1 Java

Java es un lenguaje de programación de propósito general, orientado a objetos y diseñado para ser concurrente y robusto. Su arquitectura multiplataforma permite que sea utilizado desde aplicaciones empresariales, aplicaciones móviles o tecnologías de servidor [21]. El funcionamiento de Java se basa en un modelo de compilación híbrido, donde el código fuente se traduce inicialmente a un formato intermedio denominado “bytecode”. Este código de bytes es ejecutado por la Máquina Virtual de Java (JVM), actuando como una capa de abstracción sobre el hardware y el sistema operativo [22]. Esta característica garantiza la portabilidad del sistema, permitiendo que el backend de una aplicación, como el propuesto en este proyecto, se adapte a la infraestructura del servidor siempre que se cuente con el Java Development Kit (JDK) instalado en el sistema operativo.

2.3.2 Framework

Un framework es un esquema o marco de trabajo el cual nos ofrece una estructura base para elaborar proyectos con objetivos específicos, tiene una característica de plantilla que sirve como punto de partida para la organización y desarrollo de software dentro del proyecto.

Esto nos ayuda mucho a simplificar una tarea porque nos permite acelerar el trabajo y favorecer que este reduzca errores, sea colaborativo y obtener un resultado de mejor calidad. Mediante el uso de frameworks, nos permite principalmente agilizar procesos de desarrollo porque podemos reutilizar herramientas o módulos en otras palabras ya se tiene el “esqueleto” sobre el que se trabajara. El hecho de desarrollar una aplicación fácilmente te sirve para tener una mejor organización y control de todo el código elaborado, pudiendo usarlo nuevamente en el futuro [23].

Este es un esquema de reutilización del software conformado por componentes y relaciones entre estos, por ejemplo: la abstracción de clases, objetos o componentes que la conforman; además,

contiene diferentes componentes de conexión a base de datos, como controladores para conexión directa (MySQL, SQL Server, Oracle), o mediante el estándar ODBC (Open DataBase Connectivity).

Existen varios tipos de Frameworks, usualmente los enfocados en el desarrollo Web, ofrecen una capa de controladores de acuerdo con el patrón MVC (Model View Controller) facilitando así la integración con otras herramientas para la implementación de las capas de negocio y presentación, donde: Model nos sirve en la parte de la lógica, View en el apartado visual, lo que ve el usuario como HTML, CSS etc, y final mente, el Controller, el cual funciona como intermediario [24].

2.3.3 Spring

Spring fue lanzado en 2003 siendo una plataforma de código abierto llamado “Apache 2.0”, Convirtiéndose desde entonces en el framework más popular para Java empresarial, para crear código de alto rendimiento, liviano y reutilizable. Ya que su finalidad es estandarizar, agilizar, manejar y resolver los problemas que puedan ir surgiendo en el trayecto de la programación. Spring se puede considerar como el padre de los frameworks Java, ya que da soporte a varios frameworks como: Hibernate, Struts, Tapestry, etc.

Spring está organizado de forma modular. A pesar de la cantidad de paquetes y clases que tiene, solo debemos ocuparnos de aquellos que necesitemos e ignorar el resto. Probar una aplicación escrita con Spring es un proceso simple, porque el código dependiente del entorno se traslada a este framework. Además, mediante el uso de JavaBeanstyle, se vuelve más fácil utilizar la inyección de dependencia para hacer pruebas, para ello podemos hacer uso de datos dummies o mocks, para ver las respuestas. Todo esto gracias a que Spring cuenta con suficiente documentación en la web y con una gran comunidad. [25]

Spring Framework proporciona un modelo integral de programación y configuración para aplicaciones empresariales modernas basadas en Java, en cualquier tipo de plataforma de implementación.

Un elemento clave de Spring es el soporte de infraestructura a nivel de aplicación: Spring se centra en la "plomera" de las aplicaciones empresariales para que los equipos puedan centrarse en la lógica de negocios a nivel de aplicación, sin vínculos innecesarios con entornos de implementación específicos [26].

2.3.4 Spring Boot

Spring boot es un framework el cual representa un cambio de paradigma en el ecosistema de Spring, diseñado específicamente para reducir la fricción en el desarrollo de aplicaciones mediante la eliminación de la configuración tediosa y repetitiva. Su propósito fundamental es orientar la programación a las funcionalidades del software, en lugar de dedicar tiempo a archivos de configuración. Esta agilidad se fundamenta en pilares como la configuración automática, la cual se encarga de las necesidades del sistema según las librerías detectadas, y el uso de dependencias "starter" que simplifican la gestión del proyecto al agrupar componentes esenciales bajo una sola referencia técnica. [27]

Además, reduce la necesidad de configuración con XML y facilita la seguridad e integridad del sistema, así como el uso de servidores embebidos. [28]

2.3.5 API

Son 2 componentes de software que sirven para tener una comunicación entre sí mediante protocolos, la arquitectura de las API se explica como cliente y servidor, la aplicación que envía la solicitud se llama cliente y la que envía la respuesta es servidor. [29]

Las API fueron desarrolladas para facilitar el trabajo de los desarrolladores ahorrándoles tiempo y dinero, ya que así no tendrían que crear sistemas desde 0, o incluso si solo se necesita usar un servicio es más factible para que sea más atractivo para los usuarios ya que algunos servicios tienen sus reputaciones y eso hace los usuarios tengan más confianza [30].

2.3.6 Endpoint

Un endpoint representa un punto de interacción específico a un recurso dentro de una API bajo el estilo REST. Este punto se identifica mediante una URI (Uniform Resource Identifier) y utiliza los métodos del protocolo HTTP para definir operaciones sobre los datos. Los endpoints funcionan como un contrato que oculta la complejidad del servidor, permitiendo que el cliente interactúe con los recursos sin conocer la implementación con la base de datos o la lógica de negocio. [31]

Funciona como un localizador único que dirige la solicitud hacia una función específica del servicio, garantizando la comunicación entre el frontend y el backend [16].

2.3.7 REST

REST (Representational State Transfer) fue propuesto originalmente por Roy Fielding en su tesis doctoral llamada “Architectural Styles and the Design of Network- based Software Architectures” [32] En ella se propone a REST como un estilo arquitectónico para sistemas distribuidos hipermedia. Se define como un conjunto de reglas o "contratos" que permiten que dos sistemas, por ejemplo, una aplicación móvil y un servidor en la nube, se comuniquen de forma sencilla utilizando el protocolo HTTP. En lugar de ver a la aplicación como un conjunto de funciones aisladas, REST propone ver todo como recursos [33](por ejemplo: un "Pedido", un "Repartidor" o un "Garrafón"). Cuando la aplicación solicita un recurso, el servidor no envía el objeto real, sino una representación de su estado actual, generalmente en formato JSON, permitiendo que ambos sistemas interactúen sin necesidad de conocer los detalles internos del otro. [34]

Para proyectos que utilizan diferentes tecnologías esto es de gran importancia, ya que es indiferente si el backend está programado en Spring Boot y la app en Kotlin; mientras ambos respeten las restricciones de REST (uso de métodos estándar: GET, POST, PUT, DELETE), la comunicación será fluida y escalable. Estos métodos se basan en operaciones CRUD (Create Read Update Delete), los cuales son los encargados de referirse a operaciones y mantenimiento de datos. [35]

2.3.7.1 GET

Este método se encarga de la obtención de datos de una entidad, sea una tabla o un objeto en concreto. Es el método más sencillo, ya que solamente es una lectura de nuestro sistema por medio de una URL en específico, por ejemplo, /rest/pedidos/23424. [35] Al ser una lectura, no necesita envío de información (body), simplemente la lectura de la respuesta. [36]

Comentado [GM1]: Es la fuente 41, está tomando la 37

2.3.7.2 PUT

El método PUT se define como una funcionalidad de "reemplazo" o "actualización completa" de un recurso ya existente en una ubicación específica (URI). [35] A diferencia de POST, PUT es un método idempotente; esto garantiza que realizar la misma petición varias veces tendrá el mismo efecto que realizarla una sola vez, lo cual es crucial para la estabilidad del sistema ante fallos de red. Este método siempre necesita envío de información para consumirlo. [37] Por ejemplo, cambiar de "pendiente" a "en camino", lo cual asegura que el estado del recurso sea siempre consistente.

2.3.7.3 DELETE

El método DELETE se utiliza para borrar una entidad, es importante involucrar un borrado en cascada en todas las entidades hijas. De la misma forma, si borramos una colección se deben borrar todas las entidades que pertenezcan a ella. [35] Este método no requiere un envío de información, ni necesariamente necesita información de regreso, ya que basta con una respuesta exitosa del servicio. [38]

2.3.7.4 POST

El método POST se utiliza principalmente para la creación de nuevos recursos dentro de una cierta entidad en el servidor. A diferencia del método PUT, este método no es idempotente, lo que significa que múltiples peticiones idénticas darán como resultado la creación de múltiples recursos (por ejemplo, generar varios pedidos si el usuario presiona el botón repetidamente). [35] Por ejemplo, en este proyecto se podría aplicar a la acción en la que el servidor asigna un identificador único y gestiona el almacenamiento del registro inicial. Este método necesita, de igual manera que el método PUT, un envío de información, en cualquier caso [39].

2.4. Gestión y Persistencia de Datos

2.4.1 Base de datos relacional

Es una base que almacena y organiza puntos de datos con relaciones previamente definidas en tablas para poder tener un acceso veloz a estas, esto gracias a la estructuración de los datos mediante filas y columnas, siendo esta una de las más utilizadas por su flexibilidad y eficacia al momento de usarlas.

Estas bases son usadas para comprender el lenguaje de consulta estructurado, esto para almacenar, manipular y recuperar datos, existe un lenguaje para definición de datos y uno para manipularlos. [40]

2.4.2 Normalización de bases de datos

La normalización de bases de datos es un proceso de diseño de bases de datos que organiza los datos en estructuras de tablas específicas. Ayuda a mejorar la integridad de los datos y prevenir anomalías en los datos [41].

Para que las tablas de nuestra BD estén normalizadas deben cumplir con las siguientes reglas;

- Cada tabla debe tener un nombre único
- No puede haber 2 filas iguales
- No se permiten duplicados [42].

Comentado [GM2]: Mencionar modelo entidad relación o entidad relación extendió y modelo relacional

Debemos considerar la creación de tablas y las reglas que se usan para definir las relaciones, estas reglas son diseñadas para proteger los datos y para que la base de datos sea flexible con el fin de eliminar redundancias y dependencias incoherentes [43].

2.4.4. Modelo entidad relación

Es una representación visual de cómo los elementos de una base de datos se relacionan entre sí, utilizan un conjunto de símbolos como rectángulos, óvalos y diamantes que funcionan como líneas de conexión. Este modelo cuenta con personas o roles (estudiantes, vendedores, ejecutivos o clientes), eventos (suscripciones o transacciones), objetos (productos, facturas o correos electrónicos).

- Relación uno a uno (1:1): Solo puede ser referenciado por un registro en la otra entidad por lo tanto solo uno pertenece al otro y viceversa
- Relación uno a muchos (1:M) Un registro de una entidad se relaciona con múltiples registros en otra entidad
- Relación de muchos a muchos (M:M): Se pueden conectar uno o más registros dentro de ambas entidades o sea que se pueden tener muchas relaciones [44]

Los diagramas entidad-relación se emplean para diseñar bases de datos relacionales, también nos ayuda a analizar bases de datos existentes con el fin de encontrar problemas de lógicas o implementación. Los modelos entidad relación pueden ser útiles para analizar datos que se emplean en investigaciones. [45]

2.4.3 MySQL

MySQL es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto basado en SQL, este es destacado por su arquitectura cliente-servidor siendo una parte clave en (Linux, Apache), MySQL organiza los datos en tablas mediante claves primarias y foráneas, esto para reducir la redundancia de datos y poder encontrar los más fácil.

MySQL tiene 2 funciones principales, almacenar toda la información en bases relacionales, y administrar todos los datos de manera sencilla, gracias a su interfaz visual [46].

2.4.4 MariaDB

Después de que MySQL fuera comprado por Sun Microsystems (la cual sería comprada por Oracle posteriormente en 2010) en 2008 [47], el creador de MySQL Michael Widenius creó un fork (creación de una copia que evoluciona de manera independiente) debido a su visión de mantener una versión de MySQL de código abierto [48].

Además, MariaDB se encuentra en el lugar número 13 en el ranking de motores de base de datos [49] y al tener una licencia GNU 2.0 (License General Public por sus siglas en inglés) permite su uso, modificación y distribución de forma libre y gratuita. [50] Para los fines de este proyecto, adoptar una tecnología bajo esta licencia asegura la viabilidad económica del prototipo a largo plazo, garantizando un motor de base de datos relacional robusto para almacenar la información del repartidor, el estatus del pedido y los datos del usuario, sin el riesgo de depender de corporaciones privadas o caer en costos de licenciamiento a la hora de la escalabilidad.

2.5 Algoritmo de búsqueda y heurística

2.5.1 Grafos

La teoría de grafos se inicia debido a un problema famoso que data de 1736 resuelto por Leonhard Euler, el cual planteaba la siguiente pregunta, en una ciudad con siete distintos puentes, ¿será posible planear un paseo de manera que saliendo de casa cruce los siete puentes una sola vez cada uno antes de regresar a casa? Se denomina el problema de los puentes de Königsberg [51].

Un grafo se define formalmente como un par ordenado $G = (V, E)$, donde V representa un conjunto de vértices o nodos (puntos de intersección) y E es un conjunto de aristas (rutas) que conectan dichos nodos [52].

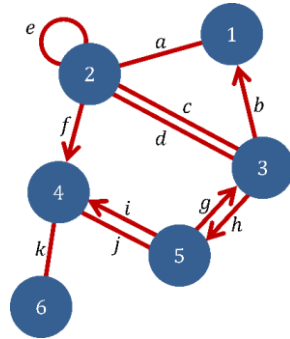


Ilustración 3 Grafo ponderado tomado de [123]

Además, existen grafos ponderados, los cuales poseen un peso asociado a cada arista como se observa en la ilustración 3. Este peso ayuda al cálculo de “costos” que se recorre en el grafo. En el contexto de este proyecto, el grafo a utilizar es ponderado, ya que cada arista representa la distancia física entre diferentes puntos del mapa.

2.5.2 Algoritmos de búsqueda

Se define a un algoritmo de búsqueda como un conjunto de instrucciones que están diseñadas para localizar un elemento en específico dentro de una estructura de datos. Estos algoritmos se dividen en tres grupos principalmente: búsqueda secuencial (lineal), búsqueda binaria y búsqueda de caminos mínimos [53]. En este caso, se utilizará la búsqueda de caminos mínimos, ya que está orientada a grafos y a la búsqueda del camino más corto.

En este grupo, existen dos categorías: búsqueda no informada (como BFS o Dijkstra) y búsqueda informada (utilizando heurística). Mientras que los primeros exploran el espacio de estados de manera exhaustiva sin conocer la ubicación del objetivo final, los algoritmos informados utilizan este conocimiento para priorizar los nodos que parecen estar más cerca de la meta, reduciendo significativamente el costo computacional y el tiempo de respuesta, aunque incrementando su dificultad [54].

2.5.4 Heurística

En ciencias de la computación, una heurística es una función que guía la búsqueda proporcionando una estimación del costo restante. Su propósito principal es mejorar la eficiencia del algoritmo al descartar caminos menos prometedores [55]. En la planificación de rutas urbanas, la heurística suele representarse como la distancia en línea recta entre dos puntos geográficos, funcionando como un "atajo" para el algoritmo.

2.5.4 Algoritmo A*

El algoritmo A* es una mejora del algoritmo de Dijkstra que logra una mayor eficiencia mediante el uso de heurísticas. Su funcionamiento se basa en la función de evaluación:

$$f(n) = g(n) + h(n)$$

Donde:

$g(n)$: representa el costo real acumulado desde el nodo de inicio hasta el nodo actual n .

$h(n)$: es el valor estimado (heurística) del costo para llegar desde el nodo n hasta el destino final.

A* es óptimo (encuentra el camino más corto) y completo (encuentra una solución), siempre que la función heurística sea admisible, es decir, que nunca sobreestime el costo real al objetivo [53].

2.5.5 Heurística Haversine

La Fórmula de Haversine permite calcular la distancia de círculo máximo entre dos puntos de una esfera a partir de sus longitudes y latitudes:

$$d = 2r \cdot \sin^{-1} \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Donde r es el radio terrestre, ϕ es la latitud y λ la longitud [56]. Esta fórmula constituye la heurística admisible para el algoritmo A* en este proyecto, asegurando que el cálculo de la ruta hacia el usuario sea preciso y eficiente dentro de la cartografía de la Ciudad de México.

Capítulo III. Estado del arte

3.1 Ámbito Tecnológico

3.1.1 API Google Maps

En la actualidad, la API de Google Maps se considera el estándar para el desarrollo de Sistemas de Información Geográfica (GIS) en dispositivos móviles. Su arquitectura se basa en microservicios que permiten la integración de mapas interactivos, geocodificación y cálculo de rutas en tiempo real. Permite la visualización de la zona urbana. En contextos de alta densidad poblacional como la Benito Juárez, la precisión del renderizado de calles es vital para el repartidor. Por otro lado, también es esencial para calcular la ruta óptima entre el centro de distribución y el domicilio del cliente. Según

la literatura, el uso de algoritmos de optimización de rutas reduce el consumo de combustible y el tiempo de entrega en un 15-20%. [57]

3.1.2 Open Street Maps

Open Street Maps (OSM), es un proyecto de cartografía de datos abiertos, permite la descarga y manipulación directa de sus nodos, vías y relaciones. [58] Esta accesibilidad facilita la extracción de la red vial de la alcaldía Benito Juárez para la construcción de un grafo local. Sobre este grafo, el algoritmo A* puede evaluar los costos de trayectoria (distancias, sentidos de calles) de manera nativa, lo que permite al algoritmo de A* decidir la ruta. Además, el uso de Open Street Maps disminuye costos de operación del sistema ya que depende de una API externa de cobro para la lógica matemática.

Asimismo, las plataformas propietarias como Google Maps operan bajo un modelo cerrado, la cual solamente proporciona el resultado de una ruta final, pero prohíbe el acceso a los datos vectoriales, lo que hace imposible la ejecución de un algoritmo de enrutamiento propio y obligando a depender de sus sistemas.

Debido a estas razones, se empleará OpenStreetMap (OSM) como la fuente de datos para el modelo del algoritmo A*.

Por otro lado, Google Maps se reservará exclusivamente para la capa de presentación y como un puente hacia la navegación externa. La familiaridad, alta disponibilidad y fluidez de renderizado cartográfico de la API de Google Maps ofrecen una experiencia con la que el usuario ya está acostumbrado. De esta forma, el sistema calculará la ruta óptima internamente procesando los nodos de OSM y posteriormente proyectará dicha geometría de forma visual aprovechando la infraestructura de Google Maps. Adicionalmente, se brindará al repartidor la opción de exportar las coordenadas hacia la aplicación nativa de Google Maps en caso de que lo prefiera.

3.1.2 Aplicaciones móviles

Según datos de la “Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2024” realizada por el INEGI, se puede observar que un 81.7% de la población mexicana tiene acceso a un dispositivo móvil y un 83.1% acceso a internet. [59]

En la figura número 3 proporcionada por la INEGI, se puede observar la tendencia de crecimiento en el acceso a un dispositivo móvil e internet.

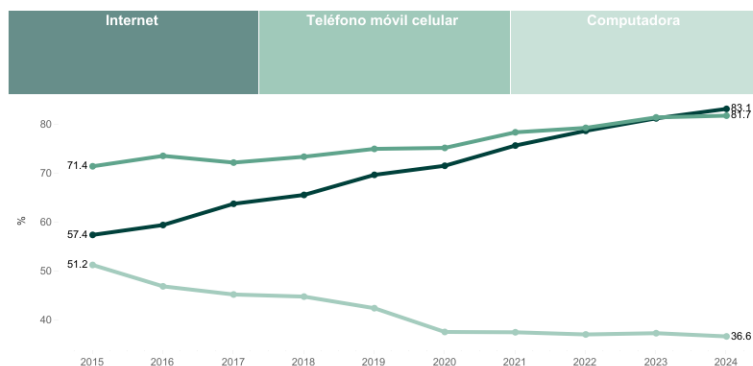


Ilustración 4 Gráfica de la encuesta ENDUTIH realizada por el INEGI [59]

En el año 2024 el acceso a los sitios de ventas en línea ha sido impulsado principalmente por dispositivos móviles, los cuales representaron el 69% del tráfico total. Lo que consolida el papel de este dispositivo como el principal medio de acceso debido a su accesibilidad y conveniencia. [60]

Como se ha observado, el uso del celular ha incrementado en los últimos años, así como su uso para la realización de compras en línea, lo que facilita la aceptación de la aplicación móvil propuesta.

En el mercado a nivel mundial, las apps invirtieron 65 mil millones de dólares en adquisición de usuarios, logrando un crecimiento del 5 % en su base de usuarios en 2024. En México, este incremento fue impulsado por una mayor inversión publicitaria del 16 %, con las apps de finanzas y entretenimiento liderando el crecimiento con un 48 % en gasto publicitario cada una, seguidas por las de Transporte con un 38 %. [61]

En México ha habido un crecimiento en las instalaciones y sesiones en aplicaciones como se observa en la ilustración 4.

Porcentajes de crecimiento de las instalaciones y sesiones en aplicaciones entre el cuarto trimestre de 2022 y el cuarto trimestre de 2023 (México)

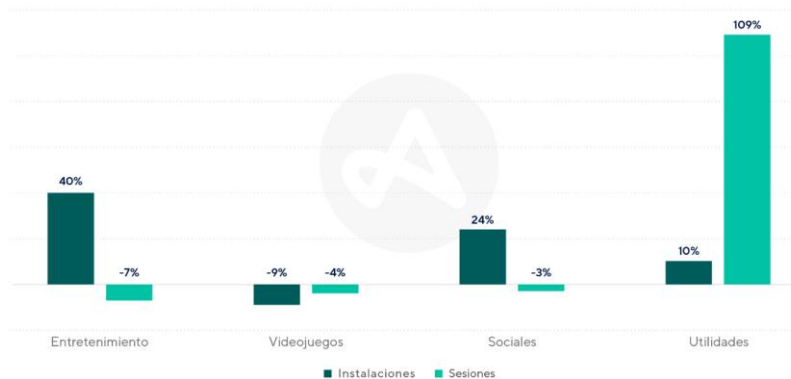


Ilustración 5 Crecimiento de las instalaciones en México 2023

3.1.3 Aplicaciones web

El ecosistema de las aplicaciones web en los últimos años ha experimentado una transformación, pasando de ser repositorios de información estática a plataformas de software complejas y altamente interactivas. En la actualidad, el desarrollo web se rige por la necesidad de ofrecer experiencias de usuario comparables a las de las aplicaciones nativas, pero con la particularidad de una mejor accesibilidad y el alcance global de un navegador. Además, la experiencia del usuario (normalmente conocida como UX) es de vital importancia, ya que incita al usuario a utilizar el aplicativo. [62]

En México, la adopción de servicios digitales ha alcanzado niveles altos. Según el Estudio de Venta Online 2024 de la AMVO, México es uno de los países con mayor crecimiento en comercio electrónico a nivel mundial, con más de 65 millones de compradores digitales. Este informe destaca que el 40% de estos usuarios realiza compras semanales, y que las categorías de servicios a domicilio y productos de consumo masivo muestran una alta tasa de recurrencia. [60]

3.1.4 Lenguajes de programación

Kotlin es un lenguaje tipado. La principal ventaja de un lenguaje de tipado estático es la protección contra errores de ejecución y una ejecución más rápida. Esto se debe a que el compilador conoce los tipos de datos exactos en uso y puede generar código máquina optimizado. [63]

Por otra parte, contamos con la ventaja del desarrollo multiplataforma que ofrece Kotlin, donde se puede utilizar para cualquier tipo de desarrollo, desde la web del lado del servidor y del lado del cliente, hasta Android e iOS. Como el lenguaje se ejecuta en JVM, permite compartir código entre diferentes plataformas [64]

3.2 Sistemas existentes

En la tabla 1, se observa una comparación sobre los sistemas existentes:

SOFTWARE	DISPONIBILIDAD EN MÉXICO	ENTREGA DE GARRAFONES	DISEÑADA PARA USUARIO	DISEÑADA PARA REPARTIDOR
Rappi	SI	NO	SÍ	SI
Uber Eats	Si	NO	SÍ	SÍ
Appgua	No	SÍ	SÍ	NO
Reparto de Agua	No	SÍ	NO	SI
HToGo	SÍ	SÍ	SÍ	SÍ

Tabla 1 Comparación de aplicaciones de reparto

Como se puede observar en la comparativa, hay diferentes aplicaciones que se encargan de la venta de productos de tiendas, pero estas no cuentan con el reuso de garrafrones ni con la entrega de los mismos, únicamente venta de agua embotellada. Existe una aplicación llamada “Appgua” la cual era para el rellenado de garrafrones, sin embargo, su aplicación no funciona y solamente es la página web. Existe otra aplicación disponible en la PlayStore llamada “Reparto de Agua”, la cual está disponible solamente para Argentina y está orientada únicamente hacia el repartidor, sin contar con la opción de vista del viaje para el cliente.

Por lo tanto, concluimos que no hay una aplicación dedicada solamente a la venta de garrafrones en México, solamente se cuenta con otras alternativas que no ofrecen exactamente este servicio, por ello surge la necesidad de esta aplicación orientada a la alcaldía Benito Juárez.

Además, anteriormente se han realizado tesis las cuales ya abarcaban la necesidad de una mejora sobre la distribución de agua, con el único detalle que estaban enfocadas en otras delegaciones, así como al giro de la ingeniería civil [65] y administración pública [66].

Refiriéndose a la parte logística, se realizó una tesis sobre la logística para empresas químicas [67], aunque indaga sobre temas logísticos no está orientada hacia la distribución de agua, como se busca en esta tesis.

Nombre	Características	Ventajas	Limitaciones
Rappi [68]	App que ofrece desde comida hasta servicios financieros (RappiPay). Basada en una red de "Socio Repartidores" independientes.	Gran presencia en el mercado mexicano. Permite programar pedidos y ofrece una interfaz muy completa tanto para el cliente como para el repartidor.	No permite el transporte de garrafrones de 20L. Sus políticas de seguridad para repartidores en moto/bici prohíben cargas que excedan dimensiones estándar.
Uber Eats [69]	Plataforma global de entrega de alimentos y productos de conveniencia	Logística optimizada y alta disponibilidad. Interfaz de usuario	Restricción de carga pesada. Al igual que su competencia, su

	integrada en el ecosistema de movilidad de Uber.	extremadamente intuitiva y un sistema de rastreo en tiempo real líder en la industria.	modelo de negocio ignora la logística de garrafones debido al peso y la necesidad de vehículos específicos.
Appgua [70]	Aplicación móvil especializada en la gestión de pedidos de agua potable directamente con purificadoras.	Especialización en el nicho de agua. El flujo de compra está diseñado específicamente para el llenado y entrega de garrafones de 20L.	Disponibilidad regional limitada. No tiene cobertura nacional en México y carece de una interfaz dedicada para la gestión del repartidor.
Reparto de agua [71]	Sistema de gestión de software (B2B) enfocado en la administración de rutas y logística de entrega de agua.	Excelente control logístico. Diseñada para que el administrador y el repartidor lleven el control de ventas y el inventario de envases.	No es una aplicación de cara al cliente. Un usuario común no puede descargarla para pedir un garrafón; requiere gestión interna previa.

3.3 Tendencias actuales

En la actualidad, la digitalización de los servicios de entrega a domicilio ha dejado de ser una ventaja competitiva para convertirse en una necesidad operativa. En esta era de la digitalización la integración de tecnologías móviles y arquitecturas robustas permite optimizar la cadena de suministro, especialmente en el sector de la distribución, donde la logística la fidelización del cliente son críticas [72].

Una de las tendencias más fuertes es el uso de GPS integrados con algoritmos de optimización de rutas. El aprovechamiento de lenguajes modernos como Kotlin para el desarrollo nativo en Android permite una gestión eficiente de los recursos del dispositivo, facilitando el seguimiento en tiempo real tanto para el repartidor como para el usuario final. Esto permite reducir tiempos de entrega y costos de combustible, alineándose con políticas de sostenibilidad urbana [73].

Capítulo IV. Análisis del Sistema

En este capítulo, se aborda el análisis y diseño del sistema para la Aplicación móvil de reparto de agua potable en la Alcaldía Benito Juárez. Ofrece una visión detallada del proceso de desarrollo, desde la identificación de los requerimientos hasta la elaboración de la arquitectura del sistema y el diseño de sus componentes principales.

4.1 Encuesta y análisis de datos

Para comprender mejor el panorama actual y tener una mejor visión sobre la aplicación que se desea realizar, será necesario analizar las respuestas del público objetivo. Esto permitirá obtener datos concretos, así como evaluar las principales dificultades que encuentra la comunidad de la alcaldía Benito Juárez.

El cuestionario que se aplicó fue el siguiente, en donde se debe de colocar una opción:

1.- ¿Crees que la alcaldía Benito Juárez hay una buena distribución de agua potable?

- ☐ Sí
- ☐ No

2.- ¿Cuál es tu método actual para conseguir agua potable a domicilio?

- ☐ Ir a la purificadora
- ☐ Pedido a domicilio
- ☐ Ir a una tienda
- ☐ WhatsApp/Repartidor de confianza

3.- ¿Cuánto tiempo se demoran en traerte el agua los repartidores?

- ☐ 30 minutos
- ☐ 1 a 2 horas
- ☐ 3 horas o más

4.- En una escala del 1 al 5 que tan buena ha sido tu experiencia con el servicio de repartición de agua potable

	1	2	4	5	
Mala	☐	☐	☐	☐	Buena

5.- ¿Te has quedado sin agua porque el repartidor no pasó o no escuchaste el timbre/claxon?

☐ Sí

☐ No

6.- ¿Qué tan útil te parece la opción de programar tus pedidos con días de anticipación?

1

2

4

5

Poco útil

☐

☐

☐

☐

Muy útil

7.- ¿Utilizas actualmente otras aplicaciones de servicios a domicilio? (Uber Eats, Rappi, etc.)

☐ Sí

☐ No

8.- ¿Qué tan útil crees que sería una app para pedir garrafones de agua a domicilio?

1

2

4

5

Poco útil

☐

☐

☐

☐

Muy útil

9.- ¿Qué tan satisfecho estás con la limpieza y el estado físico de los garrafones que recibes?

1

2

4

5

Poco útil

☐

☐

☐

☐

Muy útil

10.- ¿Has tenido que cancelar planes o dejar de salir de casa por esperar al repartidor de agua?

☐ Sí

☐ No

11.- ¿Qué tan útil te resultaría recibir una notificación automática cuando el repartidor esté cerca de tu ubicación?

	1	2	4	5	
Poco útil	O	O	O	O	Muy útil

Para determinar el número de encuestas a aplicar en el proyecto HTOGO, se tomó como base la población total residente de la colonia Tlacoquemécatl del Valle, en la Alcaldía Benito Juárez. De acuerdo con indicadores demográficos recientes, esta zona cuenta con una población de aproximadamente 7,200 habitantes y una densidad de 1,270 personas por manzana [74]. Se seleccionó esta muestra para representar la percepción individual de los ciudadanos frente a la problemática del desabasto de agua.

$$n = \frac{N * Z^2 * p * q}{e^2 * (N - 1) + Z^2 * p * q}$$

Donde los parámetros establecidos son [75]:

n: tamaño de la muestra buscado

N: tamaño de la población (7200)

Z: nivel de confianza de 95% (1.96)

e: margen de error admitido del 5% (0.05)

p: probabilidad de ocurrencia del evento (0.5)

q: probabilidad de no ocurrencia (0.5).

$$n = \frac{(7200) * (1.96)^2 * (0.5) * (0.5)}{(0.067)^2 * (7200 - 1) + (1.96)^2 * (0.5) * (0.5)} = \frac{6914.88}{32.31 + 0.96} = 207.84$$

≈ 206 Encuestados

Para obtener la información se realizó durante un periodo de una semana, logrando recolectar las 206 encuestas necesarias para garantizar que los datos obtenidos en la estadística del estudio dentro del margen de error establecido.

4.2 Requerimientos

Los requerimientos de software ayudan a determinar qué características estarán en el producto final, cuánto tiempo llevará desarrollar esas características y cuánto costará. Un requerimiento define las

funciones, capacidades o atributos intrínsecos de un sistema de software, en otras palabras, describe como un sistema debe comportarse [76].

4.2.1 Requerimientos funcionales

Tabla 2. Requerimientos funcionales. Elaboración propia.

RF	Descripción	No. Caso de Uso	Actores	RNF relacionado
RF - 001	La aplicación permitirá al cliente realizar solicitudes de pedidos de agua potable especificando cantidad y domicilio.	CU-001: Realizar pedido	Cliente	RNF-003, RNF-007
RF - 002	La aplicación deberá permitir a los nuevos usuarios registrarse proporcionando nombre, correo electrónico, número telefónico y una contraseña.	CU-002: Registrar usuario	Cliente, Repartidor	RNF-005, RNF-008
RF - 003	La aplicación deberá permitir a los usuarios registrados iniciar sesión mediante sus credenciales (correo y contraseña) para acceder a las funciones del sistema.	CU-003: Iniciar sesión	Cliente, Repartidor	RNF-004, RNF-006
RF - 004	El sistema deberá mostrar al repartidor la ruta para la entrega del pedido, utilizando la API de Google Maps.	CU-004: Visualizar ruta de entrega	Repartidor	RNF-009, RNF-011
RF - 005	La aplicación enviará una notificación al cliente cuando el repartidor se encuentre a una distancia de proximidad definida de su domicilio.	CU-005: Recibir notificación	Cliente, Repartidor	RNF-013
RF - 006	La aplicación permitirá al cliente visualizar en el mapa la ubicación actual del repartidor y el estatus de su pedido	CU-006: Rastrear pedido	Cliente	RNF-003, RNF-013
RF - 007	La interfaz debe mostrar indicadores visuales claros (colores o iconos) para diferenciar el estado de los pedidos en el mapa.	CU-007: Visualizar indicadores de estado	Cliente	RNF-002, RNF-003
RF - 008	La aplicación permitirá al repartidor elegir que pedidos tomar	CU-008: Elegir pedido	Repartidor, Cliente	RNF-007, RNF-012

Tabla 2. Requerimientos funcionales. Elaboración propia.

RF	Descripción	No. Caso de Uso	Actores	RNF relacionado
RF - 009	El sistema debe permitir al super administrador dar de alta y dar de baja usuarios.	CU-009: Dar de alta a un usuario CU-010: Dar de baja usuario	Super administrador	RNF-008
RF - 010	El sistema debe permitir al usuario y al repartidor ver el historial de pedidos asociados a su propio ID.	CU-011: Observar historial.	Cliente, repartidor	RNF-007
RF - 011	El sistema validará la hora del servidor al solicitar un pedido; si está fuera del rango de 08:00 a 18:00 hrs, programará automáticamente la entrega para el siguiente día hábil.	CU-001: Realizar pedido	Cliente	RNF-010
RF - 012	El sistema no permitirá que el repartidor acepte nuevos pedidos si la suma de los garraones de su lista actual supera la capacidad máxima registrada de su vehículo activo.	CU-008: Elegir pedido	Repartidor	RNF-007, RNF-012
RF - 013	La aplicación habilitará el botón "Cancelar Pedido" únicamente cuando el estado en la base de datos sea "Pendiente" o "Asignado", ocultándolo automáticamente si pasa a "En camino".	CU-012: Cancelar pedido	Cliente	RNF-007
RF - 014	La aplicación del repartidor activará un temporizador visual de 10 minutos tras enviar la notificación de entrega, al finalizar, habilitará la opción para marcar el pedido como "No entregado".	CU-007: Finalizar Entrega	Repartidor	
RF - 015	Durante el registro, el sistema realizará una consulta a la base de datos y bloqueará la creación de la cuenta si el número telefónico o correo electrónico ya existen.	CU-002: Registrar usuario	Cliente, Repartidor	

Tabla 2. Requerimientos funcionales. Elaboración propia.

RF	Descripción	No. Caso de Uso	Actores	RNF relacionado
RF - 016	El sistema no permitirá un nuevo inicio de sesión si un usuario ya se encuentra activo en otro dispositivo.	CU-003: Iniciar sesión	Cliente, Repartidor	

4.2.2 Requerimientos no funcionales

Tabla 3. Requerimientos no funcionales. Elaboración propia.

RNF	Descripción	Propiedad	Aplicación
RNF - 001	La aplicación móvil debe ser compatible en dispositivos que cuenten con el sistema operativo Android 8.0 o versiones superiores.	Usabilidad	Interfaz móvil
RNF - 002	La interfaz del repartidor debe tener un contraste alto y botones principales grandes para facilitar su uso bajo luz solar directa.	Usabilidad	Interfaz móvil
RNF - 003	La interfaz debe ser responsive para que el mapa y los botones se vean correctamente en diferentes resoluciones.	Compatibilidad	Interfaz móvil
RNF - 004	Si la aplicación del usuario cliente permanece en segundo plano o sin abrirse por más de 30 días, el token de sesión caducará forzando un nuevo inicio de sesión.	Seguridad	Gestión
RNF - 005	Las contraseñas de los usuarios y repartidores deben ser cifradas.	Seguridad	BD
RNF - 006	El sistema debe limitar el acceso a una sola sesión activa por cuenta de usuario	Seguridad	Gestión
RNF - 007	La base de datos debe soportar la concurrencia de al menos 100 transacciones simultáneas	Escalabilidad	Toda la app
RNF - 008	El sistema debe garantizar que los usuarios solo puedan acceder a las secciones que corresponden a su rol.	Seguridad	Usuarios y roles

4.3 Reglas de negocio

Las reglas de negocio describen las políticas, normas, operaciones, definiciones y restricciones presentes en una organización y que son de vital importancia para alcanzar los objetivos de ésta [77]. Las reglas de negocio son dinámicas, están sujetas a cambios en el tiempo y pueden encontrarse en todo tipo de aplicaciones [78].

En la Tabla 4 se muestran las reglas de negocio de la aplicación web a desarrollar.

Tabla 4. Reglas de negocio. Elaboración propia

RN	Nombre	Descripción	En caso de cumplimiento	En caso de no cumplimiento	RF/RNF	RN
RN-001	Cobertura geográfica	El sistema solo permitirá el registro de direcciones y entregas dentro de la alcaldía Benito Juárez CDMX.	Se permite al usuario continuar con la solicitud del pedido.	El sistema bloquea la solicitud y muestra una notificación o alerta de "Fuera de zona de cobertura".	RF-001, RF-011	
RN-002	Verificación de Usuario	Todos los usuarios deben verificar correo electrónico para poder usar la app.	La cuenta se activa y el usuario accede al menú de pedidos.	La cuenta no se activa y por lo tanto no se pueden solicitar pedidos.	RF-006, RF-010	
RN-003	Dirección del usuario	Las direcciones deben de contar con una descripción para facilitar su entrega.	Se guardan las coordenadas exitosamente	No se permite guardar la dirección manualmente.	RF-001	
RN-004	Horario de disponibilidad	Los pedidos solo se pueden realizar en cierto horario 08:00 am a 18:00 pm.	Se agenda en pedido con normalidad	El pedido se agenda con fecha del día siguiente o a elección.	RF-011	
RN-005	Cancelar un pedido	El usuario solo puede cancelar un pedido si el estatus es "Pendiente" o "Asignado" (no "En camino").	El pedido se cancela y se libera al repartidor.	La opción de cancelar se oculta en la interfaz.	RF-013	
RN-006	Penalización por Ausencia	Si un usuario no recoge su pedido 2 veces seguidas, el	El usuario opera con normalidad.	El sistema bloquea la	RF-013	RN-006,

Comentado [BT3]: Checar con la API

Comentado [GM4R3]: Lo cambié, mejor poner que la dirección debe de tener datos de ayuda (interior, color de puerta, etc)

RN	Nombre	Descripción	En caso de cumplimiento	En caso de no cumplimiento	RF/RNF	RN
		sistema lo suspende temporalmente.		cuenta para nuevos pedidos.		RN-011
RN-007	Inventario Inicial del repartidor	El repartidor debe registrar su inventario de garrafrones en la app antes de poder realizar pedidos.	El sistema habilita la disponibilidad de rutas.	El repartidor no puede elegir sus pedidos.	RF-008, RF-012	RN-007
RN-008	Límite de carga del repartidor	No se asignarán pedidos si la suma de garrafrones supera la capacidad física máxima del vehículo.	Se asignan pedidos hasta llenar la capacidad.	El repartidor no puede elegir más pedidos.		
RN-009	Recálculo de ruta	En caso de varios pedidos activos si un pedido es cancelado antes de que el repartidor inicie su ruta, el sistema recalculará las rutas.	El repartidor recibe la nueva ruta.	Se continua con la ruta establecida.	RF-004	
RN-010	Notificación de proximidad	El sistema enviará una notificación al usuario cuando el repartidor esté a 500 metros de distancia.	El usuario recibe la alerta notificación	No se muestra alguna notificación	RF-005	
RN-011	Entrega del pedido	El repartidor debe estar a menos de 50 metros de las coordenadas del cliente para poder marcar el pedido como "Entregado".	El botón de "Entregado" se habilita	El botón permanece bloqueado o deshabilitado	RF-013	RN-012
RN-012	Tolerancia en domicilio	El repartidor esperará como máximo 10 minutos a que el cliente salga por su pedido.	El pedido se finaliza con estatus "Entregado".	El pedido cambia a "No entregado" (Suma penalización).		
RN-013	Registro de pedidos	Todo cambio en el estado de un pedido	Se mantiene un historial del	El sistema rechaza el	RF-001,	

RN	Nombre	Descripción	En caso de cumplimiento	En caso de no cumplimiento	RF/RNF	RN
		debe generar un registro histórico con marca de tiempo (timestamp) que no podrá ser borrado ni editado.	ciclo de vida del pedido.	cambio de estado si falla el registro histórico.	RF-013	
RN-014	Caducidad de garrafrones	El sistema no permitirá que un repartidor registre o entregue garrafrones cuya fecha de caducidad haya vencido o esté a menos de 7 días de vencer	El garrafón queda registrado y disponible para entrega.	El sistema bloquea el registro del garrafón y muestra un mensaje de "Garrafón caducado o próximo a caducar".	RF-008	RN-007
RN-015	Asignación vehicular	Un repartidor no puede ponerse en estado "Activo" ni aceptar pedidos si no tiene al menos un vehículo registrado	Cada pedido queda asociado correctamente al vehículo utilizado.	El sistema bloquea el cambio de estado del repartidor	RF-003, RF-015	RNF-006
RN-016	Creación de nuevas cuentas	El sistema no permitirá la creación de un nuevo registro con un mismo teléfono o correo	Se crea la cuenta correctamente	El sistema rechaza el registro y muestra el campo que se debe cambiar		
RN-017	Visualización de datos	Al realizar una consulta de los pedidos, el usuario solo puede ver los que coinciden con su cuenta.	El cliente visualiza correctamente su historial personal.	El sistema bloquea el acceso por falta de permisos.	RF-002	RNF-005
RN-018	Cifrado de contraseña	Toda operación de creación o actualización sobre el campo password_hash en debe aplicar un	La contraseña se almacena de forma segura e ilegible.	El sistema rechaza guardar texto plano por seguridad.		

RN	Nombre	Descripción	En caso de cumplimiento	En caso de no cumplimiento	RF/RNF	RN
		algoritmo de encriptación.				
RN-019	Límite de Sesiones Activa	Una cuenta de usuario o de repartidor solo puede mantener un token de sesión activo en un solo dispositivo a la vez para evitar suplantaciones.	El usuario interactúa con la app sin interrupciones.	El usuario no podrá acceder a la app.	RF-008	
RN-020	Cambios en el perfil	Un repartidor no podrá cambiar su fotografía de perfil ni el nombre de los vehículos registrados libremente; toda actualización requiere aprobación de un administrador.	Se garantiza que el cliente vea al repartidor correcto.	Los cambios se quedan en estatus "Pendiente de revisión".		
RN-021	Concurrencia en aceptación de pedidos	Cuando dos o más repartidores intentan aceptar el mismo pedido de forma simultánea, el sistema garantiza que solo uno lo reciba. El pedido se asigna al primero en confirmar y se notifica al resto que el pedido ya no está disponible.	El pedido queda asignado únicamente al primer repartidor que confirmó.	El sistema revierte la acción del segundo repartidor y muestra el mensaje "Este pedido ya fue tomado".	RF-008	RN-008

4.4 Análisis de tecnologías

El análisis realizado se enfoca en las diferentes herramientas posibles a utilizar en el desarrollo de la aplicación móvil. Por este motivo se elaboraron tablas comparativas en cada una de las áreas colocando distintas tecnologías. Tras este análisis, se seccionará la tecnología más adecuada para el desarrollo de la aplicación móvil.

4.4.1 Tecnologías para el Front-End

En el desarrollo web, se define al front-end como aquellas tecnologías que corren del lado del cliente, del lado del navegador web, generalizándose en principalmente 3 lenguajes: HTML, CSS y JavaScript. Su principal función es encargarse de dar estilo a la página, logrando que se presente la información de forma accesible para el usuario [79].

En la Tabla 5 se muestran diferentes tecnologías comparadas para el Front-End [80], lo cual ayudará a identificar cuál de ellas será la mejor para el desarrollo de la aplicación móvil, basándose en sus ventajas, desventajas, el lenguaje de programación usado y su curva de aprendizaje.

Tabla 5. Tecnologías para el Front-End

Criterio	Angular [81]	React [82]	Vue.js [83]
Descripción	Framework para construir aplicaciones web escalables.	Biblioteca de JavaScript para construir interfaces de usuario basadas en componentes.	Framework progresivo para construir interfaces de usuario y SPAs.
Tipo	Framework	Librería	Framework
Lenguaje	TypeScript	JavaScript / JSX	JavaScript
Curva de Aprendizaje	Elevada, requiere conceptos complejos como Inyección de Dependencias y RxJS.	Media, centrada en componentes; requiere librerías externas para ruteo o estado.	Baja/Media, intuitivo con una sintaxis clara y buena documentación.
Ventajas	Estructura modular integral, herramientas potentes y alta mantenibilidad, además de soporte por parte de Google.	Gran ecosistema, alto rendimiento gracias al Virtual DOM y flexibilidad total.	Ligero, fácil integración en proyectos existentes y excelente rendimiento.
Desventajas	Archivos pesados y curva de aprendizaje grande.	No es un framework completo (requiere dependencias externas), actualizaciones frecuentes.	Ecosistema más pequeño comparado con Angular o React.
Uso Recomendado	Aplicaciones empresariales de gran escala y alta complejidad.	Aplicaciones dinámicas con alta interacción de usuario y SPAs.	Proyectos que requieren rapidez de desarrollo y ligereza.

Basándose en la comparación, se seleccionó a Angular como framework para el desarrollo del Front-End gracias a alto rendimiento y su arquitectura modular basada en componentes, además de ser una decisión orientada a su buena eficiencia.

Asimismo, el equipo de trabajo cuenta con conocimiento previo en esta tecnología, lo que ayuda a la reducción significativa en la curva de aprendizaje inicial.

4.4.2 Tecnologías para el Back-End

Se define como la capa de acceso a datos de un software, la cual no es directamente accesible por los usuarios. Asimismo, cuenta con la lógica de la aplicación que maneja dichos datos. Esta capa se encarga de acceder al servidor [84].

También es responsable de las consultas a la base de datos y la manipulación de estos datos, así como el proceso de insertado de datos [85].

En la siguiente tabla, se comparan tres diferentes tecnologías para el desarrollo back-end de la aplicación para generar un análisis del uso de estas.

Criterio	Spring Boot [28]	Django [86]	Express.js [87] [88]
Descripción	Framework basado en Spring para crear aplicaciones autónomas de grado de producción con mínima configuración.	Framework de alto nivel que fomenta el desarrollo rápido y un diseño limpio y pragmático.	Framework minimalista y flexible para Node.js que proporciona un conjunto robusto de características.
Tipo	Framework	Framework	Framework
Lenguaje	Java, Kotlin	Python	JavaScript / TypeScript
Curva de Aprendizaje	Elevada: Requiere dominio de Inyección de Dependencias, el ecosistema Spring y manejo de la JVM.	Media: Muy estructurado; una vez que entiendes su flujo, el desarrollo es muy ágil.	Baja: Muy accesible si ya conoces JavaScript, aunque difícil de organizar en proyectos grandes.
Ventajas	Alta escalabilidad, seguridad integrada (Spring Security), fuerte tipado y gran ecosistema empresarial.	Desarrollo extremadamente rápido, panel de administración automático y alta seguridad.	Extremadamente ligero, alto rendimiento en E/S asíncrona, ideal para tiempo real y gran cantidad de paquetes (NPM).
Desventajas	Requiere más memoria RAM y tiene un arranque más lento.	Rendimiento de Python menor frente a Java o Node.js.	Carece de una estructura definida y requiere muchas dependencias externas.
Uso Recomendado	Sistemas bancarios, aplicaciones empresariales	MVPs rápidos, sitios de contenido, redes sociales y aplicaciones	Aplicaciones de chat en tiempo real, streaming, APIs ligeras y

Criterio	Spring Boot [28]	Django [86]	Express.js [87] [88]
	complejas y microservicios robustos.	intensivas en datos (Data Science).	arquitecturas de microservicios.

Gracias a esta comparación, se decidió optar por el framework Spring boot principalmente debido a su robustez arquitectónica, seguridad y a la optimización del equipo de trabajo.

Dado que el equipo de desarrollo ya cuenta con conocimientos sólidos en Angular, la adopción de Spring Boot es sencilla ya que ambos ecosistemas comparten principios de diseño [89], lo que reduce la carga al cambiar entre el desarrollo del Front-End y el Back-End, asimismo como su integración nativa con Spring Data JPA facilita la interacción con la base de datos permitiendo que la información entre los pedidos, los repartidores y la ubicación geográfica sea consistente y escalable.

4.4.3 Tecnologías para la base de datos

Las bases de datos son herramientas de trabajo con la capacidad de proporcionar información sobre un sistema concreto, los datos en sí mismos son de vital importancia [90], ya que permiten un control sobre la funcionalidad del sistema.

Debido a que una base de datos en sí es la herramienta donde se guardará la información, existen los sistemas gestores de base de datos, los cuales son una capa intermedia entre el programa y los aplicativos donde se utilicen [91]. Para definir qué sistema gestor de base de datos se utilizará, se realizó la siguiente tabla comparativa:

Criterio	MariaDB [92]	MySQL [93]	MongoDB [94]
Descripción	Sistema relacional creado como un fork de MySQL, desarrollado por la comunidad.	El sistema de base de datos relacional más popular del mundo, bajo tutela de Oracle.	Base de datos NoSQL líder, orientada a documentos (JSON-like).
Tipo	Relacional	Relacional	No Relacional
Lenguaje	SQL	SQL	BSON
Curva de Aprendizaje	Baja: Si conoces SQL, la transición es inmediata.	Baja: Estándar de la industria con muchísima documentación.	Alta: Requiere un cambio de paradigma.
Ventajas	Licencia GPL, mayor velocidad en consultas complejas.	Gran soporte técnico, ecosistema masivo y muy estable.	Alta escalabilidad horizontal y manejo de datos no estructurados.
Desventajas	Menos reconocimiento comercial que MySQL en entornos corporativos.	Licencia dual (puede tener costos para software privativo bajo Oracle).	No soporta JOINS complejos de forma nativa; alto consumo de memoria RAM.

Criterio	MariaDB [92]	MySQL [93]	MongoDB [94]
Uso Recomendado	Aplicaciones que requieren integridad transaccional y libertad de licencias.	Aplicaciones web estándar y sistemas empresariales tradicionales.	Big Data, catálogos de productos dinámicos y aplicaciones de tiempo real.

Debido a la comparación, se decidió optar por MariaDB, ya que al ser una base de datos bajo la licencia GNU GPL [95], asegura que el prototipo pueda ser escalado o comercializado sin las restricciones o costos que podrían surgir con la licencia dual de MySQL (bajo Oracle).

Además, MariaDB ofrece una compatibilidad casi total con MySQL, lo que permite aprovechar las herramientas de administración existentes y la integración directa con Spring Boot (JPA) sin añadir complejidad técnica. Dado que el sistema maneja datos estructurados, el modelo relacional es superior a una opción NoSQL como MongoDB, ya que garantiza la integridad y consistencia de datos, permitiendo una seguridad mayor.

4.4.4 Tecnologías para la aplicación móvil

El desarrollo de aplicaciones para servicios móviles cuenta con diferencias con el desarrollo de software tradicional en muchos aspectos [96]. Por lo tanto, es de vital importancia decidir qué tipo de lenguaje se usará, ya que hay diferentes consideraciones para tener en cuenta [97], como pueden ser lenguajes nativos, multiplataforma, su uso recomendado entre otras características. Por lo tanto, en la siguiente tabla se realizará un análisis de estos lenguajes:

Criterio	Kotlin [18] [98]	Flutter [99]	React Native [82]
Descripción	Lenguaje moderno y estático para desarrollo nativo en Android.	SDK de Google para crear aplicaciones compiladas de forma nativa desde una sola base de código.	Framework de Meta para construir aplicaciones móviles usando React y JavaScript.
Tipo	Nativo	Framework	Framework
Lenguaje	Kotlin	Dart	JavaScript / TypeScript
Curva de Aprendizaje	Media, ya que requiere comprender el SDK de Android y Jetpack Compose.	Media, utiliza Dart y funciona con la lógica de todo es un widget.	Baja/Media, es muy accesible para quienes ya dominan el desarrollo web.
Ventajas	Máximo rendimiento, acceso total a APIs del sistema y menor consumo de batería.	Interfaz consistente en cualquier dispositivo, desarrollo rápido.	Gran ecosistema de librerías, reutilización de código web y gran comunidad.

Criterio	Kotlin [18] [98]	Flutter [99]	React Native [82]
Desventajas	Limitado a Android y mayor tiempo de desarrollo.	Aumento en el tamaño del APK y no utiliza componentes nativos.	Dependencia de un "puente" que puede afectar el rendimiento.
Uso Recomendado	Apps con uso intensivo de hardware (GPS) y alto rendimiento.	Aplicaciones con diseño UI personalizado y despliegue rápido.	Apps de contenido, redes sociales y apps de baja carga.

Como se puede observar en la tabla de comparación anterior, se decidió utilizar el lenguaje de Kotlin, al ser un lenguaje nativo, ofrece una latencia baja junto con un consumo de memoria eficiente para manejar mapas y geolocalización constante, superando en ese sentido a Flutter o React Native.

Además, el equipo de trabajo ya cuenta con el conocimiento técnico. Esto logra que la curva de aprendizaje sea menor.

4.4.5 API de visualización del mapa

La implementación de servicios de geolocalización va más allá de simplemente la visualización de un sistema, ya que estos servicios son un componente crítico para la toma de decisiones de una ruta. En un proyecto que requiere el uso de algoritmos de búsqueda de caminos (A*) la elección de la API de mapas debe tener en cuenta diferentes características, desde la accesibilidad a la topología (nodos y aristas), así como su manipulación y uso para cálculos [100]. Dependiendo las opciones, habrá diferencias en el acceso a datos.

En la siguiente tabla se presenta un análisis comparativo de los servicios de visualización del mapa:

Criterio	OpenStreetMap [101]	Google Maps Platform [102]	Mapbox [103]
Descripción	Proyecto colaborativo para crear mapas libres y editables. Proporciona datos geográficos brutos.	Plataforma propietaria líder con datos globales de alta precisión y servicios integrados.	Plataforma de mapas basada en datos abiertos (OSM) con gran enfoque en diseño y personalización.
Tipo	Datos Abiertos	SaaS / Propietario (API comercial).	Freemium / Propietario (Basado en Open Source).
Lenguaje / Integración	XML, PBF (Datos); JS (Visualización).	JavaScript, Java, Swift.	JavaScript (Mapbox GL JS), Java, Swift.
Curva de Aprendizaje	Media: Requiere procesar datos brutos.	Baja: Documentación excelente y fácil implementación mediante API Keys.	Media: Muy potente, pero requiere entender el manejo de capas y estilos vectoriales.

Criterio	OpenStreetMap [101]	Google Maps Platform [102]	Mapbox [103]
Ventajas	Acceso a nodos y vías, ideal para algoritmos personalizados (A*), gratuito y sin límites.	Datos de tráfico en tiempo real, Street View y búsqueda de lugares (POIs) inigualable.	Mapas vectoriales fluidos y alta personalización de estilos (Mapbox Studio).
Desventajas	Los datos en zonas rurales pueden estar incompletos; la estética es sencilla.	Costos elevados tras superar la cuota gratuita; no permite extraer nodos para algoritmos propios.	El modelo de precios puede ser complejo; requiere tokens de acceso para todo.
Uso Recomendado	Investigación, desarrollo de algoritmos de rutas y proyectos de datos abiertos.	Aplicaciones comerciales masivas donde el costo no es problema y se requiere Street View.	Aplicaciones con diseño de vanguardia y necesidades de visualización de datos complejos.

Gracias a esta comparación, se llegó a la conclusión sobre la integración OpenStreetMap como tecnología para la visualización del mapa, ya que al optar por Kotlin anteriormente, se asegura un equilibrio óptimo entre el rendimiento nativo y el control sobre los datos geográficos.

Además, para la implementación del algoritmo A* como generador de rutas, se requiere el acceso directo a los nodos y vías del mapa, lo cual es ideal para una plataforma abierta como OpenStreetMaps, donde no se tiene una limitación a los datos como en el caso de Google Maps.

4.5 Análisis de riesgos

Para un proyecto de esta importancia es una parte de vital realizar un análisis de riesgos, la cual se entiende como una evaluación sistemática del riesgo relacionada a cada aspecto relevante de la organización [104]. Esto ayudará a implementar la estrategia de mitigación del riesgo, además de desarrollar un plan de contingencia en casos donde una respuesta es de vital importancia.

Se puede definir como el objetivo que se tiene en el análisis de riesgos es establecer un nivel de tolerancia a cada riesgo [105]. Donde se determina cuáles son los riesgos específicos a los que se enfrenta (técnicos, económica, operativa y temporal).

4.5.1 Factibilidad técnica

La factibilidad técnica se utiliza para tener en cuenta si se cuenta con todos los conocimientos, habilidades, herramientas o licencias necesarias para realizar los servicios, funciones y procedimientos que se desarrollará en el proyecto. Este análisis permitirá conocer si es viable el desarrollo de la aplicación móvil [106].

La Tabla 6 presenta el análisis de factibilidad técnica considerando 5 dimensiones: el área de afectación de la tecnología, la opción que se contempla para la realización, la factibilidad calculada

por medio del conocimiento del equipo de trabajo, los riesgos del uso de la tecnología y la mitigación, la cual será el plan para evitar su riesgo.

Tabla 6. Análisis de factibilidad

Área	Opción	Factibilidad	Riesgos	Mitigación
Front End	Angular	Alta	El equipo cuenta con conocimiento intermedio, pero no en sistemas complejos.	Capacitación intensiva al equipo de desarrollo.
Back End	Spring Boot	Alta	Complejidad en la integración del backend con la aplicación móvil.	Uso correcto de endpoints para el consumo desde la aplicación móvil.
Base de datos	MariaDB	Alta	Posibles limitaciones con el volumen de datos generado por los pedidos.	Monitoreo continuo, posible integración en la nube.
Aplicación móvil	Kotlin	Media	El equipo cuenta con conocimiento básico.	Capacitación y desarrollo temprano de la aplicación móvil.
API de visualización del mapa	OpenStreetMaps y Google Maps	Media	Poco uso de la API del equipo.	Capacitación y pruebas con ambas APIs.

Comentado [GM5]: Me puedes ayudar a ponerla como "Tabla 5"? No me acuerdo donde era jeje

Tras evaluar las tecnologías seleccionadas, sus riesgos y planes de mitigación se concluye que el proyecto es técnicamente viable, aunque haciendo énfasis en la necesidad de capacitación del equipo de trabajo para desarrollar en Kotlin principalmente y Angular, además de la revisión y pruebas de la API principalmente de OpenStreetMaps para la implementación del algoritmo de A*.

4.5.2 Factibilidad económica

Para evaluar la factibilidad económica del desarrollo de la App Movil, se realizó una estimación detallada de los costos asociados a los recursos tecnológicos y operativos necesarios. Esta evaluación contempla tanto el uso de tecnologías de software libre como los gastos en infraestructura y desarrollo humano. La Tabla 7,8,9 y 10 desglosa los componentes considerados

Tabla 7Factibilidad económica Operativa

Cantidad	Nombre	Descripción	Sueldo (mensual)	Total (6 meses)
2	Desarrollador	Personal capacitado para	\$ 15, 824 MXN [107]	\$189,888 MXN

		el desarrollo del proyecto		
			Total	\$189,888 MXN

Tabla 8 Factibilidad económica Tecnológica HW

Cantidad	Nombre	Descripción	Costo	Total
2	Computadora	Dispositivo utilizado para el desarrollo y análisis	\$ 7, 999 MXN [108]	\$15,998 MXN
2	Dispositivo móvil	Dispositivo para pruebas	\$2,898 MXN [109]	\$5,796 MXN
			Total	\$21,794 MXN

Tabla 9 Factibilidad económica Tecnológica SW

Cantidad	Nombre	Descripción	Costo	Total
1	IDE	Desarrollo	\$0 MXN	\$0 MXN
1	Angular	Front-end	\$0 MXN	\$0 MXN
1	Spring boot	Back-end	\$0 MXN	\$0 MXN
1	MariaDB	Base de datos	\$0 MXN	\$0 MXN
			\$0 MXN	\$0 MXN

Tabla 10 Factibilidad económica Tecnológica Servicios

Cantidad	Nombre	Descripción	Costo (mes)	Total (6 meses)
1	API Google ,Maps Prueba	API para visualizar en el mapa	\$0 MXN	\$0 MXN
1	API OpenStreetView	Api para marcar la ruta	\$0 MXN	\$0 MXN
1	Electricidad	Energía necesaria para alimentar los dispositivos	\$250 MXN [110]	\$1,500 MXN
1	Internet	Servicio de internet	\$525 MXN [111]	\$3,150 MXN
1	Nube	Servicio para almacenar los datos	\$300 MXN	\$1,800 MXN
			Total	\$6,450 MXN

Operativa: \$189,888 MXN	Tecnológica: \$28,244
Total: \$218,132 (Costo de Desarrollo del Proyecto)	
TOTAL: \$327,198 MXN (Costo final)	

En este contexto, si se logra establecer un flujo de ingresos constante y suficiente, el proyecto no solo resulta económicamente viable, sino también sostenible a corto y mediano plazo. Además, el bajo nivel de inversión inicial incrementa su atractivo para posibles financiadores o aliados estratégicos, lo que abre oportunidades para escalar o mejorar la solución en etapas futuras.

4.5.3 Factibilidad operativa

La factibilidad operativa hace referencia a recursos humanos, procesos operativos y capacidades organizativas, encargadas de ejecutar el proyecto, si estas cuentan con las habilidades de implementar y darle mantenimiento al sistema. Por otro lado, si la solución satisface los requerimientos del sistema, para resolver los problemas y aprovechar las oportunidades que se visualizan en relación con el sistema [112].

Una vez teniendo estos conceptos, el equipo de trabajo encargado del proyecto cuenta con habilidades y conocimientos en desarrollo móvil, bases de datos, implementaciones de inteligencia artificial, así como en tecnologías que se usaran para el back-end y front-end como Spring-boot y Angular siendo estos adquiridos en trabajos escolares, así como en prácticas profesionales, todo esto nos ayuda en la capacidad para implementar y dar mantenimiento al sistema.

4.5.4 Factibilidad temporal

La factibilidad temporal hace referencia a la viabilidad de finalizar el proyecto dentro del tiempo planeado, tomando en cuenta la disponibilidad de recursos durante el proceso como una planificación adecuada de las tareas. Esta evaluación nos permite identificar si el tiempo estimado para llevar a cabo el proyecto se ajusta al tiempo que realmente se necesitará. Por ello, resulta esencial verificar la posibilidad de cumplir con las tareas asignadas en los plazos establecidos y definir un cronograma de trabajo lo más realista posible [113].

Para que el proyecto se complete en el tiempo definido, se elaboró un cronograma de actividades que cuenta con varias etapas desde la investigación, análisis, diseño, desarrollo, pruebas e implementación. Los cronogramas de actividades se diseñaron con base en los recursos humanos disponibles (dos desarrolladores de tiempo parcial) y una duración proyectada de un año A continuación, se presentan los cronogramas de actividades por integrante (Tabla 7 y 8).

Tabla 11 Cronograma Márquez Rodríguez Gael Alejandro

ACTIVIDAD	FEB	MAR	ABR	MAY	JUN	JUL	AGO	SEP	OCT	NOV	DIC
Investigación preeliminar											
Análisis de requerimientos											

Investigación sobre API "Google Maps"											
Diseño de diagramas de caso de uso											
Requisitos del sistema											
Construcción de las características											
Evaluación de TT1											
Integración de la API de "Google Maps"											
Implementar lógica de pedidos											
Prototipo funcional											
Corrección de posibles errores											
Despliegue del sistema											
Evaluación de TT2											

Tabla 12 Cronograma Trejo Hernández Brandon

ACTIVIDAD	FEB	MAR	ABR	MAY	JUN	JUL	AGO	SEP	OCT	NOV	DIC
Investigación preeliminar											
Análisis de requerimientos											
Diseño de la arquitectura general											
Diseño del prototipo											
Documentación general del sistema											
Creación de base de datos											
Evaluación de TT1											
Diseño de la interfaz											
Implementación de notificaciones											
Prototipo funcional											
Pruebas											
Despliegue del sistema											
Evaluación de TT2											

4.6 Arquitectura

El desarrollo de la aplicación HTToGo se desarrollará utilizando el modelo de arquitectura C4, un enfoque jerárquico que permite describir la arquitectura del software a diferentes niveles [114], logrando una mejor comprensión del proyecto.

Para este Trabajo Terminal se presentan los tres primeros niveles del modelo C4: el Diagrama de Contexto, el Diagrama de Contenedores y el Diagrama de Componentes. Estos niveles proporcionan una visión del sistema, desde su relación con actores externos hasta la organización interna de cada contenedor tecnológico, logrando de esta manera una integración de las diferentes tecnologías.

Asimismo, se considera un diagrama general de arquitectura para el desarrollo del proyecto.

4.6.1 Diagrama general

El diagrama presenta la arquitectura del sistema desplegada sobre la plataforma de nube Microsoft Azure, estructurada principalmente en 3 áreas: los actores externos, la infraestructura en la nube y los servicios externos.

En cuanto a los actores, el sistema tiene en cuenta tres roles principales: el Cliente, quien interactúa con el sistema en la aplicación móvil para realizar pedidos; el Repartidor, quien utiliza la misma aplicación móvil para recibir y gestionar las entregas; y el Administrador, quien accede al sistema a través del Panel de Administración Web para la gestión general de la plataforma. El desarrollo de estas herramientas corresponde al equipo de desarrollo, quien cuenta con el repositorio en GitHub.

Dentro de Microsoft Azure, el sistema se organiza en los siguientes componentes:

- Panel de Administración Web: alojado en Azure Static Web Apps, servicio que proporciona hospedaje estático con disponibilidad global.
- Núcleo del sistema: Reside en Azure Container Apps, donde se ejecutan dos contenedores gestionados mediante Docker Compose: el primero corresponde al backend desarrollado en Spring Boot, el cual concentra la lógica de negocio, la API REST, la comunicación en tiempo real mediante WebSocket con protocolo STOMP, el algoritmo de optimización de rutas A* y el módulo de notificaciones; el segundo contenedor aloja la base de datos MariaDB, cuya persistencia de datos se garantiza mediante un volumen montado en Azure File Share.
- Aplicación móvil: Se conecta a Azure Container Apps, que gestiona el tráfico HTTPS y las conexiones WebSocket.

Complementariamente, Azure Container Registry almacena las imágenes Docker generadas por el pipeline de integración y despliegue continuo, compuesto por Azure Pipelines, el cual automatiza el proceso de construcción y despliegue ante cada cambio en el repositorio. La administración y monitoreo del sistema se realizará desde Azure Monitor y Application Insights, que centralizan los registros y métricas de los contenedores en ejecución.

Finalmente, el sistema se apoya en dos servicios externos: la API de Google Maps, utilizada para la visualización del mapa en la aplicación móvil y el renderizado de rutas; y OpenStreetMap (OSM), que provee los datos de la red vial empleados por el algoritmo A* para el cálculo de rutas óptimas de entrega.

A continuación, en la ilustración 6 se presenta el diagrama de arquitectura general:

Comentado [GM6]: Checar el número de las ilustraciones, me sale mal

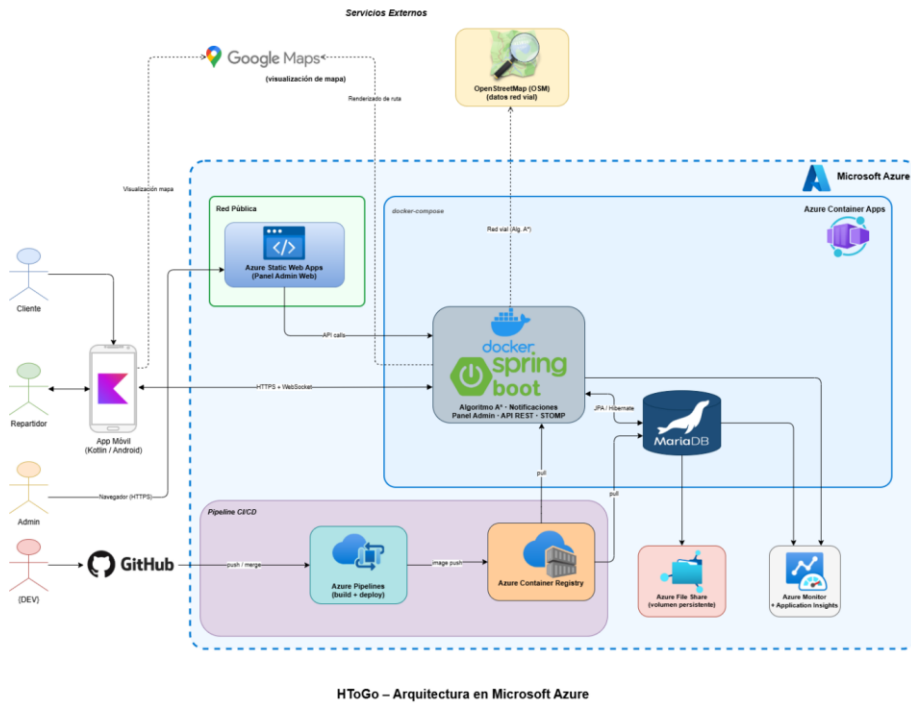


Ilustración 6 Arquitectura general del sistema, montada en Microsoft Azure. Elaboración propia.

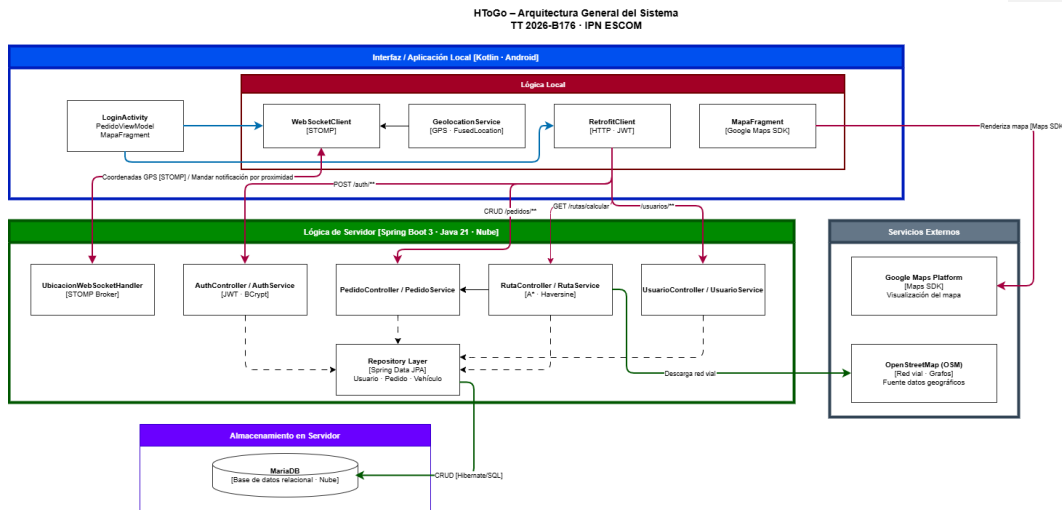


Ilustración 7 Diagrama de arquitectura en bloques

4.6.2 Diagrama de contexto

Para iniciar con la arquitectura del modelo C4, se puede observar el Diagrama de Contexto del sistema HToGo en la ilustración 10, correspondiente al Nivel 1 del modelo. En este nivel se identifica cómo los distintos actores y sistemas externos se relacionan con la plataforma [115]. Los componentes identificados son los siguientes:

- **Ciente:** Persona ubicada dentro de la Alcaldía Benito Juárez que solicita la entrega de garrafones de agua potable a través de la aplicación móvil. Tiene la capacidad de rastrear la ubicación del repartidor, recibir notificaciones de proximidad, gestionar el historial de sus pedidos y programar pedidos.
- **Repartidor:** Persona encargada de distribuir los garrafones. Utiliza la aplicación móvil para visualizar los pedidos disponibles, obtener la ruta optimizada hacia los domicilios de entrega y actualizar el estado de cada pedido.
- **Super Administrador:** Persona responsable de la gestión global del sistema. Tiene la facultad de dar de alta y de baja a usuarios y repartidores, así como de supervisar el correcto funcionamiento de la plataforma.
- **Aplicación Móvil HToGo:** Sistema central que orquesta la comunicación entre los actores. Incluye la interfaz de usuario para clientes y repartidores, la lógica de negocio expuesta mediante servicios REST, el módulo de enrutamiento con el algoritmo A*, y el canal de comunicación mediante WebSockets.

- **API de Google Maps:** Servicio externo utilizado exclusivamente como capa de presentación cartográfica. Proporciona la visualización del mapa y la posibilidad de exportar coordenadas hacia la aplicación nativa de navegación del repartidor.
- **OpenStreetMap (OSM):** Fuente de datos geográficos abiertos. Proporciona la red vial de la Alcaldía Benito Juárez en forma de grafos (nodos y aristas) que el algoritmo A* consume internamente para calcular rutas óptimas, sin depender de otros servicios.

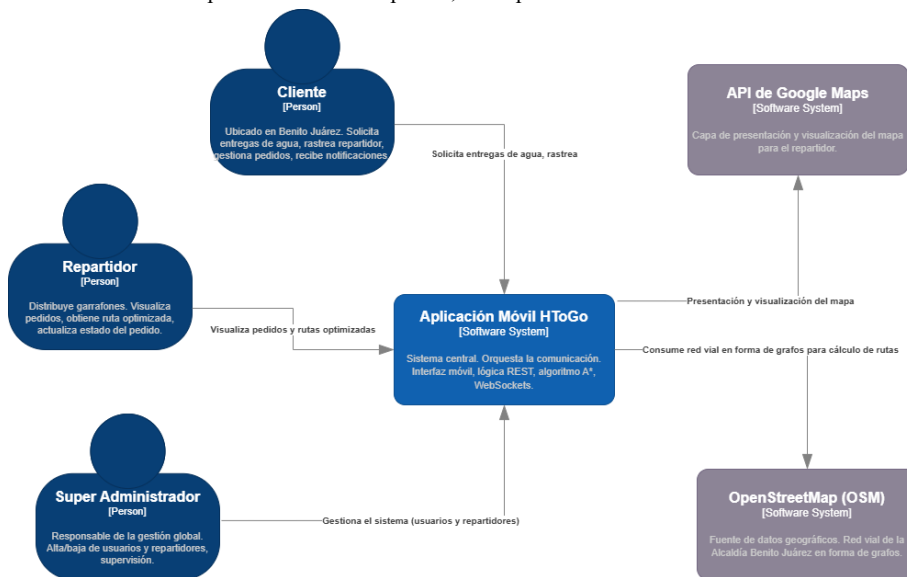


Ilustración 8 Diagrama de Contexto (C4 – Nivel 1). Elaboración propia.

La aplicación HToGo actúa como el núcleo del sistema, interactuando directamente con los tres roles de usuario (cliente, repartidor y super administrador) y consumiendo dos servicios externos. El uso de ambas APIs de mapas se debe a una decisión técnica, donde OpenStreetMap garantiza la libertad de acceso a los datos necesarios del mapa para ejecutar el algoritmo A*, a la vez que Google Maps ofrece una visualización para el usuario repartidor.

4.6.3 Diagrama de Contenedores (Nivel 2)

El Diagrama de Contenedores pertenece al Nivel 2 del modelo C4. En este nivel se representa la aplicación, donde cada contenedor es una generalidad de la tecnología a utilizar en cada acción necesaria del sistema [116]. Los contenedores identificados en el sistema son:

- **Aplicación Móvil (Kotlin/Android):** Interfaz de usuario nativa para Android, desarrollada en Kotlin. Es el punto de entrada para los actores cliente y repartidor. Se comunica con el backend mediante dos canales distintos: solicitudes HTTP REST para las operaciones de negocio (registro, login, gestión de pedidos, historial) y una conexión WebSocket persistente para el intercambio de coordenadas GPS en tiempo real, evitando la latencia de las peticiones HTTP convencionales y la sobrecarga sobre la base de datos.

- **Backend Spring Boot (Nube):** Servidor de aplicaciones desplegado en la nube, desarrollado con el framework Spring Boot sobre Java. Expone una API REST que atiende las peticiones de la aplicación móvil y un servidor de WebSockets implementado con el protocolo STOMP para la retransmisión de coordenadas en tiempo real. Adicionalmente, contiene el módulo del algoritmo A* que procesa los datos de la red vial obtenidos de OpenStreetMap para generar rutas óptimas.
- **Base de Datos MariaDB (Nube):** Sistema gestor de base de datos relacional desplegado en la nube, bajo licencia GNU GPL. Almacena de manera persistente la información estructurada del sistema: datos de usuarios y repartidores, pedidos y su ciclo de vida (con marca de tiempo), vehículos registrados, inventarios y el historial de cambios de estado. El backend accede a ella exclusivamente a través del ORM de Spring Data JPA.
- **API Google Maps (Externo):** Servicio externo de Google utilizado para renderizar el mapa en la aplicación móvil del repartidor y del cliente. Recibe la geometría de la ruta calculada internamente por el algoritmo A* (en formato de polilínea) para proyectarla visualmente, optimizando el consumo de la cuota de la API.
- **OpenStreetMap (Externo):** Fuente de datos cartográficos abiertos. El backend consume los datos de la red vial de la Alcaldía Benito Juárez (nodos y aristas del grafo) para alimentar el algoritmo A*. Al ser datos descargables y procesables localmente, reduce la dependencia de servicios externos durante el cálculo de rutas.

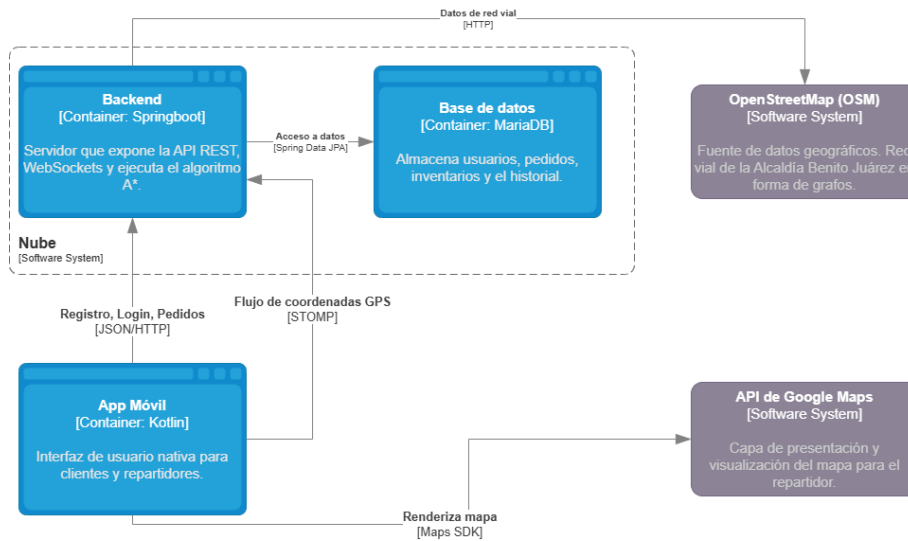


Ilustración 9 Diagrama de Contenedores (C4 – Nivel 2). Elaboración propia.

4.6.4 Diagrama de Componentes (Nivel 3)

El Nivel 3 del modelo C4 detalla los componentes internos de cada contenedor principal. A continuación, se describen los componentes del backend Spring Boot y de la aplicación móvil Kotlin.

4.6.4.1 Componentes del Backend (Spring Boot)

La Figura presenta los componentes internos del backend. El backend sigue una arquitectura en capas (Controller → Service → Repository), complementada con un módulo de WebSockets y un módulo de algoritmos de enrutamiento:

- **AuthController / AuthService:** Gestiona el registro e inicio de sesión de usuarios y repartidores. Valida credenciales contra la base de datos, verifica el correo y teléfono del usuario y emite tokens para la autenticación de sesiones posteriores. Aplica el cifrado de contraseñas (RN018) mediante BCrypt.
- **PedidoController / PedidoService:** Expone los endpoints REST para las operaciones del ciclo de vida de un pedido: creación (RF-001), cancelación (RF-013), cambio de estado y consulta de historial (RF-010). Valida el horario de disponibilidad (RF-011) y el límite de carga del repartidor (RF-012).
- **RutaController / RutaService:** Recibe las coordenadas de origen (posición actual del repartidor) y los destinos de entrega. Consulta los datos de la red vial de OpenStreetMap y ejecuta el algoritmo A* con la heurística de Haversine para calcular la ruta óptima. Devuelve la ruta resultante a la aplicación móvil y la almacena en caché (RN014) para evitar peticiones redundantes.
- **UbicacionWebSocketHandler:** Implementa el broker de mensajería STOMP para el intercambio de coordenadas en tiempo real. Recibe las actualizaciones GPS del repartidor, evalúa la proximidad mediante la fórmula de Haversine y retransmite las coordenadas al cliente. Además, llama al servicio de notificaciones cuando la distancia es menor a 500 metros (RN010).
- **NotificacionService:** Administra el envío de notificaciones push hacia el cliente. Actúa como consumidor de eventos del UbicacionWebSocketHandler y del PedidoService (cambios de estado).
- **UsuarioController / UsuarioService:** Permite al super administrador gestionar el alta y baja de cuentas (RF-009), aprobar cambios de perfil del repartidor (RN020) y consultar el estado de las sesiones activas.

- **Repository Layer (Spring Data JPA):** Capa de acceso a datos. Contiene los repositorios JPA para las entidades principales: Usuario, Repartidor, Pedido, Vehiculo, Inventario y EstadoPedido. Se comunica con MariaDB a través de Hibernate..

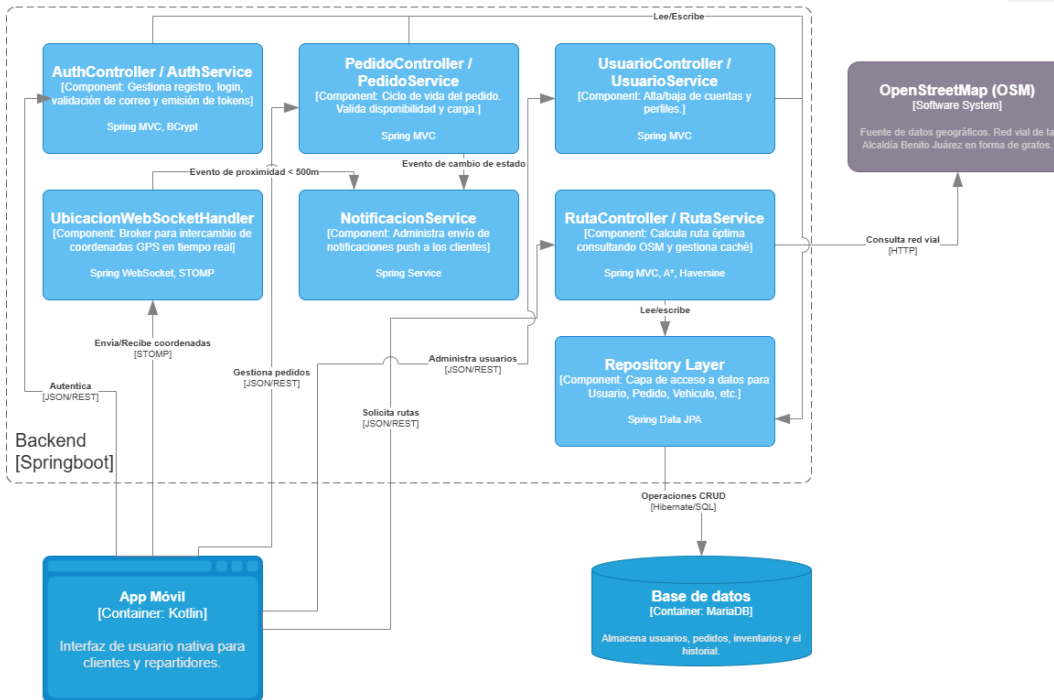


Ilustración 10 Diagrama de componentes del Backend Spring Boot (C4 – Nivel 3). Elaboración propia

4.6.4.2 Componentes de la Aplicación Móvil (Kotlin/Android)

A continuación, los componentes internos de la aplicación desarrollada con Kotlin siguiendo el patrón de arquitectura MVVM (Model-View-ViewModel):

- **LoginViewModel / LoginActivity:** Gestiona el flujo de autenticación. Envía las credenciales al AuthController del backend, recibe y almacena el token de forma segura, y verifica que no exista una sesión activa en otro dispositivo (RF-015).
- **PedidoViewModel:** Contiene la lógica de presentación para la gestión de pedidos. Se comunica con el PedidoController del backend a través del cliente HTTP para crear, cancelar y consultar pedidos.
- **MapaFragment:** Componente visual responsable de renderizar el mapa, mostrar la posición del repartidor en tiempo real (actualizando el marcador con cada coordenada recibida por WebSocket), trazar la ruta calculada por A* y diferenciar el estado de los pedidos (RF-007).
- **WebSocketClient (STOMP):** Cliente WebSocket implementado con la librería. Establece una conexión persistente con el UbicaciónWebSocketHandler del backend. En el caso del repartidor, publica sus coordenadas GPS. En el caso del cliente, se suscribe al evento del pedido para recibir actualizaciones de la posición del repartidor.

- **GeolocationService:** Servicio en segundo plano (Android Foreground Service) que accede al GPS del dispositivo. Manda las coordenadas al WebSocketClient del repartidor de forma periódica durante la entrega.
- **NotificacionReceiver:** Muestra la notificación al cliente cuando el repartidor se encuentra a 500 metros de su domicilio (RN010) y cuando el estado de su pedido cambia.

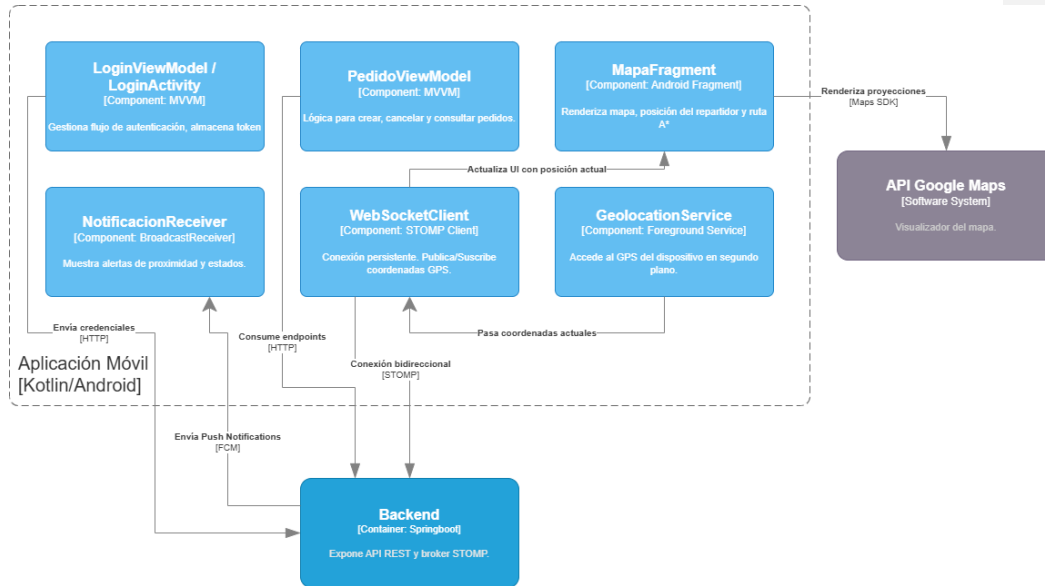


Ilustración 11 Diagrama de componentes de la Aplicación Móvil (C4 – Nivel 3). Elaboración propia.

4.6.4.3 Componentes de la Base de Datos (MariaDB)

La base de datos MariaDB, la cual estará alojada en la nube, utiliza un esquema relacional normalizado para garantizar la integridad de los datos. Los principales componentes lógicos del contenedor de base de datos son:

- **Esquema Relacional:** Conjunto de tablas que representan las entidades del dominio: Usuario, Repartidor, Pedido, EstadoPedido (historial inmutable con timestamp), Vehículo, Inventario y Dirección. Las relaciones entre tablas se definen mediante claves foráneas con integridad referencial activada.
- **Motor de Integridad y Transacciones (InnoDB):** Motor de almacenamiento de MariaDB que garantiza las propiedades ACID en cada operación.

- **Gestor de Acceso (Spring Data JPA / Hibernate):** Es parte del backend, este componente actúa como la interfaz entre el backend y MariaDB, asegurando que ningún componente acceda directamente a la base de datos. Gestiona el pool de conexiones.

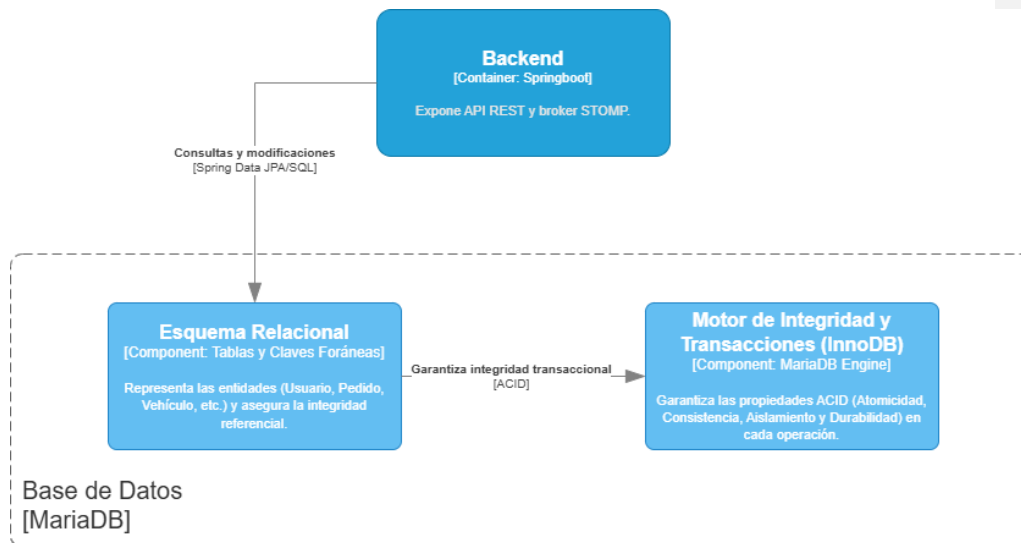
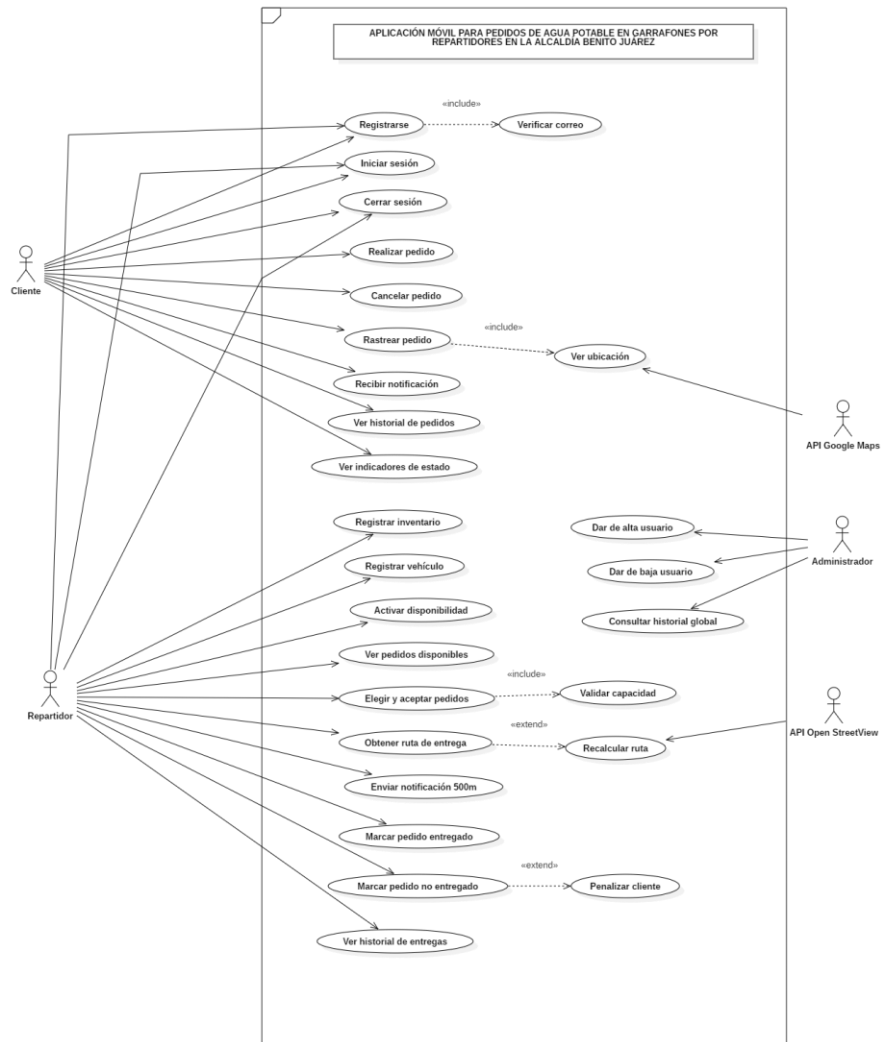


Ilustración 12 Diagrama de componentes de la Base de Datos (C4 – Nivel 3). Elaboración propia

4.7 Casos de uso

El diagrama de casos de uso UML es la principal forma de determinar los requisitos del sistema, ya que estos nos explican el comportamiento que debe de tener nuestro sistema en varios escenarios. Los casos de uso se desarrollan con el propósito de detallar un conjunto de requisitos centrados en tareas o roles específicos [117].

Los diagramas nos ayudan en diferentes escenarios desde que se está comenzado el proyecto, hasta la fase de pruebas, al inicio se usan como ideas y recopilación de requisitos para cumplir con las necesidades del sistema, posteriormente la fase de análisis nos ayuda a identificar las clases que necesitara el sistema, y finalmente en las pruebas para seguir los flujos y verificar si cumple con los requerimientos. Mientras los casos de uso describen qué acciones realizan los usuarios dentro del sistema, los procesos de negocio explican cómo se llevan a cabo dichas acciones [118].



CU - 001: Registrarse

Nombre	Registrarse
Actores	Cliente, Repartidor
Descripción	El sistema permite a nuevos usuarios (clientes o repartidores) crear una cuenta proporcionando sus datos personales. Al completar el registro, el sistema valida que no existan duplicados, cifra la contraseña y envía un correo de verificación para activar la cuenta.
Requerimiento funcional	RF - 002 RF - 014
Reglas de negocio	RN-016: No se permite duplicidad de correo o teléfono. RN-018: Cifrado obligatorio de contraseña. RN-002: Verificación de correo para activar cuenta
Disparador	El usuario accede a la pantalla de registro desde la página de inicio de sesión y selecciona "Crear cuenta".
Precondiciones	1. La comunidad debe estar conectada a Internet. 2. El usuario no debe tener una cuenta previamente registrada con el mismo correo o teléfono. 3. La aplicación debe estar instalada y operativa.
Postcondiciones	1. Se crea el registro del usuario en la base de datos con el rol correspondiente (cliente o repartidor). 2. La contraseña queda almacenada en forma de hash cifrado. 3. Se envía un correo de verificación a la dirección proporcionada. 4. La cuenta permanece inactiva hasta que se verifique el correo.
Flujo Normal	1. El usuario abre la aplicación y selecciona "Crear cuenta". 2. El sistema muestra el formulario de registro (nombre, correo, teléfono, contraseña, tipo de usuario). 3. El usuario completa todos los campos y confirma. 4. El sistema valida que el correo y teléfono no estén previamente registrados. 5. El sistema cifra la contraseña con el algoritmo de hash definido. 6. El sistema crea el registro del usuario en la base de datos. 7. El sistema envía un correo electrónico de verificación al usuario. 8. El sistema muestra un mensaje de éxito indicando que debe verificar su correo. 9. Finaliza el caso de uso.
Flujos Alternativos	S1: No hay banners disponibles para la visualización • S1.1 El sistema detecta que el correo o teléfono ya está registrado. • S1.2 El sistema muestra un mensaje indicando el campo duplicado. • S1.3 El usuario corrige el dato y reintenta. Regresa al paso 4. Finaliza el caso de uso S1: El usuario no verifica su correo • S2.1 La cuenta queda en estado inactivo. • S2.2 Al intentar iniciar sesión, el sistema bloquea el acceso hasta verificación. Finaliza el caso de uso.
Excepciones	E1: Error de conexión al servidor.

	• E1.1 El sistema muestra un mensaje de error de red. • E1.2 Invita al usuario a reintentar. Regresa al paso 3. E1: Error de conexión al servidor. • E2.1 El sistema registra la cuenta, pero notifica el fallo del correo. • E2.2 Ofrece la opción de reenviar el correo de verificación.
Prioridad	Alta
Frecuencia de uso	Media
Requerimientos especiales	El correo de verificación debe enviarse en menos de 2 minutos. La contraseña debe tener mínimo 8 caracteres
Datos de entrada	○ Nombre completo ○ Correo electrónico ○ Número de teléfono ○ Contraseña
Datos de salida	○ Mensaje de éxito con instrucción de verificar correo. ○ Correo de verificación enviado a la dirección proporcionada.
Suposiciones	El usuario proporciona datos correctos. El servicio de correo electrónico está operativo.
Notas y preguntas	*: La validación de duplicados se realiza mediante consulta directa a la base de datos antes de crear el registro.

CU – 002: Iniciar sesión

Nombre	Iniciar sesión
Actores	Cliente, Repartidor
Descripción	El sistema permite a los usuarios registrados y verificados acceder a la aplicación con sus credenciales (correo electrónico y contraseña). El sistema valida la sesión única por dispositivo y genera un token de autenticación con vigencia definida
Requerimiento funcional	RF - 003 RF - 015
Reglas de negocio	RN-019: Solo se permite una sesión activa por cuenta y dispositivo. RNF-004: El token caduco tras 30 días de inactividad.
Disparador	El usuario abre la aplicación e ingresa sus credenciales en la pantalla de inicio de sesión.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El correo electrónico debe estar verificado. 3. El dispositivo debe tener conexión a Internet. 4. No debe existir una sesión activa del mismo usuario en otro dispositivo.
Postcondiciones	1. Se genera un token de sesión asociado al dispositivo del usuario. 2. El usuario accede al menú principal dependiendo de su rol (cliente o repartidor). 3. El sistema registra la fecha y hora del último inicio de sesión.
Flujo Normal	1. El usuario abre la aplicación y visualiza la pantalla de inicio de sesión. 2. El usuario ingresa su correo electrónico y contraseña. 3. El sistema valida que las credenciales sean correctas en la base de datos. 4. El sistema verifica que no exista sesión activa en otro dispositivo. (RF-015, RN-019) 5. El sistema genera un token de sesión con vigencia de 30 días. (RNF-004) 6. El sistema redirige al usuario al menú principal según su rol. 7. Finaliza el caso de uso.
Flujos Alternativos	S1: Sesión activa en otro dispositivo • S1.1 El sistema detecta un token activo en otro dispositivo. • S1.2 El sistema muestra un aviso al usuario. • S1.3 El usuario puede elegir cerrar la sesión anterior o cancelar. • S1.4 Regresa al paso 3 si cancela, o continúa al paso 5 si cierra la sesión anterior. Finaliza el caso de uso. S2: El correo no está verificado • S2.1 El sistema bloquea el acceso y muestra un mensaje de verificación pendiente. • S2.2 El sistema ofrece la opción de reenviar el correo de verificación. Finaliza el caso de uso.

Excepciones	E1: Credenciales incorrectas. • E1.1 El sistema muestra un mensaje genérico de "Correo o contraseña incorrectos". • E1.2 El usuario puede reintentar. Regresa al paso 2. E2: Error de conexión al servidor. • E2.1 El sistema muestra un mensaje de error de red e invita a reintentar.
Prioridad	Alta
Frecuencia de uso	Alta
Requerimientos especiales	El sistema no debe mostrar si el correo existe o no (mensaje genérico por seguridad). El token de sesión debe almacenarse de forma segura en el dispositivo.
Datos de entrada	○ Correo electrónico del usuario registrado. ○ Contraseña del usuario registrado.
Datos de salida	○ Token de sesión generado y almacenado en el dispositivo. ○ Redirección al menú principal según el rol del usuario.
Suposiciones	El usuario proporciona datos correctos. El servicio de correo electrónico está funcionando.
Notas y preguntas	La validación de duplicados se realiza mediante consulta directa a la base de datos antes de crear el registro.

CU – 003: Cerrar sesión

Nombre	Cerrar sesión
Actores	Cliente, Repartidor
Descripción	El sistema permite al usuario autenticado cerrar su sesión actual de forma segura, eliminando el token de sesión del dispositivo y liberando la cuenta para acceso desde otro dispositivo o en otro momento.
Requerimiento funcional	RF - 003 RF - 015
Reglas de negocio	RN-019: La sesión activa en el dispositivo queda invalidada al cerrar sesión.
Disparador	El usuario selecciona la opción "Cerrar sesión" desde el menú de configuración o perfil de la aplicación.
Precondiciones	1. El usuario debe tener una sesión activa en la aplicación. 2. El dispositivo debe tener conexión a Internet.
Postcondiciones	1. El token de sesión es eliminado del dispositivo y del servidor. 2. El usuario es redirigido a la pantalla de inicio de sesión. 3. La cuenta queda disponible para iniciar sesión desde cualquier dispositivo.
Flujo Normal	1. El usuario accede al menú de configuración y selecciona "Cerrar sesión". 2. El sistema muestra un cuadro de confirmación. 3. El usuario confirma la acción. 4. El sistema invalida el token de sesión en el servidor. (RN-019) 5. El sistema elimina el token almacenado en el dispositivo. 6. El sistema redirige al usuario a la pantalla de inicio de sesión. 7. Finaliza el caso de uso.
Flujos Alternativos	S1: El usuario cancela la confirmación • S1.1 El sistema cierra el cuadro de confirmación. • S1.2 El usuario permanece en la pantalla actual. Finaliza el caso de uso
Excepciones	E1: Error de conexión al cerrar sesión. • E1.1 El sistema elimina el token local aunque falle la comunicación con el servidor. • E1.2 El token en el servidor expirará de forma automática tras el período de inactividad.
Prioridad	Media
Frecuencia de uso	Media
Requerimientos especiales	El cierre de sesión debe funcionar, aunque no haya conexión a Internet, eliminando el token local.
Datos de entrada	○ Confirmación de cierre de sesión
Datos de salida	○ Token de sesión eliminado del dispositivo y del servidor. ○ Redirección a la pantalla de inicio de sesión.
Suposiciones	El usuario tiene intención de liberar el dispositivo o proteger su cuenta.

Notas y preguntas	El cierre de sesión por inactividad (30 días) es automático y no requiere acción del usuario.
-------------------	---

CU – 004: Realizar pedido

Nombre	Realizar pedido
Actores	Cliente
Descripción	El cliente registrado y verificado puede realizar un pedido de garrafones de agua potable especificando la cantidad y confirmando su domicilio de entrega. El sistema valida la zona de cobertura, el horario de atención y registra el pedido en estado "Pendiente".
Requerimiento funcional	RF - 001 RF - 011
Reglas de negocio	RN-001: Solo se aceptan pedidos dentro de la Alcaldía Benito Juárez. RN-003: La dirección debe tener descripción y coordenadas GPS. RN-004: Pedidos solo entre 08:00 y 18:00 hrs; fuera del horario se programan para el día siguiente. RN-013: Cada cambio de estado genera un registro.
Disparador	El cliente accede a la aplicación, inicia sesión y selecciona la opción "Hacer pedido" desde el menú principal
Precondiciones	1. El cliente debe tener sesión activa. 2. La cuenta debe estar verificada y activa. 3. El cliente no debe estar suspendido por ausencias previas (RN-006). 4. El domicilio de entrega debe estar dentro de la Alcaldía Benito Juárez.
Postcondiciones	1. El pedido queda registrado en la base de datos con estado "Pendiente". 2. Se genera un registro. 3. El pedido es visible para los repartidores activos.
Flujo Normal	1. El cliente selecciona "Hacer pedido" en el menú principal. 2. El sistema muestra el formulario de pedido (cantidad de garrafones, domicilio, descripción de entrega). 3. El cliente ingresa la cantidad de garrafones y confirma o agrega su dirección. 4. El sistema valida que la dirección se encuentre dentro de la Alcaldía Benito Juárez (*). (RN-001) 5. El sistema valida que el pedido se realice entre las 08:00 y las 18:00 hrs. (RF-011, RN-004) 6. El sistema registra el pedido con estado "Pendiente" y genera el timestamp. (RN-013) 7. El sistema guarda la información del pedido en la base de datos 8. El sistema muestra al cliente la confirmación del pedido y su número de seguimiento. 9. Finaliza el caso de uso.
Flujos Alternativos	S1: Domicilio fuera de la zona de cobertura. • S1.1 El sistema muestra el mensaje "Dirección fuera de zona de cobertura". • S1.2 El cliente puede modificar la dirección. Regresa al paso 3. Finaliza el caso de uso S2: Pedido fuera de horario de atención. • S2.1 El sistema informa que el horario de atención es de 08:00 a 18:00 hrs.

	• S2.2 El sistema ofrece programar el pedido para el siguiente día hábil. (RF-011) • S2.3 El cliente acepta y el pedido se agenda para el día siguiente. Finaliza el caso de uso S3: El cliente tiene cuenta suspendida. • S3.1 El sistema muestra el mensaje de suspensión temporal. (RN-006) • S3.2 No permite realizar el pedido. Finaliza el caso de uso.
Excepciones	E1: El enlace fue usado. • E1.1 El sistema muestra un mensaje de error. • E1.2 El sistema invita al usuario a reintentar. Regresa al paso 6.
Prioridad	Alta
Frecuencia de uso	Alta
Requerimientos especiales	El pedido debe quedar registrado tras la confirmación del cliente.
Datos de entrada	○ Cantidad de garrafones solicitados ○ Marca del producto ○ Dirección de entrega
Datos de salida	○ Pedido registrado en estado “Pendiente” con timestamp. ○ Número de seguimiento mostrado al cliente.
Suposiciones	El cliente conoce la dirección exacta de entrega. La función de GPS del dispositivo está habilitada.
Notas y preguntas	La validación de zona utiliza las coordenadas GPS del domicilio para comparar contra los límites geográficos de la Alcaldía Benito Juárez.

CU – 005: Cancelar pedido

Nombre	Cancelar pedido
Actores	Cliente
Descripción	El cliente puede cancelar un pedido únicamente cuando su estado es "Pendiente" o "Asignado". Una vez que el pedido pasa a estado "En camino", la opción de cancelación se oculta automáticamente. Al cancelar, el repartidor asignado es liberado y la ruta se recalcula si aplica.
Requerimiento funcional	RF - 013
Reglas de negocio	RN-005: Solo se puede cancelar en estado Pendiente o Asignado. RN-009: El sistema recalcula las rutas del repartidor si cancela antes de iniciarse la ruta.
Disparador	El cliente accede al detalle de su pedido activo y selecciona el botón "Cancelar pedido", que solo es visible en estados permitidos.
Precondiciones	1. El cliente tiene sesión activa. 2. El pedido debe estar en estado "Pendiente" o "Asignado". 3. El botón "Cancelar pedido" debe estar habilitado y visible en la interfaz
Postcondiciones	1. El pedido queda registrado con estado "Cancelado" en la base de datos. 2. Se genera un registro histórico con timestamp del evento. (RN-013) 3. Si el pedido tenía repartidor asignado, el repartidor queda liberado para nuevos pedidos. 4. Si el repartidor tenía múltiples pedidos activos, el sistema recalcula la ruta.
Flujo Normal	1. El cliente accede a "Mis pedidos" y selecciona el pedido activo. 2. El sistema muestra el detalle del pedido con el botón "Cancelar pedido" habilitado (estado Pendiente o Asignado). 3. El cliente selecciona "Cancelar pedido". 4. El sistema muestra un cuadro de confirmación. 5. El cliente confirma la cancelación. 6. El sistema valida que el estado del pedido siga siendo cancelable. (RN-005) 7. El sistema actualiza el estado del pedido a "Cancelado". 8. El sistema libera al repartidor y, si aplica, recalcula la ruta. (RN-009) 9. El sistema notifica al repartidor la cancelación. 10. El sistema muestra al cliente un mensaje de confirmación de cancelación. 11. Finaliza el caso de uso.
Flujos Alternativos	S1: El pedido cambió a estado "En camino" antes de confirmar la cancelación. • S1.1 El sistema detecta que el estado ya no es cancelable. • S1.2 El sistema oculta el botón y muestra un mensaje indicando que el pedido ya no puede cancelarse. Finaliza el caso de uso. S2: El cliente cancela el cuadro de confirmación. • S2.1 El sistema cierra el cuadro sin realizar cambios.

	• S2.2 El pedido permanece en su estado original. Finaliza el caso de uso.
Excepciones	E1: Error al actualizar el estado en la base de datos. • E1.1 El sistema muestra un mensaje de error. • E1.2 El sistema invita al usuario a reintentar. Regresa al paso 6.
Prioridad	Alta
Frecuencia de uso	Media
Requerimientos especiales	El botón debe ocultarse automáticamente en tiempo real cuando el estado del pedido cambia a "En camino".
Datos de entrada	○ Pedido registrado en estado "Pendiente" con timestamp. ○ Número de seguimiento mostrado al cliente.
Datos de salida	○ Pedido actualizado a estado "Cancelado" con timestamp. ○ Notificación enviada al repartidor. Ruta recalculada si aplica.
Suposiciones	El cliente tiene acceso a Internet al momento de cancelar
Notas y preguntas	El sistema debe manejar la condición de carrera donde el estado puede cambiar entre que el cliente selecciona cancelar y confirma la acción.

CU – 006: Rastrear pedido en mapa

Nombre	Rastrear pedido en mapa
Actores	Cliente
Descripción	El cliente puede visualizar la ubicación actual del repartidor sobre un mapa interactivo, así como el estado actual de su pedido mediante indicadores visuales (colores e íconos). Solo se visualizan los datos asociados al ID del cliente autenticado.
Requerimiento funcional	RF – 006 RF – 007
Reglas de negocio	RN-017: El cliente solo visualiza información de sus propios pedidos. RN-014: La ruta del repartidor se obtiene de la caché cuando está disponible.
Disparador	El cliente accede a la sección "Mis pedidos" y selecciona un pedido en estado "Asignado" o "En camino".
Precondiciones	1. El cliente tiene sesión activa. 2. El pedido debe estar en estado "Asignado" o "En camino". 3. El repartidor debe tener la ubicación GPS habilitada. 4. La API de OpenStret debe estar disponible.
Postcondiciones	1. El cliente visualiza la ubicación del repartidor actualizada en el mapa. 2. El cliente puede ver el indicador de estado del pedido en tiempo real.
Flujo Normal	1. El cliente accede a "Mis pedidos" y selecciona el pedido activo. 2. El sistema muestra la pantalla de seguimiento con el mapa. (RF-006) 3. El sistema obtiene la ubicación GPS actual del repartidor. 4. El sistema muestra la posición del repartidor con un marcador en el mapa. 5. El sistema muestra el indicador de estado del pedido con color e ícono correspondiente. (RF-007) 6. La ubicación del repartidor se actualiza periódicamente en el mapa. 7. El cliente sigue el movimiento del repartidor hasta la entrega. 8. Finaliza el caso de uso al completarse la entrega o al cerrar la pantalla. 9. Finaliza el caso de uso
Flujos Alternativos	S1: El repartidor tiene el GPS desactivado. • S1.1 El sistema muestra un mensaje indicando que la ubicación del repartidor no está disponible. • S1.2 El sistema sigue mostrando el estado textual del pedido. • Finaliza el caso de uso
Excepciones	E1: La API de Google Maps no está disponible. • E1.1 El sistema muestra un mensaje de error indicando que el mapa no puede cargarse. • E1.2 Se muestra únicamente el estado textual del pedido.
Prioridad	Alta
Frecuencia de uso	Alta

Requerimientos especiales	La actualización de la ubicación debe realizarse como máximo cada 10 segundos. El mapa debe tener funciones como: zoom, desplazamiento.
Datos de entrada	○ Pedido a seguir ○ Identificador del pedido activo en estado “Asignado” o “En camino”
Datos de salida	○ Visualización en mapa de la ubicación actual del repartidor actualizada periódicamente. ○ Indicador de estado del pedido en tiempo real.
Suposiciones	El repartidor tiene el GPS del dispositivo habilitado durante la entrega.
Notas y preguntas	Ninguna.

CU – 007: Ver historial de pedidos

Nombre	Ver historial de pedidos
Actores	Cliente
Descripción	El cliente puede consultar el historial completo de todos sus pedidos anteriores, visualizando el estado final, la fecha, la cantidad de garraones y el repartidor asignado de cada uno. El sistema filtra los registros exclusivamente por el ID del cliente autenticado.
Requerimiento funcional	RF – 010
Reglas de negocio	RN-017: El cliente solo puede ver los pedidos asociados a su propio ID. RN-013: Los registros históricos son inmutables (no se pueden borrar ni editar).
Disparador	El cliente selecciona la sección "Mis pedidos" desde el menú principal de la aplicación.
Precondiciones	1. El cliente tiene sesión activa. 2. El cliente debe tener al menos un pedido registrado en el sistema
Postcondiciones	1. El cliente visualiza la lista de sus pedidos con sus respectivos detalles. 2. No se modifica ningún dato en la base de datos.
Flujo Normal	1. El cliente selecciona "Historial" en el menú principal. 2. El sistema consulta en la base de datos todos los pedidos asociados al ID del cliente. 3. El sistema ordena los pedidos de más reciente a más antiguo. 4. El sistema muestra la lista con: fecha, cantidad de garraones, estado y repartidor asignado. 5. El cliente puede seleccionar cualquier pedido para ver su detalle completo. 6. El sistema muestra el detalle: dirección, de cada cambio de estado, total. 7. Finaliza el caso de uso.
Flujos Alternativos	S1: El cliente no tiene pedidos. • S1.1 El sistema muestra un mensaje donde se le indica que aún no ha realizado algún pedido. Finaliza el caso de uso
Excepciones	E1: Error al consultar la base de datos. • E1.1 El sistema muestra un mensaje de error. • E1.2 Invita al usuario a reintentar más tarde.
Prioridad	Media
Frecuencia de uso	Baja
Requerimientos especiales	La consulta no debe de tardar mucho al ejecutarse. Los registros son de solo lectura, no se permite editar ni eliminar desde esta vista los datos de la base de datos.
Datos de entrada	○ Identificador del cliente autenticado (sesión activa). ○ Identificador de los pedidos.
Datos de salida	○ Lista de pedidos de la cliente ordenada de más reciente a más antiguo. ○ Fecha.

	o Cantidad de garraones, estado y repartidor asignado.
Suposiciones	La base de datos mantiene el historial completo de forma permanente.
Notas y preguntas	Los registros históricos incluyen los timestamps de cada cambio de estado del pedido, lo que permite ver el ciclo de vida completo de cada entrega.

CU – 008: Activar disponibilidad

Nombre	Registrar inventario de garrafones
Actores	Repartidor
Descripción	El repartidor debe declarar en la aplicación la cantidad de garrafones disponibles en su vehículo antes de iniciar su jornada. Este inventario determina la capacidad máxima de pedidos que puede aceptar durante el día y habilita su disponibilidad para recibir pedidos.
Requerimiento funcional	RF - 008 RF - 012
Reglas de negocio	RN-007: El repartidor debe registrar su inventario antes de poder elegir pedidos. RN-008: No se asignarán pedidos si la suma de garrafones supera la capacidad registrada del vehículo.
Disparador	El repartidor inicia sesión y accede a la sección "Iniciar jornada" o "Registrar inventario" antes de comenzar a recibir pedidos.
Precondiciones	1. El repartidor tiene sesión activa. 2. El repartidor tiene al menos un vehículo registrado en el sistema. (RN-015) 3. El repartidor debe estar en estado "Inactivo" (no tener jornada en curso).
Postcondiciones	1. El inventario queda registrado y asociado al vehículo activo del repartidor. 2. El sistema habilita la disponibilidad del repartidor para recibir pedidos. 3. La capacidad máxima queda definida para la jornada del día.
Flujo Normal	1. El repartidor accede a "Iniciar jornada" desde el menú principal. 2. El sistema muestra los vehículos registrados del repartidor. 3. El repartidor selecciona el vehículo activo para la jornada. 4. El sistema muestra el formulario de inventario con la capacidad máxima del vehículo seleccionado. 5. El repartidor ingresa la cantidad actual de garrafones disponibles. 6. El sistema valida que la cantidad ingresada no supere la capacidad máxima del vehículo. (RN-008) 7. El sistema guarda el inventario y activa la disponibilidad del repartidor. (RN-007) 8. El sistema redirige al repartidor a la vista de pedidos disponibles. 9. Finaliza el caso de uso.
Flujos Alternativos	S1: La cantidad ingresada supera la capacidad del vehículo • S1.1 El sistema muestra un mensaje indicando la capacidad máxima del vehículo. • S1.2 El repartidor corrige la cantidad. Regresa al paso 5. Finaliza caso de uso
Excepciones	E1: Error al intentar registrar el inventario. • E1.1 El sistema muestra un mensaje de error. • E1.2 El repartidor no queda habilitado hasta que el inventario se guarde correctamente. • E1.3 El sistema invita a reintentar. Regresa al paso 5.
Prioridad	Alta

Frecuencia de uso	Alta
Requerimientos especiales	El inventario debe actualizarse automáticamente conforme el repartidor acepta y entrega pedidos
Datos de entrada	○ Selección del vehículo activo. ○ Cantidad de garrafrones disponibles en el vehículo.
Datos de salida	○ Inventario registrado y asociado al vehículo. ○ Disponibilidad del repartidor habilitada. ○ Redirección a la vista de pedidos disponibles.
Suposiciones	El repartidor conoce la cantidad exacta de garrafrones que carga en su vehículo al iniciar la jornada.
Notas y preguntas	La capacidad máxima del vehículo es un dato fijo registrado al dar de alta el vehículo. El inventario disponible va disminuyendo conforme se aceptan pedidos

CU – 009: Registrar vehículo

Nombre	Registrar vehículo
Actores	Repartidor, Administrador
Descripción	El repartidor registra los vehículos con los que realiza sus entregas, indicando tipo de vehículo, placas y capacidad máxima de garraiones. Los cambios en el nombre o datos del vehículo requieren aprobación del administrador.
Requerimiento funcional	RF – 008
Reglas de negocio	RN-015: El repartidor no puede activarse ni aceptar pedidos si no tiene al menos un vehículo registrado. RN-020: Los cambios en datos del vehículo requieren aprobación del administrador.
Disparador	El repartidor accede a la sección "Mis vehículos" y selecciona "Agregar vehículo".
Precondiciones	1. El repartidor tiene sesión activa. 2. El repartidor no ha registrado un vehículo con las mismas placas. 3. El repartidor conoce la capacidad máxima de garraiones que puede soportar su vehículo.
Postcondiciones	1. El vehículo queda registrado en el sistema con su respectivo repartidor. 2. El repartidor puede seleccionar el vehículo cuando iniciara su jornada. 3. Si es el primer vehículo, el repartidor queda habilitado para activarse.
Flujo Normal	1. El repartidor accede a "Mis vehículos" y selecciona "Agregar vehículo". 2. El sistema muestra la ventana con el formulario de registro (tipo, placas, capacidad máxima de garraiones, foto). 3. El repartidor completa los datos del vehículo. 4. El sistema valida que las placas no estén duplicadas. 5. El sistema guarda el vehículo asociado al repartidor. 6. Si es el primer vehículo registrado, el sistema habilita la opción de activar disponibilidad. 7. El sistema muestra un mensaje de éxito. 8. Finaliza el caso de uso.
Flujos Alternativos	S1: El repartidor intenta modificar los datos de un vehículo existente • S1.1 El sistema envía la solicitud de cambio al administrador. • S1.2 Los datos del vehículo quedan en estado "Pendiente de revisión" hasta aprobación. • S1.3 El vehículo sigue funcionando con los datos anteriores durante la revisión.
Excepciones	E1: Los datos del vehículo ya están registrados. • E1.1 El sistema muestra un mensaje de duplicado. • E1.2 El repartidor corrige los datos. Regresa al paso 3.
Prioridad	Alta

Frecuencia de uso	Baja
Requerimientos especiales	La capacidad del vehículo es un valor que determina el límite de pedidos aceptables.
Datos de entrada	○ Tipo de vehículo, ○ Número de placa, ○ capacidad máxima de garraiones. ○ Fotografía del vehículo (opcional).
Datos de salida	○ Vehículo registrado y asociado al repartidor en el sistema. ○ Habilitación de disponibilidad si es el primer vehículo registrado.
Suposiciones	Los datos del repartidor son datos reales
Notas y preguntas	Los cambios de nombre o foto del vehículo requieren aprobación del administrador para garantizar que el cliente siempre vea información correcta.

CU – 010: Elegir y aceptar pedido

Nombre	Elegir y aceptar pedido
Actores	Repartidor
Descripción	El repartidor activo puede visualizar los pedidos disponibles en su área de y seleccionar cuáles desea aceptar, siempre que la suma total de garrafrones no supere la capacidad máxima registrada de su vehículo. Una vez que el repartidor acepte el pedido cambia a estado "Asignado" y se le asigna al repartidor.
Requerimiento funcional	RF - 008 RF - 012
Reglas de negocio	RN-007: Solo repartidores con inventario registrado pueden elegir pedidos. RN-008: No se permite aceptar si la suma de garrafrones supera la capacidad del vehículo. RN-013: El cambio de estado genera un registro con timestamp. RN-015: El repartidor debe tener vehículo activo registrado.
Disparador	El repartidor tiene sesión activa, ha registrado su inventario y visualiza la lista de pedidos disponibles en su área.
Precondiciones	1. El repartidor tiene sesión activa. 2. El repartidor tiene vehículo registrado y activo. (RN-015) 3. El repartidor ha registrado su inventario de la jornada. (RN-007) 4. El repartidor tiene capacidad disponible. 5. Existen pedidos pendientes en la zona de cobertura.
Postcondiciones	1. El pedido seleccionado cambia a estado "Asignado". 2. El pedido queda vinculado al ID del repartidor. 3. Se genera un registro histórico del cambio de estado con timestamp. (RN-013) 4. El inventario disponible del repartidor se reduce en la cantidad de garrafrones del pedido. 5. El cliente recibe una notificación informando que su pedido fue asignado a un repartidor.
Flujo Normal	1. El repartidor visualiza la lista de pedidos disponibles en su pantalla principal. 2. El repartidor selecciona un pedido para ver sus detalles (dirección, garrafrones, distancia). 3. El sistema verifica que la suma de garrafrones del pedido más los ya aceptados no supere la capacidad que soporta el vehículo. (RN-008) 4. El repartidor selecciona "Aceptar pedido". 5. El sistema actualiza el estado del pedido a "Asignado" y lo asigna al repartidor. (RN-013) 6. El sistema reduce el inventario disponible del repartidor. 7. Se actualizan los datos en la base de datos. 8. El sistema notifica al cliente que su pedido fue asignado. 9. El pedido aparece en la lista de entregas pendientes del repartidor. 10. Finaliza el caso de uso.
Flujos Alternativos	S1: El repartidor no tiene capacidad • S1.1 El sistema muestra un mensaje: "No tienes capacidad suficiente para este pedido".

	• S1.2 El pedido permanece disponible para otros repartidores. • S1.3 El repartidor puede ver otros pedidos con menor cantidad.
Excepciones	E1: Error en la base de datos • E1.1 El sistema muestra un mensaje de error. • E1.2 El pedido vuelve a estado "Pendiente" automáticamente.
Prioridad	Alta
Frecuencia de uso	Alta
Requerimientos especiales	El sistema debe manejar la concurrencia cuando dos repartidores intentan aceptar el mismo pedido simultáneamente.
Datos de entrada	○ ID del pedido seleccionado. ○ Confirmación de aceptación por parte del repartidor.
Datos de salida	○ Pedido actualizado a estado "Asignado" vinculado al repartidor. ○ Inventario del repartidor reducido. ○ Notificación enviada al cliente.
Suposiciones	Existen pedidos en estado Pendiente disponibles para el repartidor.
Notas y preguntas	El sistema debe mostrar que pedidos siguen disponibles para evitar frustración por pedidos ya tomados.

CU – 011: Obtener ruta de entrega

Nombre	Obtener ruta
Actores	Repartidor, API
Descripción	El sistema calcula y muestra al repartidor la ruta óptima para realizar todas sus entregas pendientes del día, utilizando la API de OpenStreetView con el algoritmo A* y la heurística Haversine. Si un pedido se cancela antes de iniciar la ruta, el sistema la recalcula automáticamente.
Requerimiento funcional	RF – 004
Reglas de negocio	RN-009: Si un pedido es cancelado antes de iniciar la ruta, el sistema recalcula. RN-014: La polyline de la ruta se almacena en caché para optimizar el uso de la API de OpenStreetView.
Disparador	El repartidor ha aceptado al menos un pedido y selecciona "Iniciar ruta" o "Ver ruta" en su pantalla principal.
Precondiciones	1. El repartidor tiene sesión activa. 2. El repartidor tiene al menos un pedido en estado "Asignado". 3. El dispositivo tiene GPS habilitado. 4. La API de OpenStreetView esta disponible.
Postcondiciones	1. El repartidor visualiza la ruta óptima en el mapa. 2. Los pedidos del repartidor pasan a estado "En camino". 3. Se registra el inicio de la ruta con timestamp.
Flujo Normal	1. El repartidor selecciona "Iniciar ruta" en su pantalla principal. 2. El sistema verifica si existe una polyline en caché para los pedidos actuales. (RN-014) 3. Si no hay caché, el sistema consulta la API de OpenStreetView con las coordenadas de todos los domicilios. 4. La API devuelve la ruta optimizada (polyline) con el orden óptimo de visitas. 5. El sistema almacena la polyline en caché. (RN-014) 6. El sistema muestra la ruta en el mapa del dispositivo del repartidor. 7. El sistema actualiza el estado de los pedidos a "En camino". (RN-013) 8. El repartidor sigue la ruta indicada hacia el primer domicilio. 9. Finaliza el caso de uso (la navegación continúa durante la entrega).
Flujos Alternativos	S1: Un pedido se cancela antes de iniciar la ruta • S1.1 El sistema detecta la cancelación y elimina la caché de la ruta anterior. (RN-009) • S1.2 El sistema recalcula la ruta con los pedidos restantes. • S1.3 Muestra la nueva ruta al repartidor. Regresa al paso 5.
Excepciones	E1: Fallo en la API • E1.1 El sistema intenta utilizar la ruta en caché si está disponible. • E1.2 Si no hay caché, muestra un mensaje de error e invita a reintentar cuando haya conexión
Prioridad	Alta
Frecuencia de uso	Alta

Requerimientos especiales	La ruta debe calcularse en menos de 5 segundos.
Datos de entrada	○ Ubicación GPS actual del repartidor. ○ Coordenadas GPS de todos los domicilios de entrega pendientes.
Datos de salida	○ Ruta óptima calculada y desplegada en el mapa. ○ Pedidos actualizados a estado “En camino”.
Suposiciones	El repartidor tiene conexión a internet o datos móviles.
Notas y preguntas	La consulta a la API incluye el origen (ubicación actual del repartidor) y los destinos (coordenadas de los domicilios de entrega) para calcular la ruta más eficiente con el algoritmo A* y heurística Haversine.

CU – 012: Registrar resultado de entrega

Nombre	Registrar resultado de entrega
Actores	Repartidor
Descripción	Al llegar al domicilio del cliente, el repartidor registra el resultado de la entrega. Si el cliente recibe los garraones dentro del tiempo de espera, el estado pasa a Entregado. Si el cliente no aparece en 10 minutos, el repartidor puede marcarlo como No entregado, lo que activa una penalización automática. Ambos resultados generan un registro histórico con timestamp.
Requerimiento funcional	RF-013 RF-004
Reglas de negocio	RN-006: Dos ausencias consecutivas generan suspensión temporal del cliente. RN-011: El repartidor debe estar a menos de 50 m para habilitar el botón de entrega. RN-012: El repartidor espera máximo 10 minutos. RN-013: Se genera registro histórico con timestamp.
Disparador	El sistema detecta que el repartidor está a menos de 50 m del domicilio del cliente y habilita el botón Entregado. Alternativamente, el temporizador de 10 minutos expira y el sistema habilita el botón No entregado.
Precondiciones	1. El pedido está en estado "En camino". 2. El repartidor se encuentra en el domicilio del cliente (dentro de los 50 metros). 3. El temporizador de 10 minutos ha expirado sin entrega. (RN-012)
Postcondiciones	1. El pedido cambia a estado Entregado o No entregado con timestamp. (RN-013) 2. El cliente recibe notificación del resultado. 3. El inventario del repartidor se actualiza (garraones descontados o restaurados). 4. Solo si No entregado: ausencia contabilizada al cliente; si es la segunda consecutiva, cuenta suspendida temporalmente. (RN-006)
Flujo Normal	1. El repartidor llega al domicilio del cliente 2. El sistema calcula continuamente la distancia GPS entre el repartidor y el domicilio. 3. El sistema detecta que la distancia es menor a 50 m. (RN-011)El repartidor confirma la acción. 4. El sistema habilita automáticamente el botón "Marcar como entregado". 5. El repartidor entrega los garraones al cliente 6. El repartidor selecciona "Marcar como entregado". 7. El sistema actualiza el estado a "Entregado" y genera el timestamp. 8. El sistema notifica al cliente y actualiza el inventario del repartidor 9. Finaliza el caso de uso.
Flujos Alternativos	S1: El cliente aparece justo cuando el repartidor va a marcar No entregado • S1.1 El repartidor puede cancelar la acción antes de confirmar. • S1.2 El repartidor continua con la entrega. Finaliza el caso de uso. S2: El cliente no sale en 10 minutos • S2.1 El temporizador expira. (RN-012)

	• S2.2 El sistema deshabilita el botón Entregado y habilita No entregado. • S2.3 El repartidor selecciona No entregado y confirma en el cuadro de diálogo. • S2.4 El sistema actualiza el estado a No entregado con timestamp. (RN-013) • S2.5 El sistema contabiliza la ausencia; si es la segunda consecutiva, suspende la cuenta del cliente. (RN-006) • S2.6 El repartidor recupera los garraones en su inventario disponible. • S2.7 El sistema notifica al cliente del pedido no entregado. Finaliza el caso de uso.
Excepciones	E1: Error al registrar el estado en la base de datos • E1.1 El sistema muestra un mensaje de error. • E1.2 El botón permanece activo para reintentar.
Prioridad	Alta
Frecuencia de uso	Alta
Requerimientos especiales	El sistema debe llevar un contador confiable de ausencias consecutivas por cliente para aplicar la penalización correctamente.
Datos de entrada	○ Ubicación GPS en tiempo real del repartidor. ○ Confirmación del repartidor (entregado o no entregado).
Datos de salida	○ Pedido actualizado a estado con timestamp. ○ Notificación al cliente. ○ Suspensión aplicada si es segunda ausencia consecutiva. ○ Inventario del repartidor actualizado.
Suposiciones	El repartidor llegó al domicilio correcto y esperó el tiempo reglamentario.
Notas y preguntas	Ninguna

CU – 013: Historial de entregas

Nombre	Historial de entregas
Actores	Repartidor
Descripción	El repartidor puede consultar el historial completo de todas las entregas asociadas a su cuenta, incluyendo entregas exitosas, no entregadas y canceladas. El sistema filtra los registros exclusivamente por el ID del repartidor autenticado.
Requerimiento funcional	RF – 010
Reglas de negocio	RN-017: El repartidor solo puede ver los pedidos asociados a su propio ID. RN-013: Los registros son inmutables.
Disparador	El repartidor selecciona la sección "Mi historial" o "Mis entregas" desde el menú principal.
Precondiciones	1. El repartidor tiene sesión activa. 2. El repartidor tiene al menos un pedido registrado en el sistema
Postcondiciones	1. El repartidor visualiza la lista de sus entregas con estado y detalles. 2. No se modifica ningún dato (consulta de solo lectura).
Flujo Normal	1. El repartidor selecciona "Mi historial" en el menú. 2. El sistema consulta todos los pedidos asociados al ID del repartidor. (RN-017) 3. El sistema ordena los registros de más reciente a más antiguo. 4. El sistema muestra: fecha, cliente, dirección, cantidad de garraones, estado final, y monto (si aplica). 5. El repartidor puede seleccionar un pedido para ver el detalle y los timestamps del ciclo de vida. 6. Finaliza el caso de uso.
Flujos Alternativos	S1: No hay entregas registradas • S1.1 El sistema muestra el mensaje "Aún no tienes entregas registradas". Finaliza el caso de uso.
Excepciones	E1: Error de conexión. • E1.1 El sistema muestra un mensaje de error e invita a reintentar
Prioridad	Media
Frecuencia de uso	Baja
Requerimientos especiales	La consulta debe ejecutarse en menos de 5 segundos.
Datos de entrada	○ ID del repartidor el cual accederá al historial
Datos de salida	○ Lista de entregas del repartidor ordenada de más reciente a más antiguo. ○ Fecha. ○ Cliente. ○ Dirección, cantidad de garraones ○ Estado final.
Suposiciones	El historial es permanente y no puede ser eliminado por el repartidor
Notas y preguntas	El historial puede servir como comprobante de las entregas realizadas y es útil para resolución de conflictos.

CU – 014: Dar de alta usuario

Nombre	Dar de alta usuario
Actores	Administrador
Descripción	El administrador puede registrar manualmente nuevos usuarios (clientes o repartidores) en el sistema, asignando el rol correspondiente y configurando sus datos de acceso. Al dar de alta al usuario, el sistema le notifica por correo electrónico con sus credenciales o un enlace para establecer contraseña.
Requerimiento funcional	RF – 009
Reglas de negocio	RN-016: No se permite duplicidad de correo o teléfono. RN-018: La contraseña se cifra antes de almacenarse.
Disparador	El administrador accede al panel de gestión de usuarios y selecciona "Nuevo usuario".
Precondiciones	1. El administrador tiene sesión activa con permisos de superadministrador. 2. El correo electrónico y teléfono del nuevo usuario no están previamente registrados.
Postcondiciones	1. El usuario queda registrado en la base de datos con el rol asignado. 2. La contraseña queda cifrada. (RN-018) 3. El sistema envía una notificación al nuevo usuario con sus datos de acceso. 4. El usuario puede iniciar sesión en la aplicación.
Flujo Normal	1. El administrador accede al panel de gestión y selecciona "Nuevo usuario". 2. El sistema muestra el formulario de alta (nombre, correo, teléfono, rol). 3. El administrador completa los datos y asigna el rol (cliente o repartidor). 4. El sistema valida que el correo y teléfono no estén duplicados. (RN-016) 5. El sistema genera una contraseña temporal o enlace de configuración. 6. El sistema cifra la contraseña y crea el registro. (RN-018) 7. El sistema envía un correo de bienvenida al nuevo usuario con instrucciones de acceso. 8. El sistema muestra al administrador un mensaje de alta exitosa. 9. Finaliza el caso de uso.
Flujos Alternativos	S1: El correo ya está registrado • S1.1 El sistema muestra el campo duplicado. • S1.2 El administrador corrige los datos. Regresa al paso 3
Excepciones	E1: Error de conexión. • E1.1 El sistema muestra un mensaje de error e invita a reintentar
Prioridad	Alta
Frecuencia de uso	Baja
Requerimientos especiales	Solo el superadministrador tiene permisos para crear usuarios desde el panel.
Datos de entrada	o Nombre.

	- o correo electrónico. - o número de teléfono. - o rol del nuevo usuario (cliente o repartidor) ingresados por el administrador.
Datos de salida	o Usuario registrado en la base de datos con contraseña cifrada y rol asignado.
Suposiciones	El administrador dispone de los datos correctos y verificados del nuevo usuario.
Notas y preguntas	*: El alta manual por el administrador es una alternativa al autoregistro, útil para incorporar repartidores verificados al sistema.

CU – 015: Dar de baja usuario

Nombre	Dar de baja usuario
Actores	Administrador
Descripción	El administrador puede desactivar la cuenta de cualquier usuario (cliente o repartidor) del sistema. La baja inhabilita el acceso del usuario sin eliminar su historial de la base de datos, Manteniendo todas sus interacciones anteriores.
Requerimiento funcional	RF – 009
Reglas de negocio	RN-013: El historial de pedidos del usuario no se elimina al darlo de baja. RN-017: Los datos del usuario solo son visibles para sí mismo y el administrador.
Disparador	El administrador localiza al usuario en el panel de gestión y selecciona la opción "Dar de baja".
Precondiciones	1. El administrador tiene sesión activa con permisos de superadministrador. 2. El usuario que se dará de baja existe en el sistema. 3. Si es repartidor, no debe tener pedidos activos en estado "En camino".
Postcondiciones	1. La cuenta del usuario queda en estado "Inactivo" en la base de datos. 2. El usuario no puede iniciar sesión en la aplicación. 3. Si tenía sesión activa, el token queda invalidado inmediatamente. 4. El historial del usuario se preserva íntegro en la base de datos. (RN-013)
Flujo Normal	1. El administrador busca al usuario en el panel de gestión. 2. El sistema muestra el perfil del usuario con sus datos y estado actual. 3. El administrador selecciona "Dar de baja". 4. El sistema muestra un cuadro de confirmación con el nombre del usuario. 5. El administrador confirma la acción. 6. El sistema verifica que el usuario no tenga pedidos activos en curso. 7. El sistema actualiza el estado de la cuenta a "Inactivo". 8. El sistema invalida cualquier token de sesión activo del usuario. 9. El sistema registra el evento de baja con el ID del administrador y timestamp.
Flujos Alternativos	S1: El repartidor tiene pedidos en estado "En camino". • S1.1 El sistema muestra una advertencia indicando los pedidos activos. • S1.2 El administrador puede elegir esperar o forzar la baja. • S1.3 Si fuerza la baja, el sistema reasigna los pedidos activos o los devuelve a "Pendiente".
Excepciones	E1: Error al actualizar el estado en la base de datos. • E1.1 El sistema muestra un mensaje de error. • E1.2 La cuenta permanece activa hasta que la operación sea exitosa.
Prioridad	Alta
Frecuencia de uso	Baja

Requerimientos especiales	La baja es lógica: el registro permanece en la base de datos. La cuenta puede reactivarse si el administrador lo decide
Datos de entrada	○ ID del usuario al cual se dará de baja
Datos de salida	○ Cuenta del usuario actualizada a estado “Inactivo”. ○ Token de sesión activo invalidado. ○ Evento de baja registrado con timestamp e ID del administrador.
Suposiciones	El administrador tiene justificación válida para dar de baja al usuario.
Notas y preguntas	Se recomienda que el sistema registre el motivo de la baja como campo opcional

CU – 016: Consultar historial global de pedidos

Nombre	Consultar historial global de pedidos
Actores	Administrador
Descripción	El administrador puede consultar el historial completo de todos los pedidos del sistema, sin restricción por usuario, incluyendo pedidos de todos los clientes y repartidores. Tiene acceso a filtros por fecha, estado, repartidor y cliente para facilitar auditorías y la resolución de disputas.
Requerimiento funcional	RF – 009, RF – 010
Reglas de negocio	RN-013: Los registros históricos son inmutables. RN-017: El administrador es el único actor con visibilidad global de todos los pedidos.
Disparador	El administrador accede a la sección "Historial de pedidos" o "Reportes" desde el panel de administración.
Precondiciones	1. El administrador tiene sesión activa con permisos de superadministrador. 2. Existen pedidos registrados en el sistema.
Postcondiciones	1. El administrador visualiza el historial filtrado según sus criterios. 2. No se modifica ningún dato (consulta de solo lectura). 3. Los registros pueden exportarse si la funcionalidad está disponible.
Flujo Normal	1. El administrador accede a "Historial de pedidos" en el panel. 2. El sistema muestra todos los pedidos ordenados de más reciente a más antiguo. 3. El administrador aplica filtros opcionales: rango de fechas, estado, cliente, repartidor. 4. El sistema actualiza la lista según los filtros aplicados. 5. El administrador selecciona un pedido para ver su detalle completo. 6. El sistema muestra: cliente, repartidor, dirección, cantidad, timestamps de cada estado, resultado final.
Flujos Alternativos	S1: No hay pedidos que coincidan con los filtros aplicados. • S1.1 El sistema muestra el mensaje "No se encontraron pedidos con los criterios seleccionados". • S1.2 El administrador puede modificar o limpiar los filtros. Regresa al paso 3.
Excepciones	E1: Error en la base de datos • E1.1 El sistema muestra un mensaje de error. • E1.2 Invita al administrador a reintentar.
Prioridad	Media
Frecuencia de uso	Media
Requerimientos especiales	La consulta debe soportar grandes volúmenes de datos con paginación. El tiempo de respuesta no debe superar 5 segundos.
Datos de entrada	Sesión activa del administrador. Filtros opcionales: rango de fechas, estado del pedido, cliente y/o repartidor.
Datos de salida	Lista paginada de todos los pedidos del sistema ordenada de más reciente a más antiguo, filtrada según los criterios aplicados, con detalle completo de cada pedido.

Suposiciones	El administrador conoce los criterios de búsqueda relevantes para su consulta.
Notas y preguntas	Los registros históricos son inmutables por diseño (RN-013), por lo que el administrador tampoco puede modificarlos desde este panel. La función de exportación puede considerarse para versiones futuras.

CU – 17: Desplegar mapa

Nombre	Desplegar mapa
Actores	Cliente, API Google Maps
Descripción	El sistema solicita y renderiza un mapa interactivo mediante la API de Google Maps cada vez que el usuario accede a una pantalla que requiere visualización geográfica. El mapa muestra la ubicación en tiempo real del repartidor, el domicilio del cliente y, cuando aplica, la ruta de entrega trazada. El caso de uso es invocado por CU-006 Rastrear pedido y CU-011 Obtener ruta de entrega.
Requerimiento funcional	RF-007 RF-013
Reglas de negocio	RN-014: El mapa solo se despliega si el dispositivo tiene conexión a internet activa. RN-015: La API Key de Google Maps debe estar vigente y con cuota disponible. RN-016: El mapa se actualiza con la posición del repartidor cada 5 segundos mientras el pedido esté en estado “En camino”.
Disparador	El usuario elige la opción para que se pueda desplegar el mapa de Google Mapse en lugar del de OpenStreetMaps.
Precondiciones	1. El usuario tiene sesión activa. 2. El dispositivo cuenta con conexión a internet. 3. La API Key de Google Maps está configurada y disponible. 4. Existe al menos una ubicación a mostrar (domicilio del cliente o posición del repartidor).
Postcondiciones	1. El mapa interactivo se renderiza correctamente en pantalla. 2. Se muestra el marcador de posición del repartidor actualizado. 3. Se muestra el marcador del domicilio del cliente. 4. Si el caso fue invocado desde CU-013, la ruta trazada también se despliega sobre el mapa.
Flujo Normal	1. El sistema identifica la pantalla que requiere el mapa y recopila las coordenadas necesarias (posición del repartidor y domicilio del cliente). 2. El sistema realiza la solicitud de tiles y datos a la API de Google Maps con las coordenadas y el nivel de zoom apropiado. 3. La API de Google Maps responde con los recursos del mapa. 4. El sistema renderiza el mapa en la pantalla del usuario. 5. El sistema coloca los marcadores: posición actual del repartidor y domicilio del cliente. 6. Si fue invocado desde CU-013, superpone la ruta de entrega sobre el mapa. 7. El sistema inicia las actualizaciones periódicas de posición del repartidor cada 5 segundos. (RN-016) 8. Finaliza el caso de uso. El mapa permanece activo mientras el usuario esté en la pantalla.
Flujos Alternativos	S1: El GPS del repartidor no tiene señal. • S1.1 El sistema no puede obtener coordenadas actualizadas del repartidor. • S1.2 El mapa se renderiza con la última posición conocida del repartidor.

	• S1.3 El sistema muestra un aviso: "Última ubicación conocida — GPS sin señal". Continúa. Finaliza el caso de uso
Excepciones	E1: Sin conexión a internet. • E1.1 El sistema no puede contactar la API de Google Maps. • E1.2 Se muestra un mensaje: "Sin conexión. No es posible mostrar el mapa." • E1.3 El sistema reintenta automáticamente al recuperar conexión. Finaliza el caso de uso
Prioridad	Alta
Frecuencia de uso	Alta
Requerimientos especiales	El administrador debe poder visualizar la imagen anterior y la nueva propuesta lado a lado para comparar.
Datos de entrada	○ Coordenadas GPS del repartidor ○ Coordenadas del domicilio del cliente ○ Ruta calculada desde CU-013
Datos de salida	○ Mapa interactivo con marcadores de posición y ruta
Suposiciones	El dispositivo del usuario tiene hardware GPS funcional. La cuota de la API de Google Maps es suficiente para el volumen de pedidos diarios del sistema.
Notas y preguntas	Ninguna

4.8 Diagrama de actividad

4.8.3 Diagrama de actividad para el usuario (cliente)

La Ilustración 6 presenta el diagrama de actividades del usuario (cliente), donde se detallan las acciones que puede llevar a cabo, en las cuales están:

- Crear cuenta
- Realizar pedido
- Ver historial de pedidos

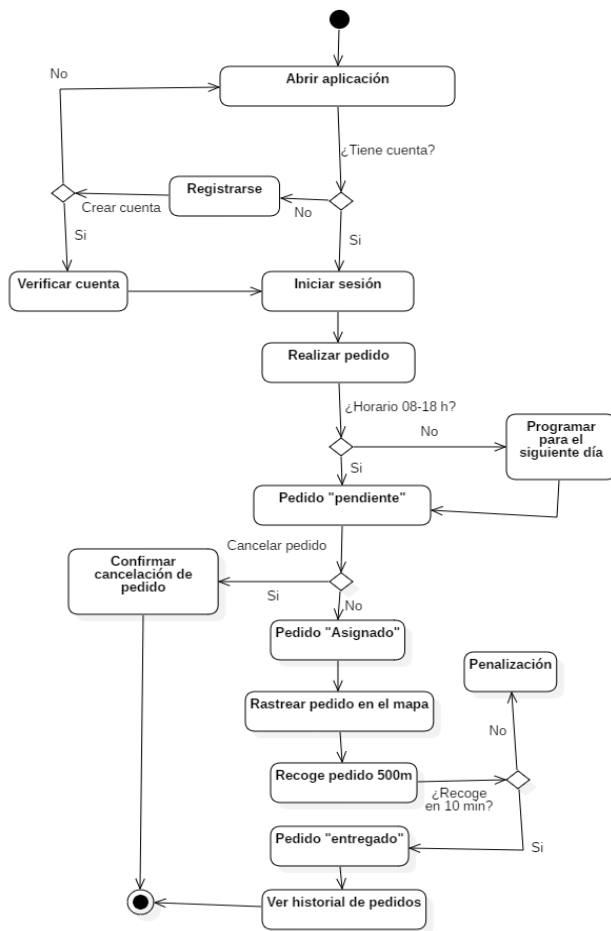


Ilustración 13 Diagrama de actividades Cliente. Elaboración propia.

4.8.4 Diagrama de actividad para el usuario (repartidor)

La ilustración 7 presenta el diagrama de actividades del usuario (repartidor), donde se detallan las acciones que puede llevar a cabo, en las cuales están:

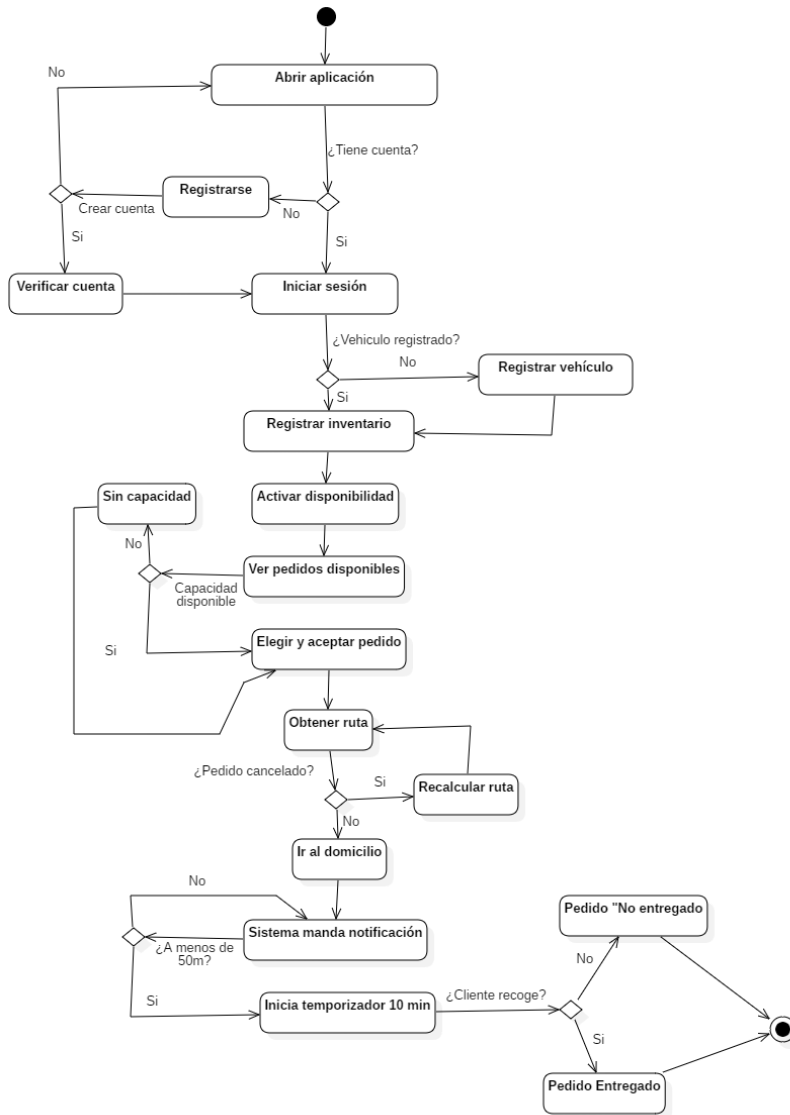


Ilustración 14 Diagrama de actividad del repartidor. Elaboración propia.

4.8.5 Diagrama de actividad para el administrador

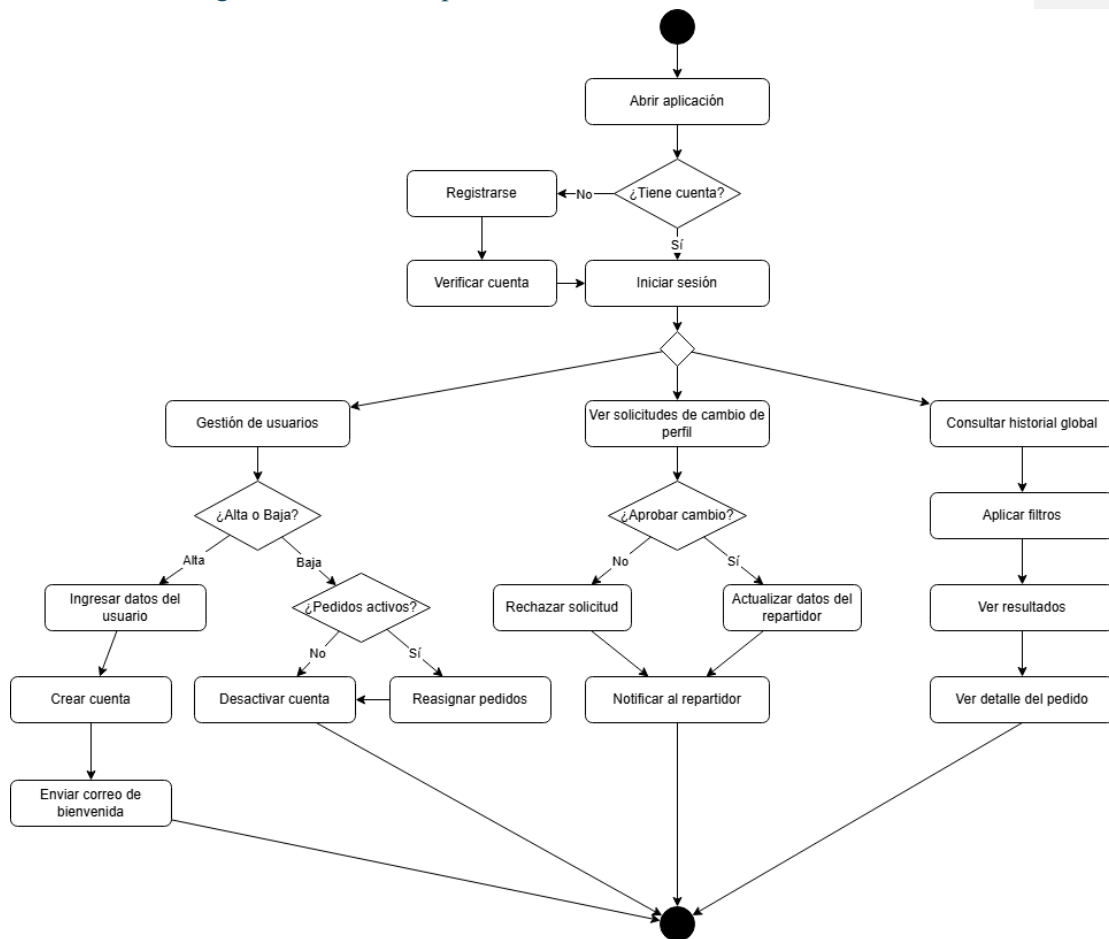
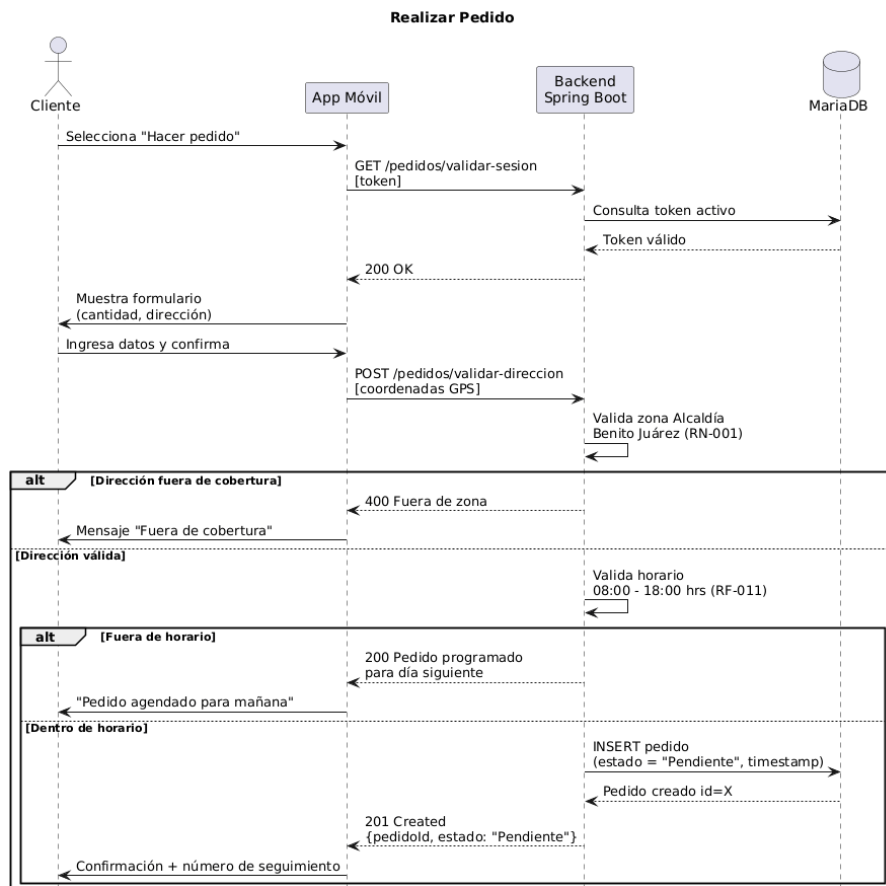


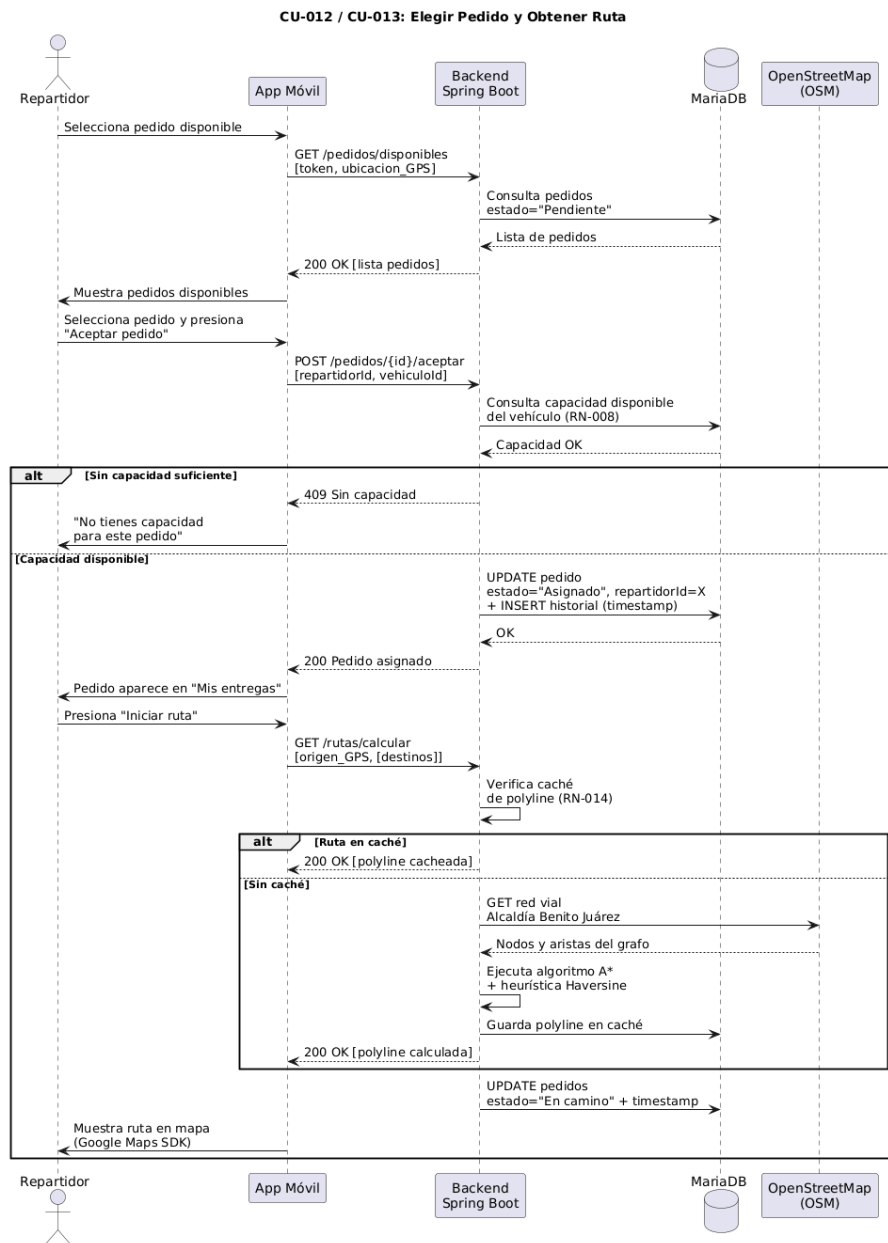
Ilustración 15 Diagrama de actividad orientado al administrador. Elaboración propia.

4.9 Diagrama de secuencia

4.9.1 Realizar pedido



4.9.2 Elegir pedido y Obtener Ruta



4.10 Diseño de Vistas





HtoGO

Crea tu cuenta

NOMBRE COMPLETO

 Ej. Juan Pérez

CORREO ELECTRÓNICO

 juan@ejemplo.com

NÚMERO DE TELÉFONO

 123 456 7890

CONTRASEÑA





TIPO DE USUARIO



Cliente



Repartidor

Registrarse →

[¿Ya tienes una cuenta? Inicia sesión](#)



HtoGO

Bienvenido de nuevo

Correo electrónico

 u@email.com

Contraseña

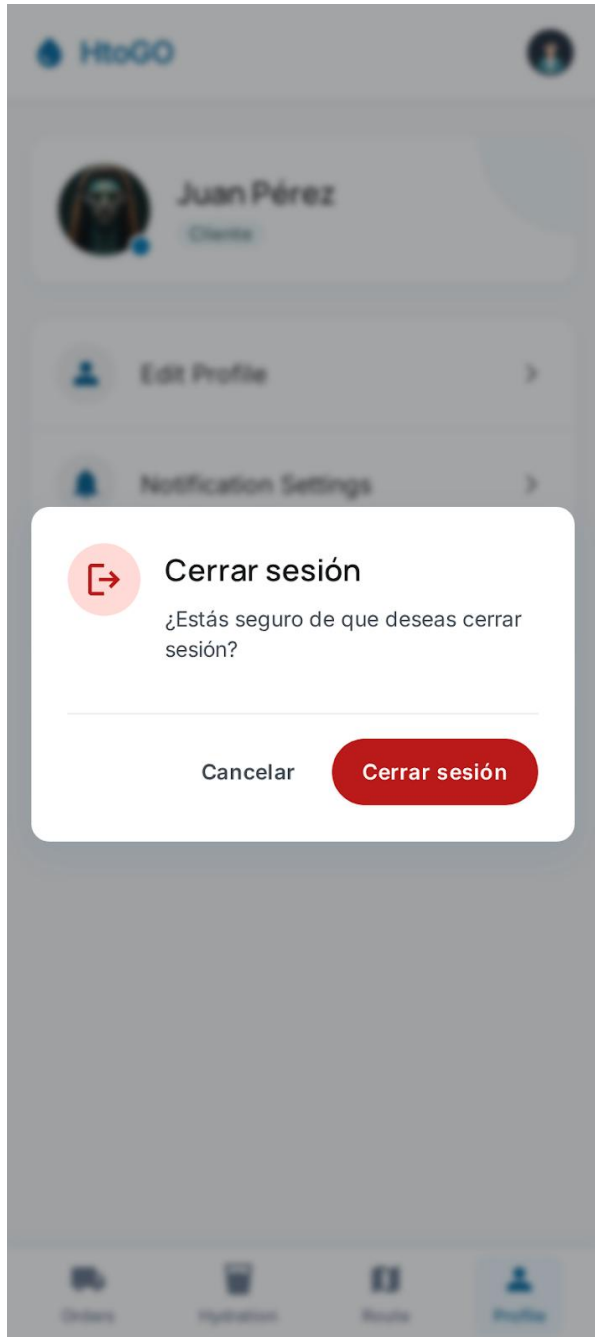
[¿Olvidaste tu contraseña?](#)

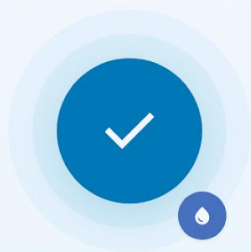




Iniciar sesión →

¿No tienes una cuenta? [Crear cuenta](#)





¡Cuenta verificada!

Tu correo ha sido verificado con éxito.
Ahora puedes disfrutar de todos los
servicios de HtoGO.

[Ir al inicio de sesión](#)

Referencias

- [1] D. Montero, 2020. [En línea]. Available: https://www.agua.unam.mx/covid19/assets/pdf/AguaEmbotallada_DeliaMontero.pdf. [Último acceso: 3 Febrero 2026].
- [2] K. & L. J. Laudon, Sistemas de información gerencial, decimocuarta ed., México: Pearson, 2016.
- [3] H. Lee, 2004. [En línea]. Available: <https://hbr.org/2004/10/the-triple-a-supply-chain>. [Último acceso: 3 Febrero 2026].
- [4] P. & G. T. Mell, 2011. [En línea]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>. [Último acceso: 3 Febrero 2026].
- [5] N. N. Group, 2024. [En línea]. Available: <https://www.nngroup.com/articles/trustworthy-design/>. [Último acceso: 3 Febrero 2026].
- [6] Z. Yan, 2007. [En línea]. Available: <https://aaltodoc.aalto.fi/items/3383cc3c-774e-4304-a84f-aadb807385c8>. [Último acceso: 5 Febrero 2026].
- [7] H. Castellanos, 2025. [En línea]. Available: <https://www.excelsior.com.mx/comunidad/temen-afectaciones-a-largo-plazo-por-agua-contaminadaen->. [Último acceso: 5 Febrero 2026].
- [8] K. E. Kendall y J. E. Kenneth, Análisis y diseño de sistemas, 8.a ed., CDMX: Pearson Educación, 2011.
- [9] L. L. V, 2025. [En línea]. Available: <https://agua.org.mx/ciudad-de-mexico-agua-contaminada-de-bj-pone-en-riesgo-la-salud-de-mascotas-publimetro/>. [Último acceso: 6 Febrero 2026].
- [10] R. ejecentral, 2024. [En línea]. Available: <https://agua.org.mx/mexico-juez-obliga-al-gobierno-de-la-cdmx-informar-del-agua-contaminada-en-bj-eje-central/>. [Último acceso: 6 Febrero 2026].
- [11] G. L. León, 2023. [En línea]. Available: <https://relatosehistorias.mx/nuestras-historias/el-aguador>. [Último acceso: 6 Febrero 2026].
- [12] R. Pacheco-Vega, 2019. [En línea]. Available: <https://www.raulpacheco.org/2019/12/human-right-to-water-and-bottled-water-consumption-governing-at-the-intersection-of-water-justice-rights-and-ethics-new-chapter/>. [Último acceso: 7 Febrero 2026].

- [13 Google, 2025. [En línea]. Available:
] <https://developers.google.com/maps/documentation/routes/compute-route-over?hl=es-419>.
[Último acceso: 8 Febrero 2026].
- [14 Mecalux, 2024. [En línea]. Available: [https://www.mecalux.com.mx/blog/logistica-de-](https://www.mecalux.com.mx/blog/logistica-de-distribucion)
] [distribucion](https://www.mecalux.com.mx/blog/logistica-de-distribucion). [Último acceso: 7 Febrero 2026].
- [15 Microsoft, 2026. [En línea]. Available: microsoft.com.. [Último acceso: 10 Febrero 2026].
]
- [16 D. y. S. Isakova, Kotlin in Action, I. S, Ed., NY: Manning Publications, 2017.
]
- [17 Google, «Kotlin en Android,» 2023. [En línea]. Available:
] <https://developer.android.com/kotlin/overview?hl=es-419>. [Último acceso: 10 Febrero 2026].
- [18 K. Documentation, 2026. [En línea]. Available: <https://kotlinlang.org/docs/java-interop.html>.
] [Último acceso: 13 Febrero 2026].
- [19 Ubiqo, 2022. [En línea]. Available: <https://www.ubiqo.net/tecnologia/que-es-un-gps/>. [Último
] acceso: 13 Febrero 2026].
- [20 V. museum, 2026. [En línea]. Available: [https://www.versionmuseum.com/history-of/google-](https://www.versionmuseum.com/history-of/google-maps-website)
] [maps-website](https://www.versionmuseum.com/history-of/google-maps-website). [Último acceso: 17 Febrero 2026].
- [21 C. Walls, Spring Boot in Action, C. Walls, Ed., NY: Manning Publications, 2016.
]
- [22 R. Martin, Clean Architecture: A Craftsman's Guide to Software, Boston: Prentice Hall, 2017.
]
- [23 I. Y. TECNOLOGIA, 2022. [En línea]. Available: [https://unirfp.unir.net/revista/ingenieria-y-](https://unirfp.unir.net/revista/ingenieria-y-tecnologia/framework/)
] [tecnologia/framework/](https://unirfp.unir.net/revista/ingenieria-y-tecnologia/framework/). [Último acceso: 17 Febrero 2026].
- [24 D. Ortega y M. Guevara, ELEMENTARY: UN FRAMEWORK DE PROGRAMACIÓN,
] Maracaibo: Universidad Privada, 2016.
- [25 Y. Muradas, Junio 2022. [En línea]. Available: [https://openwebinars.net/blog/conoce-que-es-](https://openwebinars.net/blog/conoce-que-es-spring-framework-y-por-que-usarlo/)
] [spring-framework-y-por-que-usarlo/](https://openwebinars.net/blog/conoce-que-es-spring-framework-y-por-que-usarlo/). [Último acceso: 20 Febrero 2026].
- [26 V. Tanzu, 2026. [En línea]. Available: <https://spring.io/projects/spring-framework>. [Último
] acceso: 20 Febrero 2026].
- [27 C. WALLS, Spring boot in Action, Shelter Island: Manning Publications Co., 2016.
]

- [28 Spring Org, [En línea]. Available: <https://docs.spring.io/spring-boot/index.html>. [Último acceso: 20 Febrero 2026].
- [29 M. Goodwin, 2023. [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/api>. [Último acceso: 21 Febrero 2026].
- [30 A. W. Services, 2026. [En línea]. Available: <https://aws.amazon.com/es/what-is/api/>. [Último acceso: 21 Febrero 2026].
- [31 M. Kleppmann, Designing Data-Intensive Applications, Sebastopol: O'Reilly Media, 2017.
- [32 R. T. Fielding, «Architectural Styles and the Design of Network-based,» Irvine, California, United States, 2000.
- [33 IBM Corporation, 2026. [En línea]. Available: <https://www.ibm.com/docs/es/cics-tg-multi/9.3.0?topic=services-concepts-restful-json-web>. [Último acceso: 21 Febrero 2026].
- [34 C. Castillo Rodríguez, «Definición de una arquitectura Restful orientada al recurso (Resource-Oriented Architecture - ROA en la construcción de un web service que aloja algoritmos solucionadores (solvers) orientados a problemas de programación lineal como servicios,» Pereira, 2017.
- [35 A. Enrique, Principios de diseño de APIs REST, Leanpub, Ed., Leanpub, 2013.
- [36 Mozilla Corporation, [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/GET>. [Último acceso: 23 Febrero 2026].
- [37 Mozilla Corporation, [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/PUT>. [Último acceso: 23 Febrero 2026].
- [38 Mozilla Corporation, [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/DELETE>. [Último acceso: 23 Febrero 2026].
- [39 Mozilla Corporation, [En línea]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/POST>. [Último acceso: 23 Febrero 2026].
- [40 IBM, 2026. [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/relational-databases>. [Último acceso: 2 Marzo 2026].
- [41 A. Gomstyn, 2026. [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/database-normalization>. [Último acceso: 2 Marzo 2026].
- [42 O. E. Barboza, Normalización, Universidad San Marcos: San Marcos, 2020.

- [43] A. Silberschatz, FUNDAMENTOS DE BASES DE DATOS, 4ta ed., Aravaca: McGRAW-HILL, 2002.
- [44] IBM, «IBM.COM,» [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/entity-relationship-diagram>. [Último acceso: 26 Febrero 2026].
- [45] UNIR, «La universidad en internet,» 12 04 2024. [En línea]. Available: <https://mexico.unir.net/noticias/ingenieria/modelo-entidad-relacion/>. [Último acceso: 26 Febrero 2026].
- [46] I. Y. TECNOLOGIA, 2024. [En línea]. Available: <https://www.unir.net/revista/ingenieria/mysql/>. [Último acceso: 26 Febrero 2026].
- [47] El País, «Sun Microsystems compra MySQL por 1.000 millones de dólares,» 16 Enero 2008.
- [48] MariaDB Foundation, «Maria DB en resumen,» [En línea]. Available: <https://mariadb.org/es/>. [Último acceso: 1 Marzo 2026].
- [49] DB-Engines, Marzo 2026. [En línea]. Available: <https://db-engines.com/en/ranking>. [Último acceso: 1 Marzo 2026].
- [50] Open Source Initiative, «GNU General Public License version 2,» [En línea]. Available: <https://opensource.org/license/GPL-2.0>. [Último acceso: 1 Abril 2026].
- [51] K. Apple y W. Haken, «Siete puentes, un camino: Königsberg,» *Revista suma*, vol. 45, pp. 69-78, 2009.
- [52] M. F. Á. Nuñez, «Teoría de grafos,» Universidad del Bío-Bío, Chillán, 2013.
- [53] S. G. d. I. Lama, «Aplicación de Algoritmos de Búsqueda en la optimización de caminos mínimos en grafos de decisión,» Universidad Politécnica, Escuela Técnica Superior de Ingenieros Informáticos, Madrid, 2024.
- [54] T. H. L. C. E. R. L. & S. C. Cormen, Introduction to Algorithms, Cambridge: McGraw-Hill Book Company, 2009.
- [55] S. J. R. a. P. Norvig, Artificial Intelligence A Modern Approach, New Jersey: Pearson, 2010.
- [56] Scikit Learn, «haversine_distances,» [En línea]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.haversine_distances.html. [Último acceso: 16 Marzo 2026].
- [57] J. D. a. A. Smith, Real-time Route Optimization for Last-Mile Delivery using Google Directions API, 7ma, Ed., Erodé: ICCMC, 2023.

- [58 P. Mooney y M. Minghini, «A Review of OpenStreetMap Data,» 2016.
]
- [59 INEGI, «Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2024,» 2024.
- [60 AMVO, «Estudio sobre Venta Online en México,» 2025.
]
- [61 Editorial, 2025. [En línea]. Available: <https://retailers.mx/apps-en-mexico-crecen-21-en-usuarios/>. [Último acceso: 6 Marzo 2026].
- [62 M. Blessing, «Progressive Web Apps (PWAs) and Their Impact on User Experience,» 2024.
]
- [63 A. Rodriguez, 2025. [En línea]. Available: <https://builtin.com/software-engineering-perspectives/kotlin#:~:text=%C2%BFPara%20qu%C3%A9%20se%20utiliza%20Kotlin,>
[Último acceso: 7 Marzo 2026].
- [64 E. Canorea, 2026. [En línea]. Available: <https://www.plainconcepts.com/es/kotlin-android/#:~:>
[Último acceso: 7 Marzo 2026].
- [65 C. T. Méndez, «SUPERVISION TECNICA Y ADMINISTRATIVA DE LA SECTORIZACION DE LA RED DE DISTRIBUCION DE AGUA POTABLE EN LA DELEGACION GUSTAVO A. MADERO, DF,» Ciudad de México, 2008.
- [66 «EL PROBLEMA DE ABASTECIMIENTO DE AGUA EN LA DELEGACIÓN IZTAPALAPA: OPCIONES PARA MEJORAR SU CALIDAD Y EFICIENCIA,» Ciudad de México, 2015.
- [67 K. L. RAMIREZ, «PROPUESTA DE UN MODELO DE RUTEO DE UNA EMPRESA DE LA INDUSTRIA QUIMICA,» Tulancingo, 2019.
- [68 Rappi, 2026. [En línea]. Available: <https://legal.rappi.com/mexico/terminos-y-condiciones>.
[Último acceso: 14 Marzo 2026].
- [69 Uber, «HelpUber,» 2026. [En línea]. Available: <https://help.uber.com/es/merchants-and-restaurants/article/art%C3%ADculos-prohibidos-y-restringidos-a-trav%C3%A9s-de-uber%C2%A0direct?nodeId=63024079-3274-4259-8c7c-96724b882da5>. [Último acceso: 14 Marzo 2026].
- [70 Appgua, «Appgua,» 2026. [En línea]. Available: <https://appgua.com.mx/nosotros>. [Último acceso: 14 Marzo 2026].
- [71 R. d. agua, «Reparto de agua,» 2026. [En línea]. Available: <https://www.repartodeagua.com/>.
[Último acceso: 14 Marzo 2026].

- [72 A. Villasana, 2026. [En línea]. Available: <https://www.netlogistik.com/es/blog/entregas-a-domicilio-7-tendencias-en-2024>. [Último acceso: 16 Marzo 2026].
- [73 F. P. Lara, 2024. [En línea]. Available: <https://www.godaddy.com/resources/latam/desarrollo/kotlin-lenguaje-programacion-popular>. [Último acceso: 16 Marzo 2026].
- [74 c. d. mexico, «colonias,» [En línea]. Available: <https://www.coloniasdemexico.com>. [Último acceso: 21 Abril 2026].
- [75 S. Aguilar-Barojas, «Fórmulas para el cálculo del tamaño de la muestra en investigaciones de salud,» Salud Tabasco, Tabasco, 2010.
- [76 C. Gomez, 2022. [En línea]. Available: <https://www.diariodeqa.com/post/quiero-ser-qa-que-debo-aprender-requerimientos-de-software>. [Último acceso: 18 Marzo 2026].
- [77 Software Industrial, «Guía para documentar Reglas de Negocio Software,» Software Industrial, [En línea]. Available: <https://softwareindustrial.es/producto/documentar-reglas-de-negocio/>. [Último acceso: 19 Marzo 2025].
- [78 H. Campos, S. Cai, K. Sharkey y N. Schonning, «Crear y usar reglas de negocios,» Microoft, 04 Mayo 2023. [En línea]. Available: <https://learn.microsoft.com/es-es/biztalk/core/creating-and-using-business-rules>. [Último acceso: 19 Marzo 2025].
- [79 S. G. Pérez Ibarra, J. Rolando Quispe, F. Fernando Mullicundo y D. Alberto Lamas, «HERRAMIENTAS Y TECNOLOGÍAS PARA EL DESARROLLO WEB DESDE,» Facultad de Ingeniería - Universidad Nacional de Jujuy, 2021.
- [80 J. Cincović y M. Punt, «Comparison: Angular vs. React vs. Vue. Which framework is the best choice?,» *University of Belgrade, School of Electrical Engineering, Belgrade, Serbia*, pp. 250-255, 2020.
- [81 Angular, «Angular Documentation,» Google, [En línea]. Available: <https://angular.dev/overview>. [Último acceso: 2 Abril 2026].
- [82 React, «React Documentation,» Meta Corporation, [En línea]. Available: <https://react.dev/learn>. [Último acceso: 2 Abril 2026].
- [83 Vue, «Vue Documentation,» [En línea]. Available: <https://vuejs.org/guide/introduction.html>. [Último acceso: 2 Abril 2026].
- [84 D. Gala Yalupalin, «La programación Front-End y Back-End,» UNIVERSIDAD NACIONAL DE EDUCACION Escuela Profesional de Electrónica y Telecomunicaciones, Lima, Perú, 2021.

- [85 E. Haro, T. Guarda, A. O. Zambrano Peñaherrera y G. Ninahualpa Quiña, «Desarrollo backend para aplicaciones web, Servicios Web Restful: Node.js vs Spring Boot,» *Revista Ibérica de Sistemas e Tecnologías de Informação*, nº E17, p. 309–321, 2018.
- [86 Django Software Foundation, «Documentación de Django,» [En línea]. Available: <https://docs.djangoproject.com/es/6.0/>. [Último acceso: 5 Abril 2026].
- [87 OpenJS Foundation, «Express,» [En línea]. Available: <https://expressjs.com/>. [Último acceso: 5 Abril 2026].
- [88 mdn, «Express Web Framework,» Mozilla Corporation, 27 Marzo 2025. [En línea]. Available: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs. [Último acceso: 5 Abril 2026].
- [89 A. Ugarte y K. Gilmore, «Creación de una aplicación web con Spring Boot y Angular,» Baeldung, 11 Mayo 2024. [En línea]. Available: <https://www.baeldung.com/spring-boot-angular-web>. [Último acceso: 6 Abril 2026].
- [90 M. Planas, T. Rodríguez y M. Lecha, «La importancia de los datos,» *Nutrición Hospitalaria*, vol. 19, nº 1, pp. 11-13, 2004.
- [91 R. Z. Ramírez, «Sistemas gestores de base de datos,» MEDIO, CICLO FORMATIVO GRADO, Innovación y Experiencias Educativas, 2008.
- [92 MariaDB, «MariaDB Documentation,» 2026. [En línea]. Available: <https://mariadb.com/docs/>. [Último acceso: 6 Abril 2026].
- [93 MySQL, «MySQL 8.0 Reference Manual,» Oracle, 3 Marzo 2026. [En línea]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Último acceso: 6 Abril 2026].
- [94 MongoDB, «Documentación de MongoDB,» MongoDB, Inc., [En línea]. Available: <https://www.mongodb.com/es/docs/>. [Último acceso: 6 Abril 2026].
- [95 MariaDB, «Licensing FAQ,» Diciembre 2025. [En línea]. Available: <https://mariadb.com/docs/general-resources/community/community/faq/licensing-questions/licensing-faq>. [Último acceso: 6 Abril 2026].
- [96 V. Rahimian y R. Ramsin, «Designing and agile methodology for mobile software development: a hybrid ethod engineering approach,» *Second International Conference on Research Challenges in Information Science*, 6 Junio 2008.
- [97 M. C. GASCA MANTILLA, L. L. CAMARGO ARIZA y B. MEDINA DELGADO , «Metodología para el desarrollo de aplicaciones móviles,» *Tecnura*, vol. 18, nº 40, pp. 20-35, 2014.

- [98 S. Bhawna, T. Shweta , B. Isha, S. Himanshi y R. Aanchal , «Cross-Platform Empirical Analysis of Mobile Application Development frameworks: Kotlin, React Native and Flutter,» *In Proceedings of the 4th International Conference on Information Management & Machine Intelligence (ICIMMI '22)*, vol. Article 60, pp. 1-6, 2023.
- [99 Jaspr, «Flutter Documentation,» Flutter Docs, 31 Marzo 2026. [En línea]. Available: <https://docs.flutter.dev/>. [Último acceso: 12 Abril 2026].
- [10 Google Inc, «Términos del Servicio Adicionales para Usuarios Finales de Google Maps,» 4 Junio 2025. [En línea]. Available: https://www.google.com/intl/es_ALL/help/terms_maps/. [Último acceso: 15 Abril 2026].
- [10 OpenStreetMaps, «OpenStreetMap Develop,» 2026 Marzo 12. [En línea]. Available: <https://wiki.openstreetmap.org/wiki/Develop>. [Último acceso: 12 Abril 2026].
- [10 Google Inc, «Documentación de Google Maps Platform,» Alphabet Inc, [En línea]. Available: <https://developers.google.com/maps/documentation?hl=es-419>. [Último acceso: 12 Abril 2026].
- [10 Mapbox, «Mapbox Documentation,» [En línea]. Available: <https://docs.mapbox.com/>. [Último acceso: 12 Abril 2026].
- [10 United Nations Convention against Corruption, «A Strategy for Safeguarding against Corruption in Major Public Events,» 2013.
- [10 PROGRAMA DE LAS NACIONES UNIDAS PARA EL DESARROLLO EN MÉXICO , 5] «Protocolo de análisis de riesgos,» 2018, abril.
- [10 UPIICSA, «Estudios de factibilidad,» [En línea]. Available: https://www.sites.upiicsa.ipn.mx/estudiantes/academia_de_informatica/analisis_de_sistemas/docs/PDF/Estudio_Factibilidad.pdf. [Último acceso: 2 Abril 2026].
- [10 onmex, «onmex,» [En línea]. Available: <https://onmex.mx/tecnologia-y-desarrollo/cuanto-gana-un-programador-en-mexico/>. [Último acceso: 13 Abril 2026].
- [10 Amazon, «Amazon.com,» [En línea]. Available: <https://www.amazon.com.mx/Dell-Inspiron-81-Windows-Antivirus-Teclado/dp/B0DPGNMNY1?th=1>. [Último acceso: 13 Abril 2026].
- [10 Amazon, «Amazon,» [En línea]. Available: <https://www.amazon.com.mx/Samsung-SM-A175FZAILTM-Galaxy-A17-Gray-128GB/dp/B0FLM3PHDN/ref>. [Último acceso: 13 Abril 2026].
- [11 CFE, «CFE,» [En línea]. Available: <https://app.cfe.mx/Aplicaciones/CCFE/Tarifas/TarifasCRECasa/Tarifas/TarifaDAC.aspx>. [Último acceso: 13 Abril 2026].

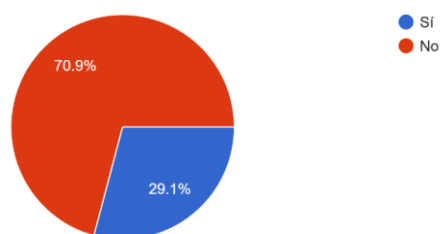
- [11 Excelsior, «Wendy Fernández,» Agosto 2025. [En línea]. Available:
1] <https://www.excelsior.com.mx/estilo-de-vida/cuanto-te-cuesta-el-internet-y-el-streaming-al-mes-en-2025/1733124>. [Último acceso: 13 Abril 2026].
- [11 economipedia, «economipedia,» [En línea]. Available:
2] <https://economipedia.com/definiciones/factibilidad-operativa.html>. [Último acceso: 5 Abril 2026].
- [11 sociedadesconsumo, «sociedadesconsumo.com,» 2026. [En línea]. Available:
3] <https://sociedadesconsumo.com/estudios-de-mercado/estudios-de-factibilidad-que-son/#8-factibilidad-de-tiempo>. [Último acceso: 5 Abril 2026].
- [11 S. Brown, «C4 model,» [En línea]. Available: <https://c4model.com/abstractions/software-system>. [Último acceso: 15 Abril 2026].
4]
- [11 S. Brown, «C4 Model,» [En línea]. Available: <https://c4model.com/abstractions/software-system>. [Último acceso: 15 Abril 2026].
5]
- [11 S. Brown, «C4 model,» [En línea]. Available: <https://c4model.com/abstractions/container>.
6] [Último acceso: 15 Abril 2026].
- [11 visual, «Paradigm,» [En línea]. Available: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/>. [Último acceso: 15 Abril 2026].
7]
- [11 IBM. [En línea]. Available: <https://www.ibm.com/docs/en/rational-soft-arch/9.6.1?topic=diagrams-use-case>. [Último acceso: 15 Abril 2026].
8]
- [11 ECMA-404, The Json Data Interchange Syntax, 2nd ed., Geneva: Standard, 2017.
9]
- [12 J. Erickson, Abril 2024. [En línea]. Available: <https://www.oracle.com/latam/database/what-is-json/>.
0]
- [12 C. TORRES, «SUPERVISION TECNICA Y ADMINISTRATIVA DE LA SECTORIZACION
1] DE LA RED DE DISTRIBUCION DE AGUA POTABLE EN LA DELEGACION GUSTAVO A. MADERO, DF.,» 2008.
- [12 Northware, «Requerimientos en el desarrollo de software y aplicaciones,» Northware, 26 Mayo
2] 2022. [En línea]. Available: <https://www.northware.mx/blog/requerimientos-en-el-desarrollo-de-software-y-aplicaciones/>. [Último acceso: 4 Marzo 2025].
- [12 Wikimedia Commons, 24 Julio 2022. [En línea]. Available: <https://w.wiki/Jpv9>. [Último acceso:
3] 16 Marzo 2026].

Anexos



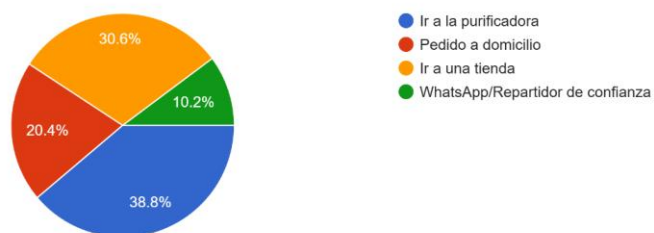
¿Crees que la alcaldía Benito Juárez hay una buena distribución de agua potable?

206 respuestas



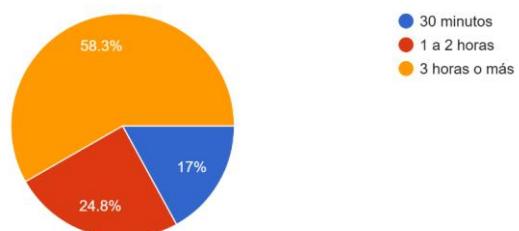
¿Cuál es tu método actual para conseguir agua potable a domicilio?

206 respuestas



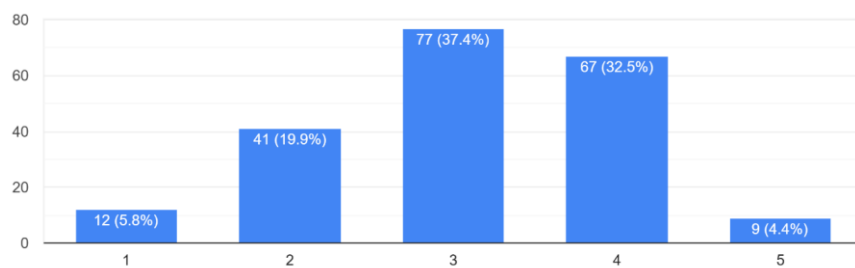
¿Cuánto tiempo se demoran en traerte el agua los repartidores?

206 respuestas



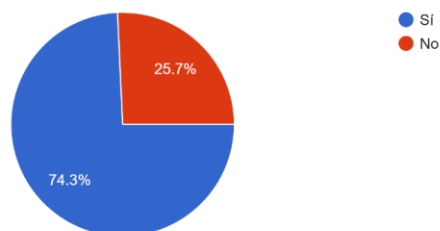
En una escala del 1 al 5 que tan buena ha sido tu experiencia con el servicio de repartición de agua potable

206 respuestas



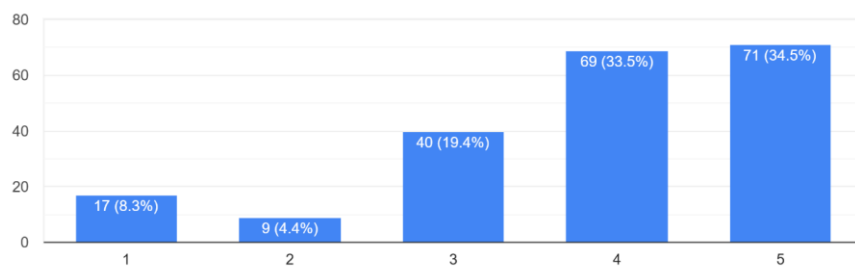
¿Te has quedado sin agua porque el repartidor no pasó o no escuchaste el timbre/claxon?

206 respuestas



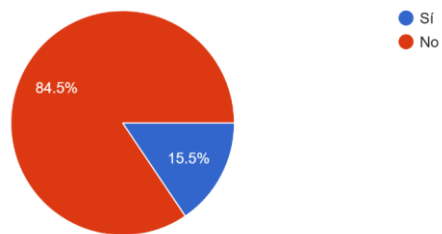
¿Qué tan útil te parece la opción de programar tus pedidos con días de anticipación?

206 respuestas



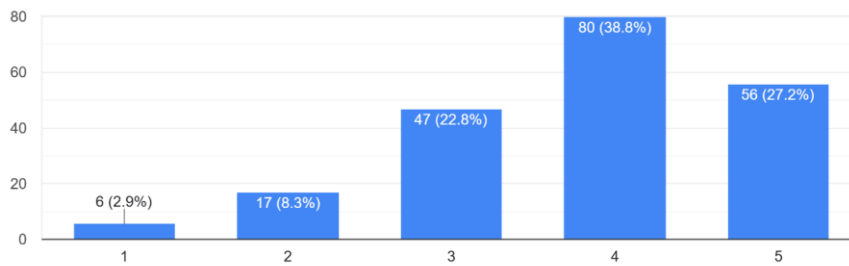
¿Utilizas actualmente otras aplicaciones de servicios a domicilio? (Uber Eats, Rappi, etc.)

206 respuestas



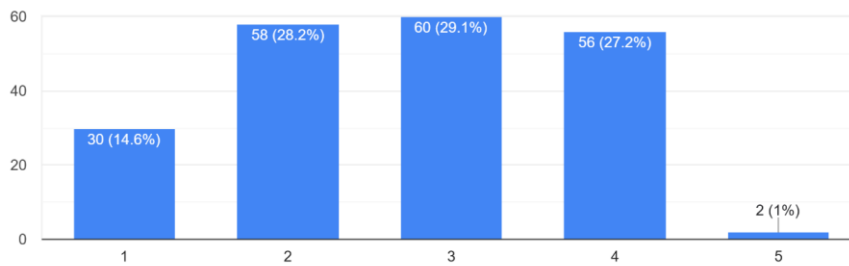
¿Qué tan útil crees que sería una app para pedir garrafones de agua a domicilio?

206 respuestas



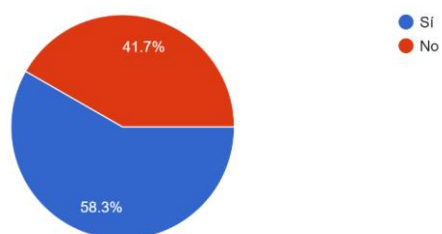
¿Qué tan satisfecho estás con la limpieza y el estado físico de los garrafones que recibes?

206 respuestas



¿Has tenido que cancelar planes o dejar de salir de casa por esperar al repartidor de agua?

206 respuestas



¿Qué tan útil te resultaría recibir una notificación automática cuando el repartidor esté cerca de tu ubicación?

206 respuestas

